

Reporte Técnico: Motor Híbrido de Recomendación de Cross-Selling (Afinidad de Canasta) para TCA - ANA-03

Diego Armando Mijares Ledezma (A01722421)
Daniel Eduardo Arana Bodart (A01741202)
Mauricio Octavio Valencia Gonzalez (A01234397)
David Alejandro Acuña Orozco (A00571187)
Diego Gutiérrez Vargas (A01285421)
Alfonso Elizondo Partida (A01285151)

7 de junio de 2026

Índice

1. Introducción	4
2. Objetivos del Proyecto	5
3. Gobierno de Datos	5
3.1. Business Case	5
3.1.1. Justificación de negocio	5
3.1.2. Viabilidad de la solución	6
3.1.3. KPIs del proyecto	6
3.1.4. Supuestos del modelo financiero	7
3.1.5. Palancas de valor y cálculo del beneficio	8
3.1.6. Costos del equipo — FTE	8
3.1.7. Costos de infraestructura en la nube — Microsoft Azure	9
3.1.8. Análisis financiero — Flujo de efectivo y ROI	12
3.2. Calidad de Datos (Data Wrangling)	13
3.2.1. Fuentes utilizadas	13
3.2.2. Estandarización de tipos y campos clave	14
3.2.3. Reconstrucción de transacciones	14
3.2.4. Filtrado de ventas válidas	15
3.2.5. Tratamiento de tickets de un solo artículo	15
3.2.6. Enriquecimiento con catálogo e inventario	15
3.2.7. Representaciones derivadas para modelado	16
3.2.8. Cobertura final de la base limpia	17
3.3. Protección de Datos y Ética del Modelo (BIAS)	17
3.3.1. Sesgos potenciales en el dataset y en las predicciones	17
3.4. Trazabilidad y Linaje	19
3.4.1. Ciclo de vida completo de los datos	19

4. Análisis Exploratorio de Datos (EDA)	20
4.1. Limpieza de Datos	21
4.2. Visualización de Datos	22
4.3. Estadísticas	23
4.4. Pruebas de Hipótesis	24
4.5. Hallazgos	26
5. Scores para el modelo	27
5.1. Explicación y relevancia	27
5.2. Topological Data Analysis (TDA)	29
5.2.1. Preparación e integración de datos	29
5.2.2. Construcción del espacio de características	29
5.2.3. Reducción de dimensionalidad	30
5.2.4. Construcción del grafo Mapper	30
5.2.5. Estrategias de clusterización	31
5.2.6. Cálculo de afinidad topológica	31
5.2.7. Generación de recomendaciones y salida del componente TDA	32
5.3. Time Series (TS)	32
5.3.1. Entrada del componente temporal	33
5.3.2. Construcción del panel semanal	33
5.3.3. Ingeniería de variables temporales	34
5.3.4. Normalización de señales	35
5.3.5. Construcción del <i>score_TS</i>	35
5.3.6. Integración de inventario	36
5.3.7. Archivos generados	36
5.3.8. Uso dentro del Transformer de fusión temprana	37
5.4. Red Neuronal Recurrente (RNN)	37
5.4.1. Entrada y construcción de canastas	37
5.4.2. Vocabulario y tokenización	38
5.4.3. Generación de secuencias de entrenamiento	38
5.4.4. Arquitectura del modelo	38
5.4.5. Entrenamiento	39
5.4.6. Métricas de evaluación	39
5.4.7. Construcción del <i>score_RNN</i>	39
6. Modelo	40
6.1. Implementación	40
6.2. Refinamiento	42
6.3. Evaluación	44
7. Implementación Técnica del Servicio	45
7.1. Cómputo en la Nube	45
7.1.1. Extracción y carga de datos	45
7.1.2. Infraestructura de despliegue del modelo en la nube (API)	46
7.2. Bases de Datos	48
7.2.1. Filtrado de datos relevantes para el análisis	48
7.2.2. Uso de funciones agregadas y/o joins en las consultas	49
7.3. MLOps	51
7.3.1. Topological Data Analysis (TDA)	51
7.3.2. Series de Tiempo (TS)	53
7.3.3. Redes Neuronales Recurrentes (RNN)	55
7.3.4. Transformer de Fusión Temprana (Transformer)	57

7.4.	Seguimiento de Modelos	60
7.4.1.	Estrategia de MLOps con Kedro y MLflow	60
7.4.2.	Optimización y Persistencia de Hiperparámetros con Optuna	61
8.	Conclusiones y Recomendaciones	61
8.1.	Resultados	61
8.1.1.	Interfaz de usuario y despliegue	62
8.1.2.	Resultados	63
8.1.3.	Ejecución del servicio	64
8.2.	Trabajo a futuro	65
8.3.	Conclusiones finales del proyecto	66
A.	Apéndices	68
A.1.	Liga de Repositorio	68
A.2.	Manual	70
A.2.1.	Flujo general de la solución	70
A.2.2.	Ejecución de los pipelines (Kedro)	70
A.2.3.	Ejecución local del API	71
A.2.4.	Consumo del API	71
A.2.5.	Despliegue en Azure	71
A.2.6.	Ejecución de la aplicación web (Front-end de Canasta)	72

1. Introducción

El presente reporte documenta el desarrollo de una solución de ciencia de datos y analítica avanzada para el caso de uso **ANA-03 — Cross-selling (Afinidad de Canasta)**, propuesto para la plataforma TCA Merksyst. El problema de negocio identificado se relaciona con una labor de venta predominantemente reactiva: el vendedor atiende los productos que el cliente solicita, pero no cuenta con una herramienta sistemática que le permita sugerir artículos complementarios con base en patrones históricos de compra. Como consecuencia, se desaprovechan oportunidades de aumentar la profundidad de canasta, elevar el valor del pedido y transformar el proceso de venta en una interacción más proactiva y asistida por datos.

Desde la perspectiva de negocio, el caso ANA-03 busca responder a la siguiente necesidad: *como vendedor, quiero ver qué productos suelen comprar otros clientes con los artículos actuales para aumentar el valor del pedido*. Esta necesidad se traduce en un problema de recomendación de canasta: dado uno o varios productos presentes en una compra en curso, el sistema debe estimar qué otros artículos tienen mayor probabilidad de ser agregados como complemento. Las acciones esperadas a partir de esta solución incluyen sugerir productos complementarios, crear combos o *bundles*, y diseñar campañas comerciales basadas en afinidades reales entre SKUs.

La solución desarrollada parte del enfoque tradicional de *Market Basket Analysis*, cuyo objetivo es descubrir relaciones entre productos mediante patrones de co-ocurrencia y métricas como *support*, *confidence* y *lift*. Sin embargo, el proyecto amplía este planteamiento mediante una arquitectura híbrida que incorpora señales adicionales de valor para el negocio. En particular, se integran tres componentes especializados: Análisis Topológico de Datos (TDA), Series de Tiempo y una Red Neuronal Recurrente (RNN). Estas señales se combinan posteriormente dentro de un modelo Transformer de fusión temprana, diseñado para generar un ranking de productos recomendados a partir de una canasta parcial.

El componente TDA aporta una lectura estructural del catálogo, permitiendo identificar productos cercanos o relacionados a partir de atributos como línea, familia y subfamilia. El componente de Series de Tiempo permite incorporar información temporal sobre tendencia, recencia, volumen, estacionalidad e inventario disponible. Por su parte, la RNN captura patrones secuenciales y de aparición conjunta observados en canastas históricas. La integración de estas tres señales busca que el recomendador no dependa únicamente de popularidad o frecuencia de compra conjunta, sino que considere estructura del catálogo, comportamiento temporal y patrones históricos de consumo.

El proyecto se desarrolló a partir de datos transaccionales reales del ERP TCA Merksyst, principalmente tablas de encabezado y detalle de ventas, catálogo de artículos, precios e inventario. A partir de estas fuentes se reconstruyeron tickets de venta, se depuraron registros no válidos, se generaron representaciones analíticas para distintos modelos y se construyeron los artefactos necesarios para la inferencia. El modelo final fue empaquetado como una API REST con FastAPI, contenedorizado con Docker y desplegado en Microsoft Azure, de forma que pudiera ser consumido desde una aplicación web funcional llamada **Canasta**.

Este reporte técnico describe el ciclo completo de la solución. Primero, se presenta el gobierno de datos, incluyendo el caso de negocio, la calidad de datos, las consideraciones éticas y la trazabilidad de los artefactos. Después, se documenta el análisis exploratorio de datos realizado para validar volumen, cobertura, profundidad de canasta, comportamiento temporal y disponibilidad. Posteriormente, se explican los *scores* generados por TDA, Series de Tiempo y RNN, así como el modelo Transformer utilizado para fusionar dichas señales. Finalmente, se detalla la implementación técnica del servicio, la integración con la aplicación web, los resultados obtenidos, el trabajo a futuro y las conclusiones finales del proyecto.

2. Objetivos del Proyecto

El objetivo general del proyecto es **desarrollar un sistema de recomendación de productos complementarios para el caso ANA-03 Cross-selling, capaz de sugerir artículos relevantes a partir de una canasta parcial de compra, utilizando datos transaccionales reales del ERP TCA Merksyst y entregando la solución como un MVP funcional integrado a una API y una aplicación web.** La finalidad de esta solución es apoyar una labor de venta más proactiva, incrementar el valor potencial del pedido y habilitar decisiones comerciales basadas en afinidades observadas entre productos.

Para cumplir este objetivo general, se definieron los siguientes objetivos específicos:

- Reconstruir y depurar la base transaccional del ERP TCA Merksyst a partir de tablas de encabezado y detalle de venta, conservando únicamente tickets y líneas válidas para el análisis de canasta.
- Analizar la estructura de los datos mediante EDA, evaluando volumen transaccional, cobertura de SKUs, profundidad de canasta, distribución de productos, comportamiento temporal e información de inventario.
- Generar señales complementarias por producto que representen distintas dimensiones del problema: afinidad estructural del catálogo mediante TDA, oportunidad temporal mediante Series de Tiempo y patrones secuenciales de compra mediante RNN.
- Diseñar, entrenar y refinar un modelo Transformer de fusión temprana que integre la identidad de los productos con los *scores* de TDA, Series de Tiempo y RNN para producir un ranking Top-K de recomendaciones.
- Evaluar el desempeño del modelo mediante métricas de ranking, principalmente MRR@10 y Hit@K, considerando que el problema consiste en ordenar candidatos relevantes y no en realizar una clasificación binaria tradicional.
- Implementar el modelo como un servicio de inferencia accesible mediante API REST, contenedorizado con Docker y desplegado en Microsoft Azure para facilitar su consumo desde sistemas externos.
- Integrar el servicio de recomendación con una aplicación web funcional que permita demostrar el flujo de uso del recomendador en un contexto de comercio electrónico y operación comercial.
- Documentar la trazabilidad de datos, modelos, artefactos y despliegue, asegurando que el proyecto pueda ser entendido, reproducido y extendido por futuros equipos técnicos o por la empresa.
- Identificar limitaciones actuales del MVP y proponer líneas de trabajo futuro orientadas a ampliar la cobertura de SKUs, mejorar la compatibilidad con servicios empresariales, automatizar actualizaciones y validar el impacto comercial en operación real.

3. Gobierno de Datos

3.1. Business Case

3.1.1. Justificación de negocio

TCA Software Solutions, bajo la marca TCA Merksyst, provee un sistema ERP para el sector retail de abarrotes y tiendas de conveniencia en México. Su plataforma atiende a clientes que

operan hasta siete formatos de venta distintos; el 50 % de quienes manejan más de dos formatos los desarrollaron a partir de su implementación, lo que evidencia el nivel de adopción y madurez operativa del sistema en el mercado.

El análisis del comportamiento de venta reveló que los tickets generados en el punto de venta contienen en promedio 5.19 artículos por transacción —dato real confirmado por TCA Merksyst—, con un alto potencial de crecimiento mediante estrategias proactivas de cross-selling. Aunque este valor supera el benchmark del sector (3.5–4.0 artículos por ticket), el volumen de aproximadamente 30,000 transacciones mensuales por tienda representativa amplifica de forma significativa el impacto potencial de cualquier incremento marginal en el número de artículos por pedido. El problema de fondo es estructural: el modelo de venta es reactivo. El vendedor atiende únicamente lo que el cliente solicita de forma explícita y no dispone de herramientas que le permitan sugerir productos complementarios de manera sistemática y basada en información histórica de consumo.

Tabla 1: Elementos del caso de uso ANA-03.

Elemento	Descripción
Problemática raíz	Venta reactiva: el vendedor no dispone de herramientas para sugerir complementos de forma proactiva y basada en datos históricos
Evidencia	5.19 artículos/ticket (real); benchmark sectorial ya superado (3.5–4.0); oportunidad de llevar a 6.0+ mediante cross-selling sistemático
Oportunidad	Cada artículo adicional representa ~\$48 MXN adicionales; con 30,000 transacciones mensuales reales, el impacto acumulado es sustancial
Solución propuesta	Motor de recomendación ANA-03: dado el contenido del ticket actual, sugiere el top-10 de productos más probables a agregar
<i>User Story</i>	“Como vendedor, quiero ver qué productos suelen comprar otros clientes con los artículos de la canasta actual, para sugerir complementos de forma proactiva”
Integración	API REST — <code>POST /recommend</code> consumible desde cualquier pantalla del ERP TCA Merksyst sin modificar su arquitectura
Infraestructura	Microsoft Azure Container Apps (Docker, FastAPI, PyTorch 2.11.0 CPU)
Datos de entrenamiento	688,322 transacciones reales del ERP; ~3,000 SKUs activos en vocabulario final

3.1.2. Viabilidad de la solución

La **viabilidad técnica** queda demostrada porque los datos transaccionales necesarios para entrenar el modelo ya existen en el ERP TCA Merksyst en formato Parquet —encabezados y detalle de ventas (`pdvhdr`, `pdvdet`), catálogo de productos (`inviar`) e inventario (`invars`)— sin requerir instrumentación nueva ni cambios en los procesos operativos del socio formador.

La **viabilidad de integración** está garantizada porque la solución fue construida como una API REST completamente independiente del ERP. Cualquier pantalla del sistema TCA Merksyst que tenga acceso a los códigos de producto del ticket en curso puede consultar el motor y recibir las recomendaciones mediante una llamada HTTP estándar, sin modificar la arquitectura interna del sistema.

La **viabilidad de ejecución** está respaldada por un equipo con competencias en modelado de aprendizaje profundo, ingeniería de datos, MLOps y despliegue en nube.

3.1.3. KPIs del proyecto

Los indicadores clave de desempeño definidos para ANA-03 cubren tanto el impacto comercial esperado en el punto de venta como la calidad técnica del servicio desplegado. Los valores actuales

corresponden a datos reales confirmados por TCA Merksyst para artículos por ticket y volumen de transacciones; los demás son metas operativas proyectadas.

Tabla 2: KPIs del proyecto ANA-03.

KPI	¿Qué mide?	Actual	Meta 12m	Meta 24m	Benchmark MX
Artículos / ticket	Promedio de productos distintos por transacción	5.19	6.0	6.5	3.5–4.0
Ticket promedio (MXN)	Valor monetario promedio por transacción	\$249	\$292	\$320	\$350–\$450
Tasa de adopción del motor	% de veces que el vendedor ofrece la sugerencia	0 %	35 %	60 %	40–65 %
Tasa de conversión cross-sell	% de sugerencias que resultan en compra	N/A	15 %	31 %	15–30 %
Cobertura del catálogo	% de SKUs activos con al menos una recomendación	0 %	55 %	75 %	—
MRR (<i>Mean Reciprocal Rank</i>)	Posición del producto correcto en el top-10	—	Optimizado (Optuna)	Reentrenamiento	—
Disponibilidad del API	SLA del servicio	—	99 %	99.5 %	99 %+
Latencia (<i>cold start</i>)	Tiempo de primera respuesta tras inactividad	—	<20 s	<5 s	<3 s

Datos actuales confirmados por TCA Merksyst: 5.19 art/ticket, 30,000 tx/mes. Benchmark del sector ya superado; la meta es consolidar la ventaja. La conversión cross-sell real deberá validarse en producción mediante pruebas A/B; para el modelo financiero se usa el hit rate offline del modelo optimizado como proxy operativo inicial. Benchmarks: ANTAD, Nielsen Retail MX 2024.

3.1.4. Supuestos del modelo financiero

Los cálculos financieros incorporan datos reales confirmados por TCA Merksyst, complementados con supuestos operativos del sector retail de abarrotes en México. La Tabla 3 presenta los parámetros base del modelo.

Tabla 3: Supuestos del modelo financiero ANA-03.

Parámetro	Valor y justificación
Ticket promedio actual	\$249 MXN — calculado a partir de datos reales: 5.19 art/ticket × \$48 MXN precio promedio del catálogo
Transacciones por día	~1,000 tx/día — dato real: 30,000 tx/mes ÷ 30 días
Transacciones por mes	30,000 tx/mes — dato real confirmado por TCA Merksyst
Conversión potencial cross-sell (Año 2+)	31 % — proxy operativo basado en el hit rate offline del modelo optimizado; requiere validación futura con pruebas A/B
Artículos adicionales por ticket	0.81 artículos — diferencia entre meta (6.0) y valor actual (5.19)
Precio promedio artículo adicional	~\$48 MXN — calculado sobre el catálogo activo
Ramp-up Año 1 / Año 2 / Año 3+	40 % / 75 % / 100 % — curva de adopción y maduración operativa
WACC	14.0 % — tasa de descuento estándar para proyectos tecnológicos en México
Tipo de cambio USD/MXN	\$17.50 MXN — referencia para costos de Azure (junio 2025)

3.1.5. Palancas de valor y cálculo del beneficio

El modelo financiero contabiliza el beneficio a través de una única palanca para evitar doble conteo: el incremento en artículos por ticket impulsado por el cross-selling. El incremento en el valor monetario del ticket (Palanca 1) es consecuencia directa de agregar artículos y describe el mismo evento, por lo que se contabiliza una sola vez a través de la Palanca 2.

Tabla 4: Cálculo de la Palanca 2: artículos adicionales por ticket.

Parámetro	Valor
Artículos actuales por ticket (real)	5.19
Meta artículos por ticket (12 meses)	6.0
Artículos adicionales por ticket impactado	0.81 artículos
Precio promedio del artículo adicional	~\$48 MXN
Transacciones mensuales por tienda (real)	30,000
Conversión potencial cross-sell (proxy Hit@10 offline)	31 %
Tickets impactados por mes	~9,300
Artículos adicionales vendidos por mes	~7,533 (9,300 × 0.81)
Ingreso adicional por mes	\$361,584 MXN (7,533 × \$48)

Palanca 2 — Artículos adicionales por ticket. El beneficio anual en operación plena (100 % ramp-up) se calcula como:

$$\text{Beneficio mensual} = 30,000 \times 31 \% \times 0.81 \times \$48 = \$361,584 \text{ MXN/mes}$$

$$\text{Beneficio anual (plena operación)} = \$361,584 \times 12 = \$4,339,008 \text{ MXN/tienda}$$

El uso del 31 % corresponde a una aproximación financiera basada en el desempeño offline del modelo. En operación real, esta proporción deberá ajustarse con mediciones de adopción del vendedor, aceptación del cliente y conversión observada mediante pruebas A/B.

3.1.6. Costos del equipo — FTE

El equipo de desarrollo está conformado por seis IDMs cubriendo el rol de Data Scientist durante los doce meses del proyecto, más cuatro roles de soporte técnico con asignación parcial

según la fase del proyecto. Los costos se calculan sobre 160 horas mensuales por persona con referencia en OCCMundial, LinkedIn Salary y Computrabajo México (2024–2025).

Tabla 5: Costos FTE del equipo ANA-03.

Rol	Cant.	Sal. mín	Sal. máx	Mes.	Total mín (MXN)	Total máx (MXN)
Data Scientist (IDM)	6	\$35,000	\$55,000	12	\$2,520,000	\$3,960,000
Data Engineer	1	\$38,000	\$60,000	6	\$228,000	\$360,000
Database Administrator	1	\$35,000	\$50,000	3	\$105,000	\$150,000
MLOps Engineer	1	\$45,000	\$70,000	8	\$360,000	\$560,000
Project Manager	1	\$40,000	\$65,000	12	\$480,000	\$780,000
Total FTE					\$3,693,000	\$5,810,000

Tabla 6: Escenarios de inversión FTE.

Escenario	Costo total (MXN)	Uso en el Business Case
Mínimo (salarios base)	\$3,693,000	Inversión real Año 0 en el flujo de efectivo
Base ponderado académico	\$4,747,840	Indicador de portada del Business Case
Máximo (salarios de mercado)	\$5,810,000	Referencia superior

3.1.7. Costos de infraestructura en la nube — Microsoft Azure

La solución se despliega sobre tres servicios de Microsoft Azure. Los precios son referencia pública de Azure Pricing Calculator (junio 2025) con tipo de cambio de \$17.50 MXN/USD.

Azure Container Registry (ACR) — nivel Basic.

Almacena la imagen `reco-api` (~2 GB) que contiene el modelo entrenado, la API FastAPI y todas sus dependencias. Suficiente para un repositorio con una imagen activa y despliegues periódicos.

Tabla 7: Costos del Azure Container Registry (ACR).

Concepto	Precio unitario	Uso estimado	Costo/mes USD	Total/año USD	Total/año MXN
ACR Basic — almacenamiento	\$0.167/GB/mes	~2 GB	\$0.33	~\$4	~\$70
ACR Basic — transferencia	\$0.10/GB	~5 GB/mes	\$0.50	~\$6	~\$105
Subtotal ACR			\$0.83	~\$10	~\$175

Azure Container Apps — ejecución de la API REST.

Servicio donde corre el motor de recomendación. Facturación por consumo: vCPU-segundos y GB-segundos de memoria activa. Escala a cero en inactividad, eliminando costos durante periodos sin tráfico. Configuración del proyecto: 0.5 vCPU, 1 GB RAM por instancia.

Tabla 8: Costos de Azure Container Apps.

Concepto	Precio unitario	Uso estimado	Costo/mes USD	Total/año USD	Total/año MXN
vCPU activa	\$0.000024 / vCPU-s	~72,000 s/mes	\$1.73	~\$21	~\$363
Memoria activa	\$0.000003 / GB-s	~144,000 GB-s	\$0.43	~\$5	~\$90
Solicitudes	\$0.40/millón	<1M req/mes	\$0.00	\$0.00	\$0
Subtotal		Escala a cero	\$2.16	~\$26	~\$455

Tabla 9: Escenarios de despliegue en Azure Container Apps.

Escenario	Descripción	Costo/año USD	Costo/año MXN
Académico (Azure for Students)	Crédito \$100 USD/año — cubre fase académica completa	\$0	\$0
Producción básica (escala a cero)	Cold start 10–20 s en primera solicitud	~\$26	~\$455
Producción estándar (instancia fija)	1 instancia permanente, sin cold start	~\$500	~\$8,750
Alta disponibilidad (2 instancias)	Redundancia y balanceo de carga	~\$1,000	~\$17,500

Azure Monitor / Log Analytics — logs y diagnóstico.

Captura los logs de ejecución del contenedor para diagnóstico de errores, trazabilidad de solicitudes y monitoreo del SLA en producción.

Tabla 10: Costos de Azure Monitor / Log Analytics.

Concepto	Precio unitario	Uso estimado	Costo/mes MXN	Total/año MXN
Log Analytics — ingesta	\$2.30 USD/GB	~3 GB/mes	~\$120	~\$1,440
Retención 0–31 días	Incluido	—	\$0	\$0
Retención extendida 31–90 días	\$0.10 USD/GB/-mes	~3 GB/mes	~\$5.25	~\$63
Alertas de disponibilidad	\$0.10 USD/regla/mes	2 reglas activas	~\$3.50	~\$42
Subtotal Azure Monitor			~\$875	~\$10,500

Otros costos de infraestructura.

Tabla 11: Otros costos de infraestructura Azure.

Concepto	Función	Frecuencia	Total/año (MXN)
Transferencia de red saliente	Respuestas JSON del API a clientes externos (\$0.087 USD/GB, ~5 GB/mes)	Mensual	~\$91
Reentrenamiento del modelo	Cómputo CPU en Azure (~\$17.50 USD/ejecución)	2 veces/año	~\$613
Dominio personalizado (Azure DNS)	URL branded para integración TCA Merksyst (\$0.90 USD/-mes)	Mensual	~\$189
Certificado SSL	HTTPS automático con Let's Encrypt — incluido en Container Apps	—	\$0

Resumen total de costos Azure anuales.

Tabla 12: Resumen de costos Azure anuales — ANA-03.

Servicio	Función principal	Costo/año (USD)	Costo/año (MXN)
Azure Container Registry	Almacenamiento imagen Docker	~\$10	~\$175
Azure Container Apps	Ejecución API REST (escala a cero)	~\$26	~\$455
Azure Monitor / Log Analytics	Logs, diagnóstico y alertas	~\$600	~\$10,500
Transferencia de red saliente	Respuestas JSON del API	~\$5	~\$91
Reentrenamiento (2×/año)	Cómputo CPU Azure cada 6 meses	~\$35	~\$613
Dominio personalizado	URL branded para TCA Merksyst	~\$11	~\$189
Subtotal base estimado		~\$687	~\$12,023
Total con margen +85 %	Cubre picos de tráfico, nuevos deploys y pruebas	~\$1,371	~\$22,243

El margen de seguridad del 85 % sobre el costo base cubre variaciones de tráfico en temporada alta, pruebas de regresión tras reentrenamiento, actualizaciones de imagen y posibles incrementos de precios de Azure. Es el valor utilizado en el flujo de efectivo (\$22,243 MXN/año).

3.1.8. Análisis financiero — Flujo de efectivo y ROI

Tabla 13: Indicadores financieros del Business Case ANA-03.

Indicador	Valor	Interpretación
VPN (Valor Presente Neto)	\$5,497,433 MXN	Valor neto generado por encima del costo de capital (WACC 14 %)
TIR (Tasa Interna de Retorno)	64.1 %	Muy superior al WACC; proyecto altamente rentable
Payback	20 meses	La inversión se recupera en el mes 20 de operación (inicio del Año 2)
WACC	14.0 %	Tasa de descuento mínima exigida para proyectos tecnológicos en México
Inversión inicial (Año 0)	\$4,747,840 MXN	Escenario base ponderado académico (indicador de portada)
Inversión real Año 0 (FTE)	\$3,693,000 MXN	Suma de salarios mínimos por meses activos; usado en el flujo de efectivo
Beneficio total a 4 años	\$13,667,875 MXN	Suma de beneficios en operación plena para una tienda representativa

Tabla 14: Flujo de efectivo detallado — 4 años (MXN).

Concepto	Año 0	Año 1 (40 %)	Año 2 (75 %)	Año 3 (100 %)	Año 4 (100 %)
<i>(+) Beneficios</i>					
Beneficios anuales	—	\$1,735,603	\$3,254,256	\$4,339,008	\$4,339,008
<i>(-) Costos</i>					
FTE desarrollo (total)	(\$3,693,000)	—	—	—	—
• Data Scientists ×6	(\$2,520,000)				
• Data Engineer (6m)	(\$228,000)				
• DBA (3m)	(\$105,000)				
• MLOps Engineer (8m)	(\$360,000)				
• Project Manager (12m)	(\$480,000)				
Gastos operativos (total)	—	(\$114,568)	(\$114,568)	(\$114,568)	(\$114,568)
• Azure Container Registry		(\$175)	(\$175)	(\$175)	(\$175)
• Azure Container Apps		(\$455)	(\$455)	(\$455)	(\$455)
• Azure Monitor / Logs		(\$10,500)	(\$10,500)	(\$10,500)	(\$10,500)
• Transferencia de red		(\$91)	(\$91)	(\$91)	(\$91)
• Reentrenamiento modelo		(\$613)	(\$613)	(\$613)	(\$613)
• Dominio personalizado		(\$189)	(\$189)	(\$189)	(\$189)
• Margen seguridad Azure		(\$10,220)	(\$10,220)	(\$10,220)	(\$10,220)
<i>(+85 %)</i>					
• FTE mantenimiento (2.5 %)		(\$92,325)	(\$92,325)	(\$92,325)	(\$92,325)
Flujo neto del periodo	(\$3,693,000)	\$1,621,035	\$3,139,688	\$4,224,440	\$4,224,440
Flujo acumulado	(\$3,693,000)	(\$2,071,965)	\$1,067,723	\$5,292,163	\$9,516,603

Ramp-up: Año 1 = 40 % | Año 2 = 75 % | Años 3-4 = 100 %. Gastos operativos = infraestructura Azure \$22,243 MXN/año (base \$12,023 + margen 85 %) + FTE mantenimiento 2.5 % de la inversión base (\$92,325 MXN/año). Punto de equilibrio: mes 20, inicio del Año 2 operativo. El análisis se estima para una tienda representativa con 30,000 transacciones mensuales; su extrapolación a múltiples tiendas dependerá del volumen transaccional, adopción operativa, disponibilidad de inventario y comportamiento de compra de cada cliente.

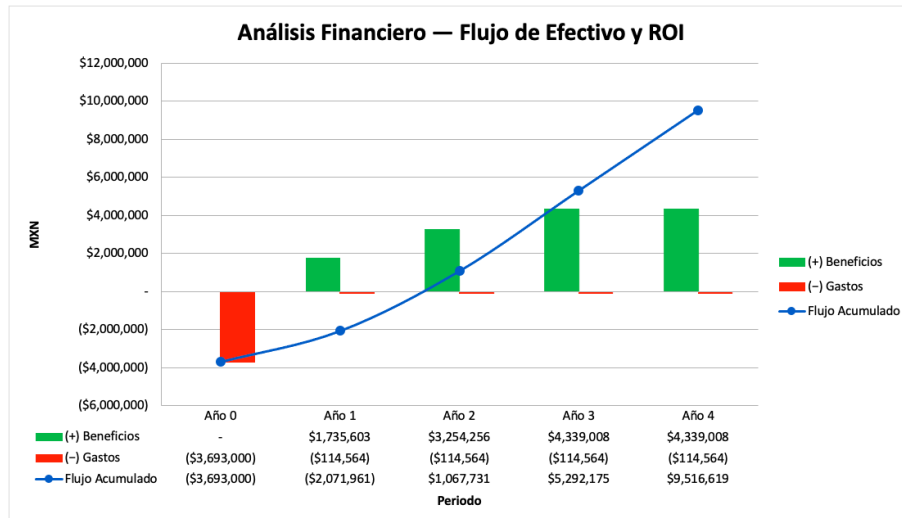


Figura 1: Flujo de efectivo anual ANA-03 — Beneficios (verde) vs. Costos (rojo) y flujo acumulado. El punto de equilibrio se alcanza en el mes 20.

3.2. Calidad de Datos (Data Wrangling)

El proceso de *data wrangling* tuvo como objetivo transformar las tablas operativas del ERP TCA Merksyst en una base analítica consistente y reutilizable para los distintos componentes del sistema de recomendación. Dado que los datos originales provienen de un sistema transaccional, su estructura no está diseñada directamente para modelado predictivo, sino para registrar operaciones de venta, detalle de productos, catálogo e inventarios. Por ello, antes de construir las señales de TDA, Series de Tiempo, RNN y el modelo Transformer de fusión temprana, fue necesario depurar, unir, estandarizar y reorganizar la información relevante.

En términos generales, el *wrangling* no se diseñó para un solo modelo, sino como una capa común de preparación de datos. A partir de la misma base limpia se generaron distintas representaciones analíticas: una base transaccional a nivel ticket-producto, una representación de canastas por ticket, una tabla de productos enriquecida con atributos de catálogo, un panel semanal por SKU y archivos de *scores* por producto. Cada componente del sistema consumió la representación más adecuada para su objetivo técnico: RNN utilizó canastas y secuencias de productos; TDA utilizó atributos estructurales del catálogo; Series de Tiempo utilizó agregaciones semanales por SKU; y el Transformer integró los *scores* generados por los módulos anteriores.

3.2.1. Fuentes utilizadas

Las principales fuentes de información utilizadas fueron las tablas de ventas de punto de venta, catálogo de productos e inventario. La base transaccional se construyó principalmente a partir de las tablas `pdvhdr` y `pdvdet`. La primera contiene el encabezado de la transacción, es decir, información general del ticket como sucursal, fecha, caja, número de transacción, tipo de operación, estatus y totales. La segunda contiene el detalle de productos vendidos dentro de cada ticket, incluyendo clave del artículo, descripción, cantidad, importes y estatus de la línea.

Adicionalmente, se utilizó la tabla `inviar` como catálogo maestro de artículos. Esta tabla permitió enriquecer los SKUs con atributos descriptivos y categóricos, como descripción, línea, sublínea, familia, subfamilia y marca. Finalmente, la tabla `invars` se utilizó para incorporar información de existencias, de manera que las recomendaciones pudieran considerar disponibilidad operativa.

Tabla	Uso principal	Campos relevantes
<code>pdvhdr</code>	Encabezado de tickets de punto de venta. Permite identificar ventas válidas, fechas de operación y llaves naturales de transacción.	<code>Suc</code> , <code>FecOpe</code> , <code>Caja</code> , <code>NumTra</code> , <code>Tipo</code> , <code>status</code> , <code>cliente</code> , <code>subtotal</code>
<code>pdvdet</code>	Detalle de productos vendidos dentro de cada ticket. Es la fuente principal para reconstruir canastas y demanda por SKU.	<code>cve_art</code> , <code>des_art</code> , <code>cantidad</code> , <code>subtotal</code> , <code>status</code> , <code>NumProm</code> , <code>TpoProm</code>
<code>inviar</code>	Catálogo maestro de productos. Permite enriquecer cada SKU con atributos descriptivos y jerárquicos.	<code>art</code> , <code>des1</code> , <code>lin</code> , <code>s_lin</code> , <code>fam</code> , <code>s_fam</code> , <code>marca</code> , <code>status</code>
<code>invars</code>	Existencias por artículo y almacén. Permite estimar disponibilidad operativa de los productos recomendados.	<code>art</code> , existencias agregadas, pedidos

Tabla 15: Fuentes principales utilizadas para la preparación de datos del modelo.

3.2.2. Estandarización de tipos y campos clave

Una de las primeras decisiones de calidad de datos fue tratar las claves de producto y las llaves transaccionales como texto. Esto es importante porque algunos SKUs pueden contener ceros a la izquierda o formatos alfanuméricos; si se interpretan como variables numéricas, pueden perder información y romper los cruces entre tablas. Por esta razón, campos como `cve_art`, `art`, `Suc`, `Caja` y `NumTra` se manejaron como cadenas de texto.

También se aplicó limpieza de espacios en blanco mediante operaciones equivalentes a `TRIM()` sobre campos clave y variables categóricas. Esta estandarización reduce el riesgo de generar llaves distintas para valores que conceptualmente son iguales, pero que en el origen pueden venir con espacios antes o después del texto.

La fecha operativa `FecOpe` se convirtió a formato de fecha para permitir el análisis temporal, la construcción de ventanas, la agregación semanal y la trazabilidad de los tickets. Esta conversión fue necesaria para el componente de Series de Tiempo, pero también aporta consistencia general al flujo de datos, ya que permite ordenar transacciones, separar periodos de entrenamiento y validación, y documentar el rango histórico utilizado.

3.2.3. Reconstrucción de transacciones

El primer paso analítico consistió en reconstruir la transacción completa a partir del encabezado y el detalle de venta. Para ello, las tablas `pdvhdr` y `pdvdet` se unieron mediante la llave natural del ticket:

$$\text{Suc} + \text{FecOpe} + \text{Caja} + \text{NumTra}.$$

A partir de estos campos se construyó un identificador único de ticket, denominado `ticket_id`. Esta llave permite agrupar todos los productos vendidos dentro de una misma transacción y constituye la base para los componentes de canasta, secuencia, co-ocurrencia y series de tiempo.

La reconstrucción de tickets es un paso central del proyecto porque el problema de negocio no se plantea a nivel de línea individual de venta, sino a nivel de canasta. En otras palabras, el modelo necesita conocer qué productos aparecen juntos dentro del mismo ticket para poder aprender patrones de recomendación. Sin esta reconstrucción, cada producto vendido quedaría aislado y no sería posible modelar afinidad de canasta, generación de secuencias ni relaciones entre productos.

3.2.4. Filtrado de ventas válidas

Después de unir encabezado y detalle, se filtraron únicamente registros que representaran ventas válidas. El objetivo de este filtro fue evitar que cancelaciones, devoluciones, líneas inactivas o registros sin producto contaminaran las señales del modelo.

Los principales filtros aplicados fueron:

- conservar únicamente operaciones de venta mediante `pdvhdr.Tipo = '00'`;
- conservar tickets con estatus válido, principalmente TI y TIF;
- conservar únicamente líneas activas del detalle mediante `pdvdet.status = '01'`;
- eliminar registros con cantidad menor o igual a cero;
- eliminar líneas sin clave de producto válida en `cve_art`;
- estandarizar campos clave para evitar inconsistencias por espacios o formatos.

Estos filtros se aplicaron antes de cualquier cálculo de *scores*, de modo que las señales posteriores se construyeran sobre ventas reales y comparables. La discusión exploratoria sobre la distribución de tickets, SKUs, productos por transacción y posibles valores atípicos se aborda en la Sección 4.

3.2.5. Tratamiento de tickets de un solo artículo

Los tickets de un solo artículo no fueron eliminados de la base general limpia. Esta decisión depende del tipo de análisis posterior. Para modelos de co-ocurrencia, reglas de asociación o aprendizaje secuencial, un ticket con un único SKU no genera relaciones directas entre productos y puede no aportar información suficiente para entrenar señales de canasta. Sin embargo, para otros componentes del sistema sí contiene información útil.

En particular, para el componente de Series de Tiempo, un ticket de un solo artículo representa demanda real del producto. Eliminarlo generaría una subestimación de ventas, volumen, recencia y tendencia. De forma similar, para el análisis de cobertura de catálogo y disponibilidad, estos tickets siguen indicando que el SKU tuvo actividad comercial. Por esta razón, el dataset limpio conserva todas las ventas válidas, y cada módulo del pipeline decide posteriormente si requiere filtros adicionales de acuerdo con su objetivo técnico.

3.2.6. Enriquecimiento con catálogo e inventario

Una vez construida la base limpia de ventas, se enriquecieron los registros con información del catálogo maestro de productos. El cruce con `inviar` permitió asociar cada SKU con atributos descriptivos como nombre del producto, línea, sublínea, familia, subfamilia y marca. Este enriquecimiento cumple tres funciones: mejora la interpretabilidad de los resultados, permite que TDA utilice atributos estructurales del catálogo para representar relaciones entre productos y facilita la presentación final de recomendaciones con descripciones comprensibles para usuarios de negocio.

También se integró información de inventario proveniente de `invars`. En esta etapa, las existencias se agregaron por producto para construir variables como `existencia_total` y `has_stock`. Aunque el inventario no forma parte necesariamente de todos los modelos, sí funciona como una restricción de negocio: una recomendación puede ser estadísticamente razonable, pero no debe mostrarse en producción si el producto no tiene disponibilidad.

3.2.7. Representaciones derivadas para modelado

A partir de la base limpia y enriquecida se generaron distintas estructuras para los módulos del sistema. Esta separación fue necesaria porque cada técnica requiere una granularidad distinta de los datos: RNN trabaja con canastas y secuencias, TDA con atributos de producto, Series de Tiempo con ventas agregadas por semana y el Transformer con *scores* consolidados por SKU.

Representación derivada	Construcción	Uso dentro del sistema
Base ticket-producto	Unión de encabezado y detalle de venta mediante Suc , FecOpe , Caja y NumTra . Cada fila representa un producto vendido dentro de un ticket válido.	Base común para EDA, reconstrucción de canastas, cálculo de demanda por SKU y generación de señales.
Canastas por ticket	Agrupación de los SKUs vendidos dentro de cada ticket_id .	Insumo principal para RNN y Transformer, ya que permite aprender patrones de co-compra y secuencia.
Tabla producto-catálogo	Cruce de SKUs vendidos con inviar para incorporar descripción, línea, sublínea, familia, subfamilia y marca.	Insumo para TDA e interpretabilidad de recomendaciones, al representar relaciones estructurales entre productos.
Panel semanal por SKU	Agregación de ventas por producto y semana, conservando semanas sin venta como ceros cuando fue necesario.	Insumo del componente de Series de Tiempo para calcular tendencia, recencia, volumen, estacionalidad y actividad reciente.
Inventario agregado por SKU	Integración de existencias desde invars , agregadas a nivel producto.	Filtro o restricción de negocio para evitar recomendar productos sin disponibilidad operativa.
Scores por producto	Consolidación de señales RNN, TDA y TS a nivel SKU.	Insumo del Transformer de fusión temprana, que combina identidad de producto con señales externas.

Tabla 16: Representaciones derivadas del proceso de *data wrangling* y uso por componente del sistema.

Para los componentes de canasta, se agruparon productos por **ticket_id**, creando listas de SKUs que representan las compras realizadas por los clientes. Esta estructura permite generar ejemplos de entrenamiento donde una canasta parcial se utiliza para predecir un producto siguiente o complementario.

Para el componente de Series de Tiempo, la base a nivel línea se agregó por producto y semana, generando el archivo **weekly_series_ana03.csv**. Posteriormente, este panel temporal se resumió a nivel producto para construir **ts_scores_ana03.csv**. Los detalles técnicos de estas transformaciones, así como la construcción del **score_TS**, se documentan en la Sección 5.3.

Para el componente RNN, las canastas limpias se utilizaron para construir secuencias de entrenamiento a partir de productos observados dentro de un mismo ticket. Esta representación permite aprender patrones de aparición conjunta y orden relativo de productos en canastas históricas. Los detalles técnicos de tokenización, vocabulario, generación de secuencias y construcción del **score_RNN** se presentan en la Sección 5.4.

Para el componente TDA, la base enriquecida con catálogo sirvió como insumo para representar cada producto mediante atributos estructurales como línea, familia y subfamilia. Esta representación permitió construir una estructura topológica del catálogo e identificar relaciones entre productos desde una perspectiva geométrica y de conectividad. La descripción completa de este componente se presenta en la Sección 5.2.

Finalmente, los *scores* generados por RNN, TDA y Series de Tiempo se integraron a nivel SKU para alimentar el Transformer de fusión temprana. Esta integración permitió que el modelo final combinara identidad del producto, comportamiento histórico de canasta, estructura del catálogo y oportunidad temporal.

3.2.8. Cobertura final de la base limpia

Después del proceso de depuración, la base transaccional limpia quedó conformada por aproximadamente 3,989,985 líneas de venta, 688,322 transacciones únicas y 7,792 SKUs vendidos. Para el componente temporal se conservaron 5,757 SKUs con historial suficiente para calcular métricas de tendencia, recencia, volumen y estacionalidad.

Métrica	Valor
Líneas de venta limpias	3,989,985
Transacciones únicas	688,322
SKUs únicos vendidos	7,792
SKUs elegibles para Series de Tiempo	5,757

Tabla 17: Cobertura de datos después del proceso de *data wrangling*.

Estos resultados muestran que el proceso de *wrangling* permitió pasar de tablas operativas del ERP a una base analítica consistente para modelado. La salida de esta etapa no es todavía el análisis exploratorio ni el modelo final, sino una serie de datasets limpios y estructurados que sirven como insumo para el EDA, la generación de *scores*, el entrenamiento del Transformer y la posterior inferencia del sistema de recomendación.

3.3. Protección de Datos y Ética del Modelo (BIAS)

Implicaciones éticas y de privacidad

El modelo ANA-03 opera sobre datos transaccionales agregados a nivel de ticket y producto, sin contener información de identificación personal de los clientes finales. Los registros de venta del ERP TCA Merksyst corresponden a transacciones completadas en el punto de venta donde el cliente es anónimo: no existe identificador de persona física, número de cliente ni dato biográfico en los datos de entrenamiento. Esto reduce significativamente el riesgo de privacidad respecto a sistemas de recomendación basados en perfiles de usuario individual.

Los datos pertenecen al socio formador y son clasificados como **confidenciales**. El equipo estableció como política que el repositorio debe ser privado. Esta decisión evita exponer datos transaccionales crudos y reduce el riesgo de reconstrucción de operaciones históricas sensibles.

En un escenario de despliegue comercial con identificadores de cliente, segmentos demográficos, domicilios, métodos de pago u otras variables personales, sería necesario revisar el cumplimiento con la *Ley Federal de Protección de Datos Personales en Posesión de los Particulares* (LFPDPPP) de México, obtener consentimiento informado de los titulares y establecer mecanismos de anonimización o seudonimización.

3.3.1. Sesgos potenciales en el dataset y en las predicciones

- **Sesgo histórico.** El modelo aprende exclusivamente de tickets completados, que reflejan también las limitaciones del proceso de venta actual. Si ciertas sucursales, categorías o periodos registran más diversidad de productos por ticket, el modelo puede aprender esas diferencias y reproducirlas en sus recomendaciones.

- **Sesgo de reporte.** Los datos capturan únicamente las ventas efectuadas, no los intentos de sugerencia rechazados por el cliente. El modelo interpreta la ausencia de una combinación como señal de baja afinidad, cuando puede existir demanda latente no convertida.
- **Sesgo de popularidad (*cold start* de ítem).** Los productos de reciente incorporación o baja rotación quedan fuera del vocabulario del modelo o reciben *scores* muy bajos, sub-recomendando artículos nuevos o de nicho que pueden ser de alto valor para el negocio.
- **Sesgo de confirmación.** La selección de umbrales mínimos de soporte, frecuencia o presencia durante el *Data Wrangling* puede influir en qué productos entran al entrenamiento. Umbrales demasiado restrictivos pueden excluir combinaciones válidas; umbrales demasiado permisivos pueden introducir ruido como señal.
- **Sesgo de retroalimentación.** Una vez en producción, los productos sugeridos con más frecuencia pueden venderse más, reforzando al modelo en el siguiente ciclo de reentrenamiento y reduciendo progresivamente la diversidad de recomendaciones.
- **Sesgo de disponibilidad operativa.** Una recomendación puede ser estadísticamente adecuada, pero no viable si el producto no tiene inventario. Si no se incorpora disponibilidad, el sistema puede sugerir productos imposibles de vender en ese momento, afectando la confianza del vendedor y del cliente.

Métricas de equidad y mitigaciones

Dado que el conjunto de entrenamiento no contiene variables demográficas ni identificadores personales de clientes, las métricas de equidad no deben interpretarse en esta etapa como auditorías completas sobre grupos sensibles de personas. En el alcance actual del proyecto, el enfoque ético se concentra principalmente en equidad operativa y de exposición de productos. Esto implica monitorear si el sistema recomienda de forma excesiva los productos más populares, si deja fuera productos nuevos o de baja rotación, si concentra las sugerencias en pocas categorías y si las recomendaciones son viables de acuerdo con el inventario disponible.

Tabla 18: Métricas de equidad operativa recomendadas para ANA-03.

Métrica	Definición	Umbral de alerta
Cobertura del catálogo	Porcentaje de SKUs activos que aparecen al menos una vez dentro de las recomendaciones generadas	Cobertura persistentemente menor a la meta definida para el periodo
Concentración de exposición	Proporción de recomendaciones acumuladas concentradas en los productos más sugeridos	Concentración excesiva en pocos SKUs o categorías
Diversidad por familia	Distribución de recomendaciones entre líneas, familias y subfamilias del catálogo	Sobrerrepresentación de una familia sin justificación comercial
Disponibilidad operativa	Porcentaje de recomendaciones con inventario disponible al momento de sugerirse	Recomendaciones sin stock por encima del umbral aceptado por negocio
Exposición de productos nuevos o baja rotación	Participación controlada de productos nuevos, estacionales o de baja rotación dentro de las recomendaciones	Exclusión sistemática de productos elegibles fuera del top de popularidad

Las mitigaciones implementadas y recomendadas son: (1) auditoría de la distribución de transacciones por producto, categoría, sucursal y periodo antes del entrenamiento; (2) desglose de métricas de evaluación MRR y Hit@K por familia de producto y formato de tienda; (3) monitoreo

continuo de la distribución de recomendaciones en producción para detectar deriva o concentración excesiva; (4) integración de inventario como restricción operativa; y (5) incorporación periódica de un componente de exploración que permita exponer productos sub-representados en un porcentaje controlado de las recomendaciones.

En caso de que una versión futura del sistema incorpore identificadores de cliente, segmentos demográficos o perfiles personalizados, deberán añadirse métricas de equidad a nivel usuario, como paridad demográfica, igualdad de oportunidades e impacto dispar. En ese escenario, un ratio de impacto dispar inferior a 0.8 entre segmentos activaría una revisión del modelo, de los datos de entrenamiento y de la política de recomendación.

3.4. Trazabilidad y Linaje

Desde la perspectiva de los artefactos del modelo, el linaje de la solución de inferencia es el siguiente: a partir de las transacciones de origen se derivan tres señales de comportamiento por producto (*scores* de RNN, TDA y Series de Tiempo), persistidas en `scores_dict.pkl`; se construye un vocabulario de productos en `transformer_vocab.pkl` y se conserva el catálogo con descripciones y precios en `catalogo.pkl`. Estos insumos alimentan el entrenamiento del modelo, cuyos pesos finales se serializan en `transformer_fusion.pt`. Ese conjunto de cuatro artefactos es exactamente el que carga el servicio de inferencia en producción, lo que garantiza correspondencia entre lo entrenado y lo desplegado.

La trazabilidad del código y de los artefactos se mantiene mediante un repositorio Git (GitHub), que versiona el código de la API, los *notebooks* de entrenamiento e inferencia y los artefactos necesarios para ejecutar el servicio. Adicionalmente, la imagen de contenedor se versiona en Azure Container Registry, permitiendo identificar qué versión del servicio fue desplegada en la nube.

3.4.1. Ciclo de vida completo de los datos

La trazabilidad del proyecto documenta la ruta íntegra desde el origen en el ERP TCA Merksyst hasta el endpoint de producción en Azure, registrando cada transformación, decisión de diseño y artefacto generado.

- **Origen.** Los datos fuente fueron proporcionados por TCA Software Solutions en formato Parquet bajo acuerdo de confidencialidad académica. Las tablas principales utilizadas fueron `pdvhdr`, `pdvdet`, `inviar` e `invars`. Estos archivos fueron clasificados como confidenciales; residen únicamente en el entorno controlado del equipo y no fueron versionados en ningún repositorio público.
- **Ingestión y exploración.** Los archivos Parquet se leyeron con herramientas de análisis en Python, principalmente `pandas` y `DuckDB`. En esta etapa se documentaron esquema, volumen, distribución de nulos, rango de fechas, cardinalidad de identificadores y consistencia de llaves entre encabezado y detalle de venta.
- **Transformación — Data Wrangling.** Las estrategias descritas en la Sección 3.2 se ejecutaron para estandarizar campos clave, reconstruir tickets, filtrar ventas válidas, enriquecer productos con catálogo e integrar información de inventario. Como resultado se generaron representaciones intermedias para distintos componentes: base ticket-producto limpia, canastas por ticket, tabla de producto-catálogo, panel semanal por SKU y variables de disponibilidad.
- **Feature engineering por componente.** A partir de las representaciones limpias se generaron tres señales principales por SKU. El componente RNN produjo un *score* secuencial basado en patrones de canastas históricas; el componente TDA produjo un *score* topológico basado en la estructura del catálogo; y el componente de Series de Tiempo produjo un

score temporal basado en tendencia, recencia, volumen y estacionalidad. Estas señales se consolidaron posteriormente en `scores_dict.pkl`.

- **Entrenamiento del Transformer.** El modelo `EarlyFusionTransformer` utilizó el vocabulario de productos, el catálogo y los *scores* RNN/TDA/TS como insumos. Durante el refinamiento se evaluaron configuraciones con Optuna, registrando hiperparámetros, MRR de validación, métricas Hit@K, duración e identificador del modelo seleccionado. Los pesos finales se serializaron en `transformer_fusion.pt` y el vocabulario en `transformer_vocab.pkl`.
- **Empaquetado.** Los cuatro artefactos necesarios para inferencia (`transformer_fusion.pt`, `transformer_vocab.pkl`, `catalogo.pkl`, `scores_dict.pkl`) se copiaron al directorio de la API junto con `app.py`, `Dockerfile` y `requirements.txt`. El `Dockerfile` construye una imagen sobre Python 3.12 con PyTorch CPU y ejecuta una prueba de humo automatizada antes de aprobar la imagen.
- **Publicación y despliegue.** La imagen `acrrecobasket.azurecr.io/reco-api:latest` fue publicada en Azure Container Registry y desplegada en Azure Container Apps mediante `az containerapp up`, con ingreso externo en puerto 8000, CORS habilitado y recursos suficientes para ejecutar inferencia en CPU.
- **Consumo final.** El endpoint `POST /recommend` recibe una canasta de 1–10 códigos de producto en JSON y devuelve el ranking de los 10 productos recomendados con código, descripción, precio, score relativo y diferencial respecto al siguiente candidato. Este endpoint puede consumirse desde una interfaz web, un punto de venta o cualquier pantalla del ERP que tenga acceso a los productos del ticket actual.
- **Versionamiento y reproducibilidad.** El código fuente, los notebooks de entrenamiento e inferencia y los artefactos del modelo están versionados en el repositorio Git. La imagen del contenedor está versionada en Azure Container Registry. La documentación de integración se concentra en la carpeta de la API, particularmente en el `readme.md` y en los notebooks de guía de consumo del servicio.

El diagrama de linaje completo es el siguiente:

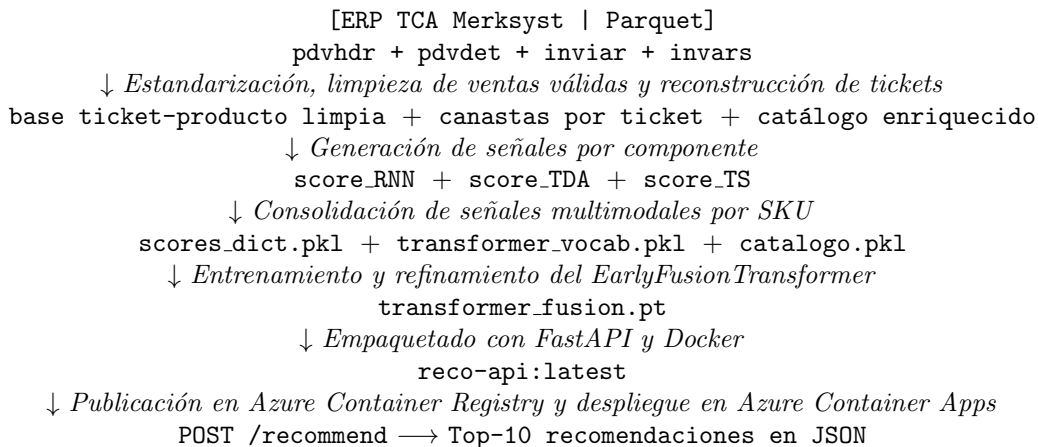


Figura 2: Diagrama de linaje de datos y artefactos — ANA-03.

4. Análisis Exploratorio de Datos (EDA)

El análisis exploratorio de datos tuvo como objetivo validar que la información transaccional del ERP TCA Merksyst fuera suficiente, consistente y útil para alimentar un sistema de reco-

mendación de canastas. A diferencia del *data wrangling*, descrito en la Sección 3.2, el EDA no se enfocó en transformar los datos para modelado, sino en comprender la estructura, cobertura, distribución y comportamiento de la base limpia resultante.

Dado que el proyecto integra distintos enfoques de modelado —TDA, Series de Tiempo, RNN y Transformer—, el EDA se abordó desde una perspectiva global. Primero, se analizaron las propiedades generales de la base transaccional: volumen de líneas, número de tickets, rango de fechas, cobertura de SKUs y consistencia de las llaves. Después, se revisó la profundidad de canasta, ya que el caso ANA-03 depende de identificar productos que aparecen juntos dentro de una misma transacción. Finalmente, se exploraron atributos de producto, disponibilidad de inventario y comportamiento temporal, con el fin de justificar las señales que posteriormente alimentan al modelo final.

En este sentido, el EDA funcionó como una etapa de diagnóstico para responder cuatro preguntas principales:

1. ¿La base limpia conserva suficiente volumen y cobertura para entrenar modelos de recomendación?
2. ¿Existen canastas con suficiente profundidad para aprender relaciones de co-compra?
3. ¿El catálogo cuenta con atributos útiles para enriquecer o interpretar las recomendaciones?
4. ¿Los productos presentan patrones de venta, actividad reciente o disponibilidad que deban considerarse antes de recomendar?

Los resultados de esta sección respaldan la decisión de construir una arquitectura híbrida. La base transaccional permite aprender patrones de canasta; el catálogo aporta estructura para el análisis topológico; las secuencias de productos permiten entrenar el componente RNN; y el historial temporal permite incorporar señales de tendencia, recencia, volumen e inventario. Los detalles metodológicos de cada componente se presentan en la Sección 5.

4.1. Limpieza de Datos

La etapa de limpieza exploratoria permitió verificar que los filtros aplicados durante el *data wrangling* produjeran una base coherente para análisis. En particular, se revisó que las transacciones conservaran únicamente ventas válidas, que los identificadores de ticket se reconstruyeran correctamente, que los productos vendidos tuvieran una clave de artículo interpretable y que la base final mantuviera suficiente cobertura para los distintos componentes del sistema.

La base limpia se construyó a partir de la unión entre `pdvhdr`, que contiene el encabezado de la venta, y `pdvdet`, que contiene las líneas de productos vendidos. La unión se realizó mediante la llave compuesta:

$$\text{Suc} + \text{FecOpe} + \text{Caja} + \text{NumTra}.$$

A partir de esta llave se creó un identificador único de ticket, lo que permitió agrupar todos los productos pertenecientes a una misma transacción. Esta reconstrucción fue indispensable para analizar afinidad de canasta, ya que el problema de negocio no se define a nivel de línea individual, sino a nivel de conjunto de productos comprados en el mismo ticket.

Los filtros principales aplicados antes del análisis fueron:

- conservar únicamente operaciones de venta mediante `pdvhdr.Tipo = '00'`;
- conservar tickets con estatus válido, principalmente TI y TIF;
- conservar únicamente líneas activas del detalle mediante `pdvdet.status = '01'`;

- eliminar registros con cantidad menor o igual a cero;
- eliminar registros sin clave válida de producto en `cve_art`;
- estandarizar campos clave como texto para evitar pérdida de ceros a la izquierda o inconsistencias por espacios.

Después de este proceso, la base quedó conformada por 3,989,985 líneas de venta limpias, 688,322 transacciones únicas y 7,792 SKUs vendidos. Esta cobertura confirmó que el volumen de datos era suficiente para construir señales de recomendación y analizar comportamiento de canasta.

Métrica	Valor
Líneas de venta limpias	3,989,985
Transacciones únicas	688,322
SKUs únicos vendidos	7,792
Periodo de ventas transaccionales	2024-03-28 a 2026-02-17
Promedio de SKUs únicos por ticket	5.19
Mediana de SKUs únicos por ticket	3
Tickets con dos o más SKUs	77.67 %
Tickets con un solo SKU	22.33 %

Tabla 19: Cobertura general de la base limpia después del proceso de preparación.

Un punto importante del análisis fue la decisión de no eliminar los tickets de un solo artículo en la base general. Para modelos de co-ocurrencia, este tipo de tickets no genera pares de productos; sin embargo, sí representa demanda real de un SKU y aporta información útil para el análisis de volumen, recencia, actividad comercial e inventario. Por ello, la base limpia conserva todas las ventas válidas y cada componente del pipeline aplica posteriormente los filtros específicos que requiere su técnica.

Desde el punto de vista exploratorio, esta decisión permite mantener una base común para todos los análisis: las canastas con dos o más productos alimentan la lectura de afinidad y secuencia, mientras que todos los tickets válidos contribuyen a entender la demanda real del catálogo.

4.2. Visualización de Datos

Las visualizaciones exploratorias se utilizaron para interpretar la estructura general del dataset y validar si existían patrones suficientes para justificar un sistema de recomendación. Aunque el modelo final integra distintos enfoques, las visualizaciones se organizaron alrededor de cuatro dimensiones: profundidad de canasta, concentración de productos, comportamiento temporal e inventario.

Primero, se analizó la distribución de productos por ticket. Esta revisión es relevante porque el caso ANA-03 busca incrementar la afinidad de canasta mediante sugerencias proactivas. La base limpia mostró un promedio de 5.19 SKUs únicos por ticket y una mediana de 3 SKUs. Esta diferencia entre media y mediana sugiere que existen tickets grandes que elevan el promedio, pero que una proporción importante de transacciones mantiene baja profundidad de canasta. En términos de negocio, esto confirma que existe espacio para recomendar productos complementarios sin asumir que todos los tickets ya están saturados.

En segundo lugar, se revisó la proporción de tickets con uno, dos o más productos. Se identificó que aproximadamente 77.67 % de los tickets contienen dos o más SKUs, mientras que 22.33 % corresponden a tickets de un solo SKU. Esta distribución es importante porque confirma que existe suficiente información de co-ocurrencia para modelos de afinidad, y al mismo tiempo

muestra que los tickets de un artículo siguen siendo relevantes para señales individuales de demanda.

En tercer lugar, se analizaron productos frecuentes y categorías dominantes. Este tipo de visualización permite identificar si el dataset está excesivamente concentrado en pocos SKUs o si existe diversidad suficiente para generar recomendaciones útiles. También permite anticipar posibles sesgos de popularidad: si los productos más vendidos dominan por completo la base, el modelo podría tender a recomendar siempre artículos frecuentes, incluso cuando no sean los más pertinentes para una canasta específica.

En cuarto lugar, se visualizaron series semanales de SKUs específicos. Estas gráficas permitieron observar productos con demanda estable, productos con ventas intermitentes, productos con crecimiento reciente y productos con patrones asociados a temporada. Esta exploración ayudó a justificar la inclusión de variables como tendencia, recencia, volumen y estacionalidad dentro del *score* temporal.

Finalmente, se revisaron visualizaciones relacionadas con disponibilidad de inventario. Esta dimensión no determina por sí sola la afinidad entre productos, pero sí es crítica para la operación: una recomendación puede ser estadísticamente razonable y aun así no ser accionable si el producto no tiene stock disponible. Por ello, el inventario se considera como una variable de validación de negocio y no únicamente como una variable descriptiva.

4.3. Estadísticas

El análisis estadístico descriptivo se realizó en cuatro niveles: transacción, canasta, producto y serie temporal. Esta separación permitió evaluar la base desde las distintas perspectivas que requiere el sistema: la transacción como unidad operativa, la canasta como unidad de afinidad, el producto como unidad de recomendación y la serie semanal como unidad temporal.

A nivel transacción, se validó que el volumen de tickets fuera suficiente para entrenar modelos sobre comportamiento real de compra. La base limpia contiene 688,322 transacciones únicas y 3,989,985 líneas de venta, lo que proporciona una muestra amplia para estudiar patrones de co-compra. A nivel producto, se identificaron 7,792 SKUs vendidos, lo que confirma una alta diversidad de catálogo y justifica la necesidad de un modelo capaz de ordenar candidatos en un espacio amplio de productos.

A nivel canasta, la media de SKUs únicos por ticket fue de 5.19, mientras que la mediana fue de 3. Esta diferencia indica una distribución asimétrica: algunos tickets contienen muchos productos y elevan el promedio, pero el ticket típico sigue siendo más pequeño. Desde el punto de vista del negocio, esta observación es relevante porque la oportunidad de cross-selling no necesariamente se encuentra en aumentar indiscriminadamente todos los tickets, sino en identificar transacciones con potencial real de complemento.

Métrica de canasta	Valor
Transacciones únicas	688,322
SKUs únicos vendidos	7,792
Promedio de SKUs únicos por ticket	5.19
Mediana de SKUs únicos por ticket	3
Tickets con un solo SKU	22.33 %
Tickets con dos o más SKUs	77.67 %

Tabla 20: Estadísticas generales de profundidad de canasta.

Para el componente RNN, el análisis de canastas permitió identificar un subconjunto con contexto suficiente para entrenamiento secuencial. Las canastas con tres o más artículos son especialmente útiles porque permiten construir ejemplos de canasta parcial y producto objetivo. Esta lectura exploratoria justificó la generación de secuencias mediante ventana deslizante y la

construcción de un vocabulario restringido a los productos más frecuentes.

Métrica asociada a canastas para RNN	Valor
Canastas útiles con tres o más artículos	437,488
Media de artículos por canasta útil	8.3
Tamaño del vocabulario principal	3,000 SKUs
Cobertura aproximada del vocabulario	80–90 % de ocurrencias

Tabla 21: Estadísticas exploratorias asociadas a la preparación del componente RNN.

A nivel temporal, el panel semanal final quedó formado por 5,757 SKUs elegibles y 97 semanas de observación. Esto generó 558,429 registros producto-semana. La construcción del panel completo incluyó semanas sin venta registradas como cero, lo cual permitió analizar inactividad, reactivación y recencia sin sobreestimar la continuidad de ventas.

A partir de ese panel se generó el archivo `ts.scores_ana03.csv`, con una fila por SKU. Este archivo resume el comportamiento temporal de cada producto mediante variables de tendencia, recencia, volumen, estacionalidad e inventario. En la salida generada, el `score_TS` tuvo un rango aproximado de 0.044 a 1.000, con media de 0.437 y mediana de 0.495. Esta distribución indica que el `score` no se concentró únicamente en valores extremos, sino que permitió diferenciar productos con distintos niveles de oportunidad temporal.

Métrica del componente TS	Valor
SKUs elegibles	5,757
Semanas analizadas	97
Registros producto-semana	558,429
Productos con stock disponible	4,965
Productos sin stock disponible	792
Productos vendidos en la semana de corte	3,610
Productos vendidos en las últimas 4 semanas	4,495
Productos vendidos en las últimas 8 semanas	4,723
Rango de <code>score_TS</code>	0.044 a 1.000
Media de <code>score_TS</code>	0.437
Mediana de <code>score_TS</code>	0.495

Tabla 22: Estadísticas descriptivas del componente de Series de Tiempo.

Estas estadísticas muestran que el análisis exploratorio no se limitó a validar el módulo temporal. La base también presentó suficiente volumen de canastas para aprender patrones de co-compra, suficiente cobertura de productos para justificar un vocabulario amplio y suficiente estructura de catálogo para enriquecer la interpretación de recomendaciones. En conjunto, los resultados indican que los datos disponibles son adecuados para una solución híbrida de recomendación.

4.4. Pruebas de Hipótesis

Como parte del EDA se aplicaron pruebas estadísticas formales sobre las series semanales por SKU. Estas pruebas se concentraron en el componente temporal porque, dentro del proyecto, las propiedades de estacionariedad y ciclicidad son las que pueden evaluarse mediante hipótesis estadísticas tradicionales. En contraste, los componentes de canasta, RNN y TDA se validan principalmente mediante métricas de cobertura, distribución, estructura de datos y desempeño predictivo, que se documentan en las secciones posteriores del reporte.

El objetivo de estas pruebas no fue validar reglas de asociación entre productos, sino caracterizar propiedades temporales del catálogo: estabilidad, estacionariedad y posible comportamiento cíclico. Estas pruebas se aplicaron sobre el panel producto-semana descrito en la Sección 5.3.

La primera prueba utilizada fue Augmented Dickey-Fuller (ADF), cuyo objetivo es evaluar si una serie temporal es estacionaria. En términos generales, una serie estacionaria mantiene propiedades estadísticas relativamente estables en el tiempo, como media y varianza. Esto es útil para distinguir productos con comportamiento relativamente estable de productos con tendencias fuertes, cambios estructurales o patrones no constantes.

La prueba se planteó con las siguientes hipótesis:

H_0 : la serie no es estacionaria,

H_1 : la serie es estacionaria.

Se utilizó un nivel de significancia de 5 %. Por lo tanto, cuando el valor-p fue menor a 0.05, la serie se clasificó como estacionaria. Cuando el valor-p fue mayor o igual a 0.05, no se contó con evidencia suficiente para rechazar la hipótesis nula. Además, las series con historial insuficiente fueron clasificadas por separado para evitar conclusiones estadísticas débiles.

Los resultados mostraron que 63.2 % de las series fueron clasificadas como estacionarias, 24.0 % como no estacionarias y 12.9 % como series sin datos suficientes. Esto sugiere que una mayoría de productos presenta comportamiento relativamente estable, mientras que un subconjunto relevante muestra variaciones, tendencias o cambios que deben interpretarse con mayor cuidado.

La segunda evaluación estadística se enfocó en identificar evidencia de comportamiento cíclico o repetitivo. Para ello se calcularon autocorrelaciones en rezagos semanales seleccionados:

$$L = \{4, 13, 26, 52\}.$$

Estos rezagos representan aproximaciones a ciclos mensuales, trimestrales, semestrales y anuales, respectivamente. Para un producto p y un rezago ℓ , la autocorrelación puede expresarse conceptualmente como:

$$\rho_\ell(p) = \text{Corr}(y_{p,t}, y_{p,t-\ell}).$$

Una autocorrelación elevada en alguno de estos rezagos sugiere que la serie puede tener un patrón repetitivo en esa ventana temporal. Este análisis se interpretó como evidencia exploratoria de ciclicidad, no como una afirmación causal ni como una prueba definitiva de estacionalidad. Su propósito fue identificar productos cuyo comportamiento podría beneficiarse de estrategias de actualización o priorización temporal.

Los resultados generales indicaron que 40.8 % de las series presentaron evidencia de comportamiento cíclico, 46.4 % no mostraron evidencia cíclica y 12.9 % no contaron con datos suficientes. Entre los patrones detectados, el ciclo mensual aproximado fue el más frecuente, seguido por patrones anuales, trimestrales y semestrales. Esto indica que una parte importante del catálogo tiene comportamiento repetitivo y que, por lo tanto, la temporalidad es una señal relevante para el sistema de recomendación.

Análisis	Categoría	Porcentaje
Estacionariedad	Estacionaria	63.2 %
Estacionariedad	No estacionaria	24.0 %
Estacionariedad	Sin datos suficientes	12.9 %
Ciclicidad	Con evidencia cíclica	40.8 %
Ciclicidad	Sin evidencia cíclica	46.4 %
Ciclicidad	Sin datos suficientes	12.9 %

Tabla 23: Resultados generales de las pruebas estadísticas aplicadas a las series temporales.

Estas pruebas aportan evidencia adicional para justificar el componente de Series de Tiempo. Sin embargo, su interpretación debe mantenerse dentro de su alcance: no prueban causalidad ni afinidad entre productos, sino que demuestran que una parte del catálogo contiene estructura temporal útil. En conjunto con las estadísticas de canasta, cobertura de SKUs e inventario, estas pruebas respaldan la necesidad de combinar señales temporales con señales estructurales y secuenciales dentro del recomendador final.

4.5. Hallazgos

El EDA permitió identificar hallazgos relevantes para el diseño del sistema de recomendación.

En primer lugar, la reconstrucción de tickets a partir de las tablas de encabezado y detalle fue viable. La base limpia conserva un volumen alto de transacciones y productos, lo que permite entrenar modelos de recomendación sobre datos reales de comportamiento de compra. Esto es particularmente importante para ANA-03, ya que el problema depende de conocer qué productos aparecen juntos dentro de una misma canasta.

En segundo lugar, la distribución de productos por ticket confirma que existe información útil para modelar afinidad de canasta. La mayoría de las transacciones contiene dos o más productos, por lo que se pueden estudiar patrones de co-ocurrencia. Al mismo tiempo, la mediana de tres SKUs por ticket sugiere que todavía existe oportunidad de incrementar la profundidad de ciertas canastas mediante recomendaciones complementarias.

En tercer lugar, el catálogo presenta una alta diversidad de productos. La base limpia contiene 7,792 SKUs vendidos, pero no todos tienen el mismo nivel de información histórica. Esta diferencia es relevante porque el sistema debe manejar productos con distintos niveles de madurez, frecuencia de venta y disponibilidad. Por ello, el modelo final no puede depender únicamente de popularidad, ya que esto excluiría productos menos frecuentes pero potencialmente relevantes.

En cuarto lugar, el análisis de canastas permitió justificar el componente RNN. La existencia de un subconjunto amplio de tickets con tres o más artículos permite generar secuencias de entrenamiento donde una canasta parcial se usa para predecir un producto siguiente o complementario. Esto respalda la utilidad de incorporar una señal secuencial basada en patrones observados dentro de tickets históricos.

En quinto lugar, los atributos del catálogo justifican el uso de TDA como señal estructural. Al contar con información de línea, familia, subfamilia y descripción de producto, es posible representar los SKUs en un espacio de características y analizar relaciones que no dependen únicamente de frecuencia de compra conjunta. Esto permite capturar similitudes o proximidades entre productos que podrían no aparecer claramente mediante conteos simples de co-ocurrencia.

En sexto lugar, el análisis temporal mostró que una proporción relevante de productos tiene actividad reciente y que una parte del catálogo presenta comportamiento estable o cíclico. Esto respalda la incorporación del `score_TS` como una señal complementaria, ya que el tiempo aporta información que no se captura únicamente con co-ocurrencia, estructura de catálogo o secuencia de canasta.

En séptimo lugar, la integración de inventario mostró que no basta con estimar afinidad o probabilidad de compra. Una recomendación debe ser accionable en el punto de venta; por lo tanto, la disponibilidad operativa debe considerarse como filtro o ajuste antes de mostrar productos al vendedor o al cliente.

Finalmente, el EDA permitió concluir que el sistema de recomendación debe ser híbrido. Un enfoque basado solo en popularidad podría favorecer siempre los productos más vendidos; un enfoque basado solo en co-ocurrencia podría ignorar temporalidad e inventario; un enfoque puramente secuencial podría no capturar estructura de catálogo; y un enfoque únicamente temporal podría recomendar productos oportunos pero no necesariamente compatibles con la canasta actual. Por ello, los hallazgos exploratorios respaldan la decisión de combinar TDA, Series de Tiempo, RNN y Transformer en una arquitectura de fusión temprana.

5. Scores para el modelo

5.1. Explicación y relevancia

El sistema de recomendación desarrollado para el caso ANA-03 se planteó como un problema de *basket completion*: dada una canasta parcial de productos, el objetivo es estimar qué otros artículos tienen mayor probabilidad de ser agregados por el cliente. Formalmente, si una canasta parcial se representa como una secuencia de SKUs

$$B_t = (p_1, p_2, \dots, p_t),$$

el modelo debe producir un *ranking* de productos candidatos

$$\hat{p}_{t+1}^{(1)}, \hat{p}_{t+1}^{(2)}, \dots, \hat{p}_{t+1}^{(k)}$$

que maximice la probabilidad de acierto dentro de las primeras k posiciones. Sin embargo, una canasta de mercado no solo contiene información secuencial. Cada producto también posee relaciones de similitud con otros productos, comportamiento temporal propio y patrones de co-ocurrencia observados en compras previas. Por esta razón, antes del modelo final se generaron *scores* especializados que capturan distintas dimensiones del fenómeno de compra.

En este proyecto, un *score* se define como una variable numérica asociada a un SKU que resume una propiedad específica del producto. Estos *scores* se construyen a partir de los datos transaccionales y posteriormente se normalizan para que puedan ser comparables e integrables dentro del modelo de fusión. Para cada producto p , se define un vector de señales:

$$\mathbf{s}_p = \begin{bmatrix} s_{\text{TDA}}(p) \\ s_{\text{TS}}(p) \\ s_{\text{RNN}}(p) \end{bmatrix},$$

donde $s_{\text{TDA}}(p)$ representa la señal topológica, $s_{\text{TS}}(p)$ representa la señal temporal y $s_{\text{RNN}}(p)$ representa la señal secuencial o conductual aprendida a partir de canastas históricas. La función de este vector no es decidir por sí solo la recomendación final, sino proporcionar al Transformer una descripción enriquecida de cada producto.

La relevancia de utilizar *scores* previos al Transformer se basa en tres razones principales. Primero, el espacio de productos es grande y disperso: existen miles de SKUs, muchos de los cuales no aparecen con la misma frecuencia en las transacciones. Si el modelo solo aprendiera a partir de identificadores de producto, tendría que inferir todas las relaciones desde cero. Al incluir señales externas, se reduce parte de esta carga de aprendizaje y se incorpora conocimiento estadístico previamente calculado.

Segundo, los patrones de compra son multidimensionales. Dos productos pueden ser relevantes entre sí por razones distintas. Por ejemplo, pueden ser topológicamente similares porque pertenecen a familias o subfamilias cercanas; pueden estar relacionados porque aparecen frecuentemente en las mismas canastas; o pueden ser oportunos porque presentan crecimiento reciente en ventas. Ninguna técnica individual captura completamente todas estas dimensiones. Por ello, el diseño del sistema combina enfoques complementarios.

Tercero, los *scores* permiten que el modelo final sea más interpretable desde una perspectiva de negocio. Si un producto es recomendado, es posible analizar si la recomendación fue favorecida por su cercanía topológica, su comportamiento temporal o su relación secuencial con canastas anteriores. Esta interpretabilidad es importante porque el caso de uso no solo busca predecir productos, sino también habilitar acciones comerciales como sugerencias proactivas, creación de combos y campañas.

El primer componente es el **score topológico**, generado mediante Análisis Topológico de Datos (TDA). Esta señal busca representar la estructura del catálogo y las relaciones de cercanía entre productos. A diferencia de una simple comparación directa de categorías, el enfoque

topológico permite observar agrupamientos y conexiones entre SKUs a partir de sus características. En términos conceptuales, TDA responde la pregunta: *qué productos son estructuralmente similares o cercanos dentro del espacio del catálogo*. Esta información es valiosa porque productos cercanos pueden funcionar como sustitutos, complementos o candidatos naturales dentro de una misma lógica de compra.

El segundo componente es el **score de Series de Tiempo**. Esta señal evalúa el comportamiento temporal de cada SKU a partir de sus ventas históricas. Su objetivo no es identificar afinidad de canasta, sino medir la oportunidad temporal de recomendar un producto. En otras palabras, responde la pregunta: *de los productos que podrían ser recomendados, cuáles conviene priorizar en este momento*. Para ello se consideran variables como crecimiento reciente, recencia de venta, volumen de ventas, estacionalidad e inventario disponible. Este componente es especialmente relevante en retail, ya que la demanda de productos puede cambiar por temporadas, promociones, hábitos de consumo o cambios recientes en la preferencia de los clientes.

El tercer componente es el **score secuencial**, generado mediante una Red Neuronal Recurrente (RNN). Esta señal busca capturar patrones de comportamiento observados en canastas históricas. Desde esta perspectiva, el problema se aproxima a una tarea de predicción condicionada: dado un conjunto o secuencia parcial de productos, se estima qué productos han aparecido de forma recurrente en contextos similares. Conceptualmente, la RNN responde la pregunta: *qué productos suelen aparecer después o junto con una combinación observada de productos*. Esta señal complementa a TDA y Series de Tiempo porque aprende directamente de la frecuencia y orden relativo de las compras pasadas.

La integración de estos tres enfoques se realiza mediante un Transformer de fusión temprana. Para cada producto de entrada, el modelo no recibe únicamente su identificador, sino también su vector de *scores*. En términos generales, cada SKU se representa mediante dos tipos de información: una representación aprendida de identidad del producto y una representación derivada de los *scores* externos. Si $\mathbf{e}_p$ denota el *embedding* de identidad del producto y $\mathbf{s}_p$ su vector de señales, la representación enriquecida puede expresarse de forma conceptual como

$$\mathbf{z}_p = [\mathbf{e}_p; f(\mathbf{s}_p)],$$

donde $f(\cdot)$ es una transformación aprendible que proyecta los *scores* al espacio de representación del modelo, y $[\cdot; \cdot]$ representa concatenación. Esta representación enriquecida se alimenta al Transformer, que utiliza mecanismos de atención para ponderar la importancia relativa de los productos presentes en la canasta.

El uso de fusión temprana tiene una ventaja importante: permite que el Transformer aprenda interacciones entre la identidad del producto y sus señales externas desde las primeras capas del modelo. En lugar de combinar recomendaciones al final mediante un promedio simple de modelos independientes, la arquitectura permite que las señales TDA, TS y RNN influyan en la forma en que se construye la representación contextual de la canasta. Esto es especialmente útil en el caso de canastas parciales, donde el significado de un producto puede cambiar dependiendo de los demás artículos presentes.

La salida final del modelo es un vector de puntuaciones sobre el vocabulario de productos. Después de aplicar una función de normalización como *softmax*, estas puntuaciones pueden interpretarse como una distribución de probabilidad sobre el siguiente producto candidato:

$$P(p \mid B_t) = \text{softmax}(g(\mathbf{h}_t)),$$

donde $\mathbf{h}_t$ representa la codificación contextual de la canasta producida por el Transformer y $g(\cdot)$ es la capa de salida que proyecta dicha representación al espacio de productos. A partir de esta distribución se obtiene el *Top-K* de recomendaciones.

Es importante enfatizar que los *scores* no deben interpretarse como reglas finales de decisión. Un producto con alto *score* temporal puede estar en tendencia, pero no necesariamente ser compatible con la canasta actual. Del mismo modo, un producto cercano topológicamente puede

no ser oportuno en el momento de compra, o un producto frecuente en canastas pasadas puede no tener inventario disponible. La aportación del modelo final consiste precisamente en aprender cómo combinar estas señales dependiendo del contexto de la canasta.

En síntesis, la sección de *scores* constituye la etapa de ingeniería de características avanzada del sistema. TDA aporta estructura, Series de Tiempo aporta oportunidad temporal y RNN aporta comportamiento histórico de compra. La combinación de estas señales dentro de un Transformer permite construir un recomendador más robusto que un modelo basado únicamente en co-ocurrencia o popularidad, ya que integra similitud, tendencia y patrones de canasta en una misma arquitectura predictiva.

5.2. Topological Data Analysis (TDA)

El componente de *Topological Data Analysis* (TDA) se desarrolló para identificar relaciones de afinidad entre productos a partir de sus atributos descriptivos y su posición dentro del catálogo. A diferencia de los enfoques tradicionales basados exclusivamente en frecuencia de compra conjunta o reglas de asociación, este componente busca capturar la estructura geométrica y topológica del espacio de productos para descubrir similitudes no evidentes entre artículos.

La idea central consiste en representar cada producto dentro de un espacio de características, construir una representación topológica de dicho espacio mediante el algoritmo Mapper y utilizar la conectividad resultante para estimar afinidades entre productos. De esta manera, productos que comparten regiones cercanas dentro de la estructura topológica pueden considerarse relacionados, aun cuando no aparezcan conjuntamente con alta frecuencia en las transacciones históricas.

En el contexto del sistema de recomendación, el componente TDA genera una señal de afinidad estructural entre productos. Conceptualmente, esta señal puede expresarse como una función de afinidad $A_{TDA}(p_i, p_j)$ entre pares de productos, la cual posteriormente puede resumirse o integrarse como un *score* topológico por SKU dentro del modelo de fusión temprana. Esta señal complementa a Series de Tiempo y RNN porque aporta información sobre similitud estructural del catálogo, no sobre actualidad temporal ni sobre secuencias de compra.

Conceptualmente, el componente TDA responde la siguiente pregunta:

¿Qué productos ocupan posiciones cercanas dentro de la estructura topológica del catálogo?

Esta aproximación permite descubrir relaciones latentes entre artículos que comparten características descriptivas similares y que podrían representar oportunidades de recomendación no evidentes mediante métodos tradicionales.

5.2.1. Preparación e integración de datos

El proceso inicia con la integración de información proveniente de fuentes transaccionales y maestras de productos. Estas fuentes contienen información de ventas, detalle de artículos y atributos descriptivos necesarios para caracterizar cada SKU.

Antes de construir el espacio de análisis se realizan procesos de limpieza, homologación y normalización de cadenas de texto con el objetivo de garantizar consistencia en identificadores, categorías y atributos descriptivos. Una vez consolidadas las fuentes, se genera un universo único de productos elegibles para el análisis topológico.

El resultado de esta etapa es una tabla maestra donde cada fila representa un producto y contiene los atributos descriptivos utilizados para construir el espacio de características.

5.2.2. Construcción del espacio de características

Cada producto se representa mediante un conjunto de atributos categóricos seleccionados por su capacidad para describir la posición comercial del artículo dentro del catálogo. Entre los

atributos utilizados se encuentran:

- código de artículo;
- línea;
- familia;
- subfamilia.

Debido a que estas variables son de naturaleza categórica, se aplica una transformación mediante *Label Encoding*, asignando un identificador numérico a cada categoría observada. Esta codificación se utiliza como una representación operativa para construir el espacio de análisis; no debe interpretarse como una jerarquía ordinal natural entre categorías.

Como resultado, cada producto queda representado mediante un vector numérico:

$$\mathbf{x}_p = [x_1, x_2, \dots, x_n],$$

donde cada componente corresponde a alguno de los atributos descriptivos codificados.

Esta representación constituye el espacio inicial sobre el cual se aplican posteriormente técnicas de reducción de dimensionalidad y análisis topológico.

5.2.3. Reducción de dimensionalidad

La construcción del algoritmo Mapper requiere una representación compacta de los datos. Para ello se implementa un proceso de reducción de dimensionalidad en dos etapas complementarias.

En una primera fase se utiliza ISOMAP, una técnica basada en distancias geodésicas que permite aproximar la geometría intrínseca del espacio de características. Esta transformación ayuda a preservar relaciones no lineales que podrían perderse mediante métodos puramente lineales.

Posteriormente se aplica UMAP sobre la representación generada por ISOMAP. UMAP permite obtener una proyección de menor dimensión preservando tanto estructura local como relaciones globales entre productos.

La combinación de ambas técnicas genera una representación reducida que facilita la construcción de la estructura topológica y mejora la separación de regiones con características similares.

5.2.4. Construcción del grafo Mapper

Una vez obtenida la proyección reducida, se utiliza el algoritmo Mapper para construir una representación topológica del catálogo de productos.

El procedimiento se desarrolla en tres etapas principales:

1. cubrimiento del espacio reducido mediante regiones parcialmente superpuestas;
2. aplicación de un algoritmo de agrupamiento dentro de cada región;
3. construcción de un grafo donde cada agrupamiento constituye un nodo y las intersecciones entre grupos generan conexiones entre nodos.

El resultado es una red que resume la forma global del espacio de productos y permite identificar regiones densas, conexiones entre categorías y estructuras complejas difíciles de observar mediante técnicas convencionales.

Formalmente, cada nodo puede representarse como:

$$N_i = \{p_1, p_2, \dots, p_n\},$$

donde los elementos corresponden a los productos contenidos dentro de una región topológica específica.

Esta representación constituye la base para el cálculo posterior de afinidades entre productos.

5.2.5. Estrategias de clusterización

El desempeño del algoritmo Mapper depende del método utilizado para agrupar observaciones dentro de cada región del cubrimiento. Por esta razón se evaluaron distintas estrategias de clusterización.

La primera alternativa corresponde a DBSCAN. Este algoritmo presenta ventajas importantes para el problema analizado, ya que no requiere especificar previamente el número de grupos, permite identificar estructuras de forma arbitraria y es robusto ante observaciones consideradas ruido.

Como estrategia alternativa se evaluó K-Means. Debido a que este algoritmo requiere definir el número de grupos de manera anticipada, se realizó un análisis exploratorio para determinar valores apropiados de K .

Para ello se utilizaron dos criterios complementarios. El primero fue el coeficiente de silueta (*Silhouette Score*), que evalúa simultáneamente la cohesión interna de los grupos y la separación respecto a otros conglomerados. El segundo fue el Método del Codo, basado en la evolución de la inercia al incrementar el número de grupos.

La combinación de ambas métricas permitió identificar configuraciones de agrupamiento adecuadas para la construcción final del grafo Mapper.

5.2.6. Cálculo de afinidad topológica

Una vez construido el grafo Mapper, se calcula la afinidad entre productos utilizando la conectividad de la estructura topológica.

Para cada producto objetivo se identifican los nodos donde aparece y posteriormente se construye una estructura inversa que permite mapear productos hacia los nodos asociados:

$$p_j \rightarrow \{N_1, N_3, N_7, \dots\}.$$

Esta estructura permite localizar eficientemente las regiones topológicas relacionadas con cada producto y facilita la exploración del grafo.

Sea $\mathcal{N}(p)$ el conjunto de nodos del grafo Mapper que contienen al producto p . Para dos productos p_i y p_j , una primera medida de similitud se obtiene mediante una modificación al índice de Jaccard aplicado sobre sus conjuntos de nodos:

$$J_{mod}(p_i, p_j) = |\mathcal{N}(p_i) \cap \mathcal{N}(p_j)| + \frac{|\mathcal{N}(p_i) \cap \mathcal{N}(p_j)|}{|\mathcal{N}(p_i) \cup \mathcal{N}(p_j)|}.$$

Esta métrica toma valores en el intervalo $[0, 1]$, donde un valor cercano a uno indica que ambos productos aparecen en las mismas regiones topológicas del espacio Mapper.

Sin embargo, dos productos pueden no compartir exactamente los mismos nodos y aun así encontrarse próximos dentro de la estructura global del grafo. Por esta razón también se considera la distancia topológica mínima entre los nodos asociados a ambos productos.

Sea $d(p_i, p_j)$ la longitud del camino más corto entre los nodos que contienen a p_i y los nodos que contienen a p_j , calculada mediante una búsqueda en anchura (*Breadth First Search*, BFS):

$$d(p_i, p_j) = \min_{u \in \mathcal{N}(p_i), v \in \mathcal{N}(p_j)} \text{dist}(u, v),$$

donde $\text{dist}(u, v)$ representa la distancia geodésica dentro del grafo Mapper.

A partir de esta distancia se define una medida de proximidad topológica:

$$P(p_i, p_j) = \frac{1}{d(p_i, p_j)}.$$

Esta formulación evita divisiones entre cero cuando dos productos comparten un mismo nodo y asigna valores más altos a productos ubicados en regiones cercanas del grafo. De esta forma, productos con coincidencia directa o cercanía estructural reciben mayor afinidad topológica.

En conjunto, el índice de Jaccard y la proximidad topológica permiten capturar dos dimensiones complementarias: la coincidencia directa dentro de regiones topológicas comunes y la cercanía estructural dentro del grafo global.

5.2.7. Generación de recomendaciones y salida del componente TDA

A partir de las afinidades topológicas se construye un ranking de productos candidatos para cada artículo del catálogo.

Para un producto objetivo p , se calculan las afinidades $A_{\text{TDA}}(p, q)$ respecto a todos los productos candidatos q y posteriormente se ordenan de manera descendente:

$$\mathcal{R}_{\text{TDA}}(p) = \underset{q}{\text{argsort}}(A_{\text{TDA}}(p, q)).$$

Los primeros elementos de este ranking corresponden a los productos con mayor cercanía estructural de acuerdo con el componente TDA.

Esta estrategia permite identificar relaciones entre artículos que no necesariamente presentan alta frecuencia de compra conjunta, pero que ocupan posiciones cercanas dentro de la estructura geométrica y topológica del espacio de características.

Desde la perspectiva del pipeline completo, el componente TDA transforma atributos descriptivos de productos en una red de relaciones interpretables sustentadas en geometría, agrupamiento y teoría de grafos. Su aportación principal es enriquecer al sistema de recomendación con una señal estructural del catálogo, complementando la oportunidad temporal capturada por TS y los patrones secuenciales aprendidos por RNN.

5.3. Time Series (TS)

El componente de Series de Tiempo (TS) se desarrolló para incorporar al sistema de recomendación una señal temporal por producto. Su función dentro del pipeline no es generar reglas de asociación ni predecir de manera aislada qué artículo debe recomendarse, sino cuantificar qué tan oportuno es recomendar un SKU en el momento actual. En otras palabras, mientras los enfoques de TDA y RNN aportan información sobre similitud estructural y patrones históricos de canasta, el componente TS aporta una señal de actualidad comercial basada en el comportamiento reciente de ventas.

En el contexto del modelo final, el *score* de Series de Tiempo se representa como $s_{\text{TS}}(p)$ para cada producto p . Esta variable resume distintas propiedades temporales del SKU, tales como crecimiento reciente, recencia de venta, volumen comercial y posible comportamiento estacional. El objetivo es que, al momento de fusionar señales dentro del Transformer, el modelo no solo considere qué productos son similares o frecuentes en canastas históricas, sino también cuáles muestran mayor relevancia temporal.

Conceptualmente, el componente TS responde la siguiente pregunta:

De los productos candidatos a recomendar, ¿cuáles conviene priorizar ahora?

Esta pregunta es especialmente relevante en un entorno de retail, ya que la demanda de productos puede cambiar por temporalidad, promociones, hábitos de consumo, rotación de inventario o variaciones recientes en las preferencias de los clientes. Por ello, un producto históricamente relacionado con una canasta no necesariamente debe recibir la misma prioridad si actualmente presenta baja actividad, caída reciente o falta de inventario.

5.3.1. Entrada del componente temporal

El módulo de Series de Tiempo parte de la base transaccional limpia construida a partir de los tickets de venta y sus líneas de detalle. La descripción completa del proceso de limpieza, unión entre tablas y filtrado de transacciones válidas se documenta en la subsección *Calidad de Datos y Data Wrangling*. En esta sección únicamente se retoma el resultado necesario para el componente temporal: una base a nivel línea de venta con fecha de operación, identificador de ticket, SKU, cantidad vendida, venta monetaria y atributos descriptivos del producto.

Para el componente TS no se eliminaron tickets de un solo artículo. Aunque este tipo de transacciones no aporta pares de productos para un análisis de co-ocurrencia, sí representa demanda real de un SKU. Por lo tanto, conservar estos tickets es necesario para estimar correctamente el volumen, la recencia y la tendencia temporal de cada producto.

La unidad mínima de análisis temporal se definió como la combinación producto-semana. Para cada SKU p y semana t , se agregaron las ventas observadas en las transacciones válidas:

$$y_{p,t}^{qty} = \sum_{i \in (p,t)} \text{cantidad}_i,$$

$$y_{p,t}^{sales} = \sum_{i \in (p,t)} \text{venta}_i,$$

$$y_{p,t}^{tickets} = |\{\text{tickets únicos que contienen el producto } p \text{ durante la semana } t\}|.$$

La frecuencia semanal se eligió como un punto medio entre granularidad y estabilidad. Una frecuencia diaria puede introducir demasiado ruido para productos con venta intermitente, mientras que una frecuencia mensual reduce el número de observaciones disponibles para calcular tendencias y patrones de corto plazo. La agregación semanal permite suavizar fluctuaciones operativas sin perder sensibilidad temporal.

El resultado de esta etapa se almacena en el archivo `weekly_series_ana03.csv`, el cual contiene una fila por SKU y semana. Este archivo funciona como la base histórica del componente temporal y permite analizar la evolución de cada producto en términos de unidades, ventas monetarias y presencia en tickets.

5.3.2. Construcción del panel semanal

Después de la agregación inicial, se construyó un panel temporal completo. Esto significa que cada SKU elegible aparece en todas las semanas del periodo analizado, incluso si no registró ventas en alguna de ellas. Cuando un producto no tuvo ventas en una semana específica, sus variables de cantidad, venta monetaria y tickets se registraron como cero:

$$y_{p,t}^{qty} = 0, \quad y_{p,t}^{sales} = 0, \quad y_{p,t}^{tickets} = 0.$$

Esta decisión es importante porque una semana sin venta también contiene información. Si se eliminaran las semanas con cero ventas, el modelo observaría únicamente los periodos activos y podría sobreestimar la continuidad de demanda del producto. Mantener los ceros permite medir inactividad, caídas, reactivaciones y comportamiento intermitente.

Además, antes de generar los *scores*, se conservaron únicamente productos con historial suficiente para que las métricas temporales fueran interpretables. Los criterios específicos de

calidad y elegibilidad de datos se describen en la subsección *Calidad de Datos y Data Wrangling*. En términos operativos, este filtrado evita que productos con muy pocas observaciones obtengan *scores* artificialmente altos por variaciones porcentuales extremas.

El panel semanal generado contiene 5,757 SKUs elegibles y 97 semanas de observación, para un total de 558,429 registros producto-semana. Esta estructura es la base para calcular las variables temporales agregadas que posteriormente se sintetizan en el *score* TS.

5.3.3. Ingeniería de variables temporales

A partir del panel semanal se generó una tabla final a nivel producto, donde cada fila resume el comportamiento temporal de un SKU hasta una semana de corte. Esta tabla corresponde al archivo `ts_scores_ana03.csv`. Para construirla se calcularon variables de tendencia, recencia, volumen y estacionalidad.

La primera familia de variables corresponde a la comparación de ventanas recientes y ventanas previas. Para cada producto se calcularon acumulados de unidades, ventas monetarias y tickets en ventanas de 4 y 8 semanas. Por ejemplo, para unidades vendidas en una ventana reciente de 4 semanas se define:

$$Q_{recent}^{4w}(p) = \sum_{t=T-3}^T y_{p,t}^{qty},$$

donde T representa la semana de corte. De forma análoga, la ventana previa de 4 semanas se define como:

$$Q_{prev}^{4w}(p) = \sum_{t=T-7}^{T-4} y_{p,t}^{qty}.$$

Con estas dos cantidades se estima el crecimiento reciente del producto:

$$trend_{qty}^{4w}(p) = \frac{Q_{recent}^{4w}(p) - Q_{prev}^{4w}(p)}{Q_{prev}^{4w}(p) + \epsilon},$$

donde ϵ es una constante pequeña utilizada para evitar divisiones entre cero. Esta misma lógica se aplica a ventas monetarias y a ventanas de 8 semanas.

La segunda familia de variables corresponde a la pendiente de tendencia. Para cada SKU se ajustó una relación lineal simple sobre las últimas 12 semanas:

$$y_{p,t}^{qty} = \alpha_p + \beta_p t + \varepsilon_{p,t}.$$

El coeficiente β_p resume la dirección local de la demanda. Un valor positivo indica una trayectoria creciente, mientras que un valor negativo indica una trayectoria decreciente. Esta señal complementa las comparaciones de ventanas porque captura una dirección más sostenida y no depende únicamente de dos acumulados.

La tercera familia de variables corresponde a la recencia. Para cada SKU se identificó la última semana con venta positiva:

$$last_sale(p) = \max\{t : y_{p,t}^{qty} > 0\},$$

y posteriormente se calculó el número de semanas transcurridas desde dicha venta:

$$weeks_since_last_sale(p) = T - last_sale(p).$$

Esta variable permite penalizar productos que, aunque tuvieron ventas históricas, no han registrado actividad reciente. Desde una perspectiva de negocio, esto es relevante porque un

producto inactivo o con baja rotación reciente puede no ser una recomendación adecuada para el vendedor.

La cuarta familia de variables corresponde al volumen. Esta señal se incorporó para evitar que productos con ventas muy pequeñas dominaran el *score* únicamente por crecimiento porcentual. Por ejemplo, un producto que pasa de una a dos unidades vendidas presenta un crecimiento de 100 %, pero esta variación no necesariamente representa una oportunidad comercial robusta. Por ello, el volumen reciente se considera como una señal independiente.

Finalmente, se calculó una señal de estacionalidad. Esta variable busca identificar si el periodo de corte es relativamente favorable para un producto en comparación con su comportamiento histórico. A diferencia de las pruebas estadísticas de ciclicidad, cuya discusión corresponde a la subsección *Pruebas de hipótesis sobre series temporales*, la señal de estacionalidad utilizada en el *score* TS funciona como una variable de oportunidad comercial: aumenta la relevancia de productos que suelen comportarse mejor en el periodo actual.

5.3.4. Normalización de señales

Las variables generadas tienen escalas distintas: las cantidades son conteos, las ventas son valores monetarios, los crecimientos son proporciones, la recencia se mide en semanas y la estacionalidad se expresa como índice relativo. Para combinarlas en un único *score*, todas las señales se transformaron a una escala común entre 0 y 1.

La normalización se realizó cuidando que valores mayores siempre representaran mayor conveniencia de recomendación. En tendencia, volumen y estacionalidad, valores más altos implican directamente mayor relevancia. En recencia, la dirección es inversa: un menor número de semanas desde la última venta debe transformarse en un mayor *recency_score*. Por ello, la recencia se normaliza de tal forma que productos vendidos recientemente obtengan valores más altos.

De esta forma, cada producto queda representado por cuatro señales principales:

$$trend_score(p), \quad recency_score(p), \quad volume_score(p), \quad seasonality_score(p).$$

Estas señales constituyen la base del *score* temporal final.

5.3.5. Construcción del *score*_TS

El *score* temporal final se definió como una combinación ponderada de las cuatro señales normalizadas:

$$score_{TS}(p) = 0,40 \cdot trend_score(p) + 0,25 \cdot recency_score(p) + 0,20 \cdot volume_score(p) + 0,15 \cdot seasonality_score(p).$$

Los pesos fueron definidos como una primera aproximación funcional de acuerdo con el objetivo del módulo. La tendencia recibe el mayor peso porque el componente TS busca identificar productos con comportamiento reciente favorable. La recencia recibe el segundo peso más alto porque permite evitar productos inactivos. El volumen se incluye para reducir el impacto de señales débiles basadas en pocas observaciones, mientras que la estacionalidad se integra como una señal complementaria de oportunidad temporal.

El resultado es una variable continua entre 0 y 1. Un valor alto de *score*_TS no significa necesariamente que el producto sea el más vendido del catálogo, sino que combina características de oportunidad temporal: crecimiento reciente, actividad actual, volumen suficiente y/o comportamiento favorable en el periodo de análisis.

Componente	Peso	Interpretación
<code>trend_score</code>	40 %	Crecimiento reciente y dirección de demanda
<code>recency_score</code>	25 %	Actividad reciente del producto
<code>volume_score</code>	20 %	Suficiencia de volumen comercial
<code>seasonality_score</code>	15 %	Relevancia temporal del periodo actual

Tabla 24: Componentes utilizados para construir el *score* de Series de Tiempo.

5.3.6. Integración de inventario

Después de calcular el *score* temporal, se integró información de inventario para agregar una restricción operativa al sistema de recomendación. Esta información proviene de las tablas de existencias y permite identificar si un SKU cuenta con disponibilidad comercial.

La variable principal generada en esta etapa es `has_stock`, definida como:

$$\text{has_stock}(p) = \begin{cases} 1, & \text{si la existencia total del producto es positiva,} \\ 0, & \text{en caso contrario.} \end{cases}$$

Esta variable no forma parte directa de la fórmula base de `score_TS`; sin embargo, es fundamental para la capa de negocio, ya que un producto no debería mostrarse como recomendación final si no tiene inventario disponible. Por lo tanto, `has_stock` se conserva como una bandera operativa dentro de `ts_scores_ana03.csv`.

5.3.7. Archivos generados

El componente TS genera dos archivos principales: `weekly_series_ana03.csv` y `ts_scores_ana03.csv`. El primero conserva el historial semanal por producto, mientras que el segundo resume dicho historial en una fila por SKU.

`weekly_series_ana03.csv` tiene una granularidad producto-semana y se utiliza para validar visualmente el comportamiento temporal de los productos, analizar casos específicos y servir como insumo para pruebas estadísticas o modelos temporales posteriores.

`ts_scores_ana03.csv` tiene una granularidad producto y constituye la salida modelable del componente temporal. Sus columnas principales incluyen:

- `cve_art`: identificador del SKU.
- `trend_qty_4w`, `trend_sales_4w` y `trend_qty_8w`: métricas de crecimiento reciente.
- `slope_qty_12w`: pendiente de demanda en las últimas 12 semanas.
- `weeks_since_last_sale`: semanas desde la última venta.
- `recency_score`: señal normalizada de recencia.
- `trend_score`: señal normalizada de tendencia.
- `volume_score`: señal normalizada de volumen.
- `seasonality_score`: señal normalizada de estacionalidad.
- `score_TS`: *score* temporal final.
- `existencia_total`: existencia total agregada del producto.
- `has_stock`: indicador de disponibilidad.

Desde la perspectiva del pipeline completo, `ts_scores_ana03.csv` funciona como una tabla de enriquecimiento. Cuando los módulos de TDA, RNN o Transformer generan productos candidatos, estos pueden unirse con las variables temporales mediante la clave `cve_art`. Conceptualmente, la operación es:

$$\text{candidatos enriquecidos} = \text{candidatos} \bowtie \text{ts_scores_ana03.csv}.$$

Así, cada producto candidato queda acompañado de su `score_TS` y de variables auxiliares como `has_stock`. Esta información puede utilizarse tanto en la fusión temprana del Transformer como en reglas de negocio previas a la presentación final de recomendaciones.

5.3.8. Uso dentro del Transformer de fusión temprana

En el modelo final, el *score* temporal se utiliza como una de las señales exógenas del producto. Para cada SKU p , el vector de señales externas se representa como:

$$\mathbf{s}_p = \begin{bmatrix} s_{\text{TDA}}(p) \\ s_{\text{TS}}(p) \\ s_{\text{RNN}}(p) \end{bmatrix}.$$

Este vector se proyecta mediante una red densa y se concatena con el *embedding* de identidad del producto. La explicación detallada de la arquitectura del Transformer se presenta en la Sección 6; sin embargo, la intuición es que `score_TS` permite que el modelo distinga entre productos que pueden tener relaciones de canasta similares, pero diferente oportunidad temporal.

Por ejemplo, dos productos pueden tener una afinidad estructural o secuencial comparable con una canasta dada. Sin embargo, si uno de ellos presenta ventas recientes, tendencia positiva, volumen suficiente y stock disponible, mientras que el otro está inactivo o fuera de temporada, el componente TS aporta información para que el modelo favorezca al primero.

En síntesis, el componente de Series de Tiempo transforma el historial semanal de ventas en una señal cuantitativa de oportunidad temporal. Su aportación al sistema de recomendación es permitir que las sugerencias no dependan únicamente de relaciones históricas entre productos, sino también del contexto comercial actual. De esta manera, el recomendador final puede priorizar productos que no solo tienen sentido dentro de la canasta, sino que además conviene ofrecer en el momento de la recomendación.

5.4. Red Neuronal Recurrente (RNN)

El componente de Red Neuronal Recurrente (RNN) se desarrolló para incorporar al sistema de recomendación una señal conductual por producto derivada de los patrones observados en canastas históricas. Su función dentro del pipeline es capturar la probabilidad condicional de que un producto aparezca en una canasta dado el contexto parcial de productos ya presentes en ella. A diferencia del componente TS, que cuantifica la oportunidad temporal de un SKU, el componente RNN aprende directamente de la frecuencia y el orden relativo de las compras pasadas.

5.4.1. Entrada y construcción de canastas

La unidad mínima de análisis es la canasta, definida como el conjunto de artículos presentes en un mismo ticket. Para cada ticket se construyó un identificador único a partir de los campos `Suc`, `FecOpe`, `Caja` y `NumTra`, agrupando todos sus artículos válidos en una secuencia:

$$B = (p_1, p_2, \dots, p_n).$$

Se conservaron únicamente canastas con tres o más artículos, ya que canastas de uno o dos productos no generan suficiente contexto relacional para el entrenamiento. Después de este filtrado, la base contiene 437,488 canastas con una media de 8.3 artículos por ticket.

5.4.2. Vocabulario y tokenización

El modelo opera sobre índices enteros en lugar de claves de artículo originales. El vocabulario se construyó seleccionando los $K = 3,000$ artículos más frecuentes del conjunto de canastas, lo que garantiza un número fijo de clases en la capa de salida. Se definieron dos tokens especiales: $\langle \text{PAD} \rangle$ (índice 0) para igualar longitudes dentro de un batch, y $\langle \text{UNK} \rangle$ (índice 1) para artículos fuera del top- K . El vocabulario final contiene 3,002 tokens y cubre aproximadamente el 80–90 % de todas las ocurrencias en los tickets. Los artículos fuera del vocabulario reciben $score_{\text{RNN}} = 0$.

5.4.3. Generación de secuencias de entrenamiento

A partir de las canastas tokenizadas se generaron pares de entrenamiento mediante una ventana deslizante. Para cada canasta $(p_1, p_2, \dots, p_n)$ se producen $n - 2$ pares de la forma:

$$((p_1, p_2), p_3), ((p_1, p_2, p_3), p_4), \dots, ((p_1, \dots, p_{n-1}), p_n).$$

Se exigió un contexto mínimo de dos artículos porque con un único producto no existe información relacional que el modelo pueda aprovechar. Los artículos codificados como $\langle \text{UNK} \rangle$ se eliminan tanto del contexto como del objetivo. El conjunto de pares se dividió en 85 % para entrenamiento y 15 % para validación con semilla fija.

5.4.4. Arquitectura del modelo

Se implementó una Red Neuronal Recurrente de tipo GRU (*Gated Recurrent Unit*). Se eligió GRU sobre LSTM porque ofrece rendimiento equivalente para secuencias de longitud moderada con aproximadamente 25 % menos parámetros y menor tiempo de entrenamiento. Dado que las canastas tienen una longitud media de 8.3 artículos, las dependencias de largo plazo propias del LSTM no representan una ventaja significativa en este caso.

Sea $\mathbf{e}_{p_i} \in \mathbb{R}^{d_e}$ el vector de *embedding* del artículo p_i , con $d_e = 64$. La secuencia de *embeddings* se alimenta a una capa GRU con dimensión oculta $d_h = 128$:

$$\mathbf{h}_t = \text{GRU}(\mathbf{e}_{p_1}, \dots, \mathbf{e}_{p_t}; \theta_{\text{GRU}}),$$

donde $\mathbf{h}_t \in \mathbb{R}^{d_h}$ es el estado oculto final que captura la representación contextual acumulada de los artículos vistos hasta el momento t . Para evitar que el modelo procese tokens de relleno durante el cómputo recurrente, se utilizó la técnica *pack_padded_sequence*, que comprime las secuencias eliminando los tokens $\langle \text{PAD} \rangle$ antes de la capa GRU. Esto mejora la precisión del estado oculto final y reduce el cómputo innecesario.

Tras extraer el estado oculto, se aplica normalización BatchNorm y una proyección lineal al espacio de productos:

$$\hat{y} = W \text{BN}(\mathbf{h}_t) + b, \quad \hat{y} \in \mathbb{R}^{|V|}.$$

Aplicando *softmax* sobre $\hat{y}$ se obtiene la distribución de probabilidad sobre el siguiente artículo:

$$P(p \mid p_1, \dots, p_t) = \text{softmax}(\hat{y})_p.$$

5.4.5. Entrenamiento

El modelo se entrenó minimizando la entropía cruzada con suavizado de etiquetas (*label smoothing* = 0,05), que distribuye una fracción de la masa de probabilidad entre todas las clases del vocabulario y actúa como regularización suave en vocabularios grandes. Se utilizó el optimizador AdamW con $\eta = 10^{-3}$ y decaimiento de pesos $\lambda = 10^{-4}$, con programación de tasa de aprendizaje *OneCycleLR* y *gradient clipping* de norma máxima igual a 1 para prevenir la explosión de gradientes. El entrenamiento se ejecutó durante un máximo de 10 épocas con parada anticipada de 3 épocas sin mejora en la pérdida de validación.

5.4.6. Métricas de evaluación

La evaluación se realizó sobre el conjunto de validación con Hit Rate @K y Mean Reciprocal Rank (MRR):

$$\text{Hit@K} = \frac{1}{|D_{val}|} \sum_{(B_t, p^*) \in D_{val}} \mathbf{1}[p^* \in \text{Top-}K(P(\cdot | B_t))],$$

$$\text{MRR} = \frac{1}{|D_{val}|} \sum_{(B_t, p^*) \in D_{val}} \frac{1}{\text{rank}(p^* | B_t)}.$$

Los resultados obtenidos se presentan en la Tabla 25.

Métrica	Valor
val_loss	6.0013
Hit@1	12.39 %
Hit@5	24.26 %
Hit@10	31.20 %
MRR	0.1755

Tabla 25: Resultados del modelo GRU sobre el conjunto de validación.

Con un vocabulario de 3,000 artículos, una predicción aleatoria produciría un Hit@10 de aproximadamente 0.33 %. El modelo alcanza 31.20 %, lo que representa una mejora de aproximadamente 94 veces sobre el nivel aleatorio y 3 veces sobre un *baseline* de frecuencia marginal. La cercanía entre pérdida de entrenamiento (6.05) y pérdida de validación (6.00) indica ausencia de sobreajuste relevante.

5.4.7. Construcción del *score_RNN*

El *score RNN* se construye a partir del comportamiento del modelo durante la evaluación sobre el conjunto de validación. Para cada artículo p del vocabulario se calcularon tres señales sobre las muestras en que p fue el artículo objetivo:

- **prob_raw**: probabilidad softmax promedio asignada al artículo p .
- **hit_rate_como_target**: fracción de muestras en que el modelo predijo p correctamente en posición 1.
- **top5_rate**: fracción de muestras en que p apareció entre las 5 predicciones con mayor probabilidad.

Las tres señales se normalizaron de forma robusta al intervalo $[0, 1]$ mediante winsorización en los percentiles 2 y 98. El *score* final se definió como:

$$score_{\text{RNN}}(p) = 0,40 \cdot prob_norm(p) + 0,35 \cdot hitrate_norm(p) + 0,25 \cdot top5_norm(p).$$

Componente	Peso	Interpretación
prob_norm	40 %	Probabilidad directa asignada por el modelo
hitrate_norm	35 %	Consistencia del acierto exacto Top-1
top5_norm	25 %	Frecuencia de aparición en las primeras posiciones

Tabla 26: Componentes utilizados para construir el *score* RNN.

El resultado es una variable continua entre 0 y 1 almacenada en `rnn_scores.csv`, con una fila por artículo. Esta tabla funciona como insumo directo del Transformer de fusión temprana, junto con `ts_scores_ana03.csv` y los *scores* de TDA, enriqueciendo la representación de cada SKU con su señal de comportamiento secuencial histórico.

6. Modelo

6.1. Implementación

El modelo final de recomendación corresponde a un **Transformer de fusión temprana** (EarlyFusionTransformer), implementado en PyTorch. El problema se formuló como una tarea de *predicción del siguiente artículo* (*next-item prediction*): dada una canasta parcial de productos, el modelo estima un ranking de SKUs candidatos que podrían complementar la compra.

Formalmente, una canasta parcial se representa como una secuencia de productos:

$$B_t = (p_1, p_2, \dots, p_t),$$

donde cada p_i corresponde a un SKU observado dentro del ticket. El objetivo del modelo es estimar la probabilidad de que un producto p sea el siguiente artículo agregado a la canasta:

$$P(p \mid B_t).$$

La arquitectura utiliza una estrategia de **fusión temprana** porque combina, desde la entrada del modelo, dos tipos de información para cada producto de la secuencia:

- un **embedding de identidad** del producto, generado mediante `nn.Embedding`, que permite representar cada SKU como un vector denso aprendible;
- una **proyección de señales externas**, construida a partir de tres *scores* por producto: RNN, TDA y Series de Tiempo.

Para cada producto p , el modelo recibe un vector de señales externas:

$$\mathbf{s}_p = \begin{bmatrix} s_{\text{RNN}}(p) \\ s_{\text{TDA}}(p) \\ s_{\text{TS}}(p) \end{bmatrix}.$$

Este vector se transforma mediante una pequeña red densa compuesta por capas lineales y una activación ReLU:

$$\mathbf{q}_p = f(\mathbf{s}_p).$$

Posteriormente, la representación proyectada de los *scores* se concatena con el embedding de identidad del producto:

$$\mathbf{z}_p = [\mathbf{e}_p; \mathbf{q}_p],$$

donde $\mathbf{e}_p$ representa el embedding del SKU y $\mathbf{q}_p$ representa la proyección aprendida de sus señales externas. Esta concatenación permite que el Transformer no dependa únicamente del identificador del producto, sino que también utilice información precalculada sobre comportamiento secuencial, estructura topológica y oportunidad temporal.

A la representación enriquecida de cada producto se le agrega un embedding posicional, lo que permite al modelo distinguir la posición relativa de los artículos dentro de la secuencia. Después, la secuencia enriquecida se procesa mediante un **TransformerEncoder**. La representación contextual de la última posición se utiliza como resumen de la canasta parcial y se proyecta, mediante una capa lineal de salida, al espacio completo del vocabulario de productos.

La salida del modelo es un vector de puntuaciones sobre todos los SKUs candidatos. Después de aplicar *softmax*, estas puntuaciones pueden interpretarse como una distribución de probabilidad sobre el siguiente producto posible:

$$P(p \mid B_t) = \text{softmax}(g(\mathbf{h}_t)),$$

donde $\mathbf{h}_t$ representa la codificación contextual generada por el Transformer y $g(\cdot)$ es la capa de salida que proyecta dicha representación hacia el vocabulario de SKUs.

La configuración final de la arquitectura seleccionada fue la siguiente:

Elemento	Configuración final
Arquitectura	EarlyFusionTransformer
Tipo de tarea	<i>Next-item prediction</i>
Framework	PyTorch
Dimensión de embedding	128
Número de cabezas de atención	8
Número de capas del encoder	2
Longitud máxima de secuencia	10
Tamaño de lote	128
Tasa de aprendizaje	0.000794
Vocabulario de productos	3,000 SKUs + token de padding

Tabla 27: Configuración final del modelo Transformer de fusión temprana.

El modelo fue entrenado minimizando la pérdida de entropía cruzada (**CrossEntropyLoss**), comparando la distribución predicha por el Transformer contra el producto objetivo observado en los datos históricos. Para construir los ejemplos de entrenamiento, las canastas históricas se transformaron en pares de entrada y objetivo, donde una canasta parcial sirve como contexto y el siguiente producto funciona como etiqueta supervisada.

Una decisión importante de implementación fue el tratamiento de productos sin alguno de los *scores* externos. Cuando un SKU no contaba con señal RNN, TDA o TS disponible, el valor faltante se imputó como cero. Esta decisión permitió mantener un vocabulario común para el Transformer sin eliminar productos por falta de información en alguno de los componentes. No obstante, esta imputación debe interpretarse como una limitación del modelo: un cero puede representar ausencia de evidencia y no necesariamente una evaluación negativa del producto. Por ello, en futuras iteraciones podría explorarse una máscara adicional que distinga entre *score* real igual a cero y *score* faltante imputado.

La elección de un Transformer de fusión temprana se justifica por la naturaleza del problema. Las canastas de compra no siempre siguen una secuencia estrictamente temporal como una frase, pero sí contienen relaciones complejas entre productos. El mecanismo de atención permite que el modelo aprenda qué artículos de la canasta son más relevantes para predecir el siguiente producto, mientras que la fusión temprana permite incorporar señales externas desde

las primeras capas de representación. De esta forma, el modelo combina identidad del producto, patrones históricos de canasta, estructura topológica del catálogo y comportamiento temporal reciente en una sola arquitectura predictiva.

6.2. Refinamiento

Una vez definida la arquitectura base del `EarlyFusionTransformer`, se realizó una etapa de refinamiento enfocada en la optimización de hiperparámetros. El objetivo no fue modificar la lógica general del modelo ni cambiar la forma en que se integran los *scores* de RNN, TDA y Series de Tiempo, sino identificar una configuración más adecuada para entrenar el mismo Transformer de fusión temprana.

El refinamiento se llevó a cabo con **Optuna**, utilizando el muestreador TPE (*Tree-structured Parzen Estimator*) y un podador **MedianPruner**. Esta combinación permitió explorar configuraciones candidatas y detener de forma anticipada aquellas que no mostraban un desempeño competitivo durante las primeras épocas.

La métrica objetivo fue el **MRR@10 de validación** (*Mean Reciprocal Rank*), ya que esta métrica no solo evalúa si el producto correcto aparece dentro del ranking, sino también qué tan alta es su posición. Para el caso de negocio, esto es relevante porque una recomendación ubicada en las primeras posiciones tiene mayor probabilidad de ser vista y utilizada por el vendedor o el cliente.

El experimento final de refinamiento se realizó con el archivo actualizado `rnn_item_scores.parquet`. A partir de este archivo se construyó el vocabulario del modelo, compuesto por 3,000 SKUs reales y un token adicional de *padding*, dando una dimensión total de vocabulario de 3,001. Para estos productos se integraron los tres *scores* utilizados por el Transformer: RNN, TDA y Series de Tiempo.

Con este vocabulario se generaron 532,889 ejemplos supervisados de canasta. La partición de datos se mantuvo igual que en el modelo base: 80 % para entrenamiento y 20 % para validación, con semilla fija para asegurar comparabilidad. Esto produjo 426,311 ejemplos de entrenamiento y 106,578 ejemplos de validación.

El espacio de búsqueda se definió sobre hiperparámetros propios del Transformer y del proceso de entrenamiento: dimensión de embedding, número de cabezas de atención, número de capas del encoder, tamaño de lote, tasa de aprendizaje y longitud máxima de secuencia.

Hiperparámetro	Valores explorados
EMBED_DIM	{32, 64, 128}
N_HEADS	{2, 4, 8}
N_LAYERS	1 a 3
BATCH_SIZE	{64, 128, 256}
LR	$[10^{-4}, 3 \times 10^{-3}]$ en escala logarítmica
MAX_LEN	{5, 10, 15}

Tabla 28: Espacio de búsqueda utilizado durante la optimización con Optuna.

El proceso se realizó en dos etapas. Primero se ejecutó una búsqueda exploratoria con 20 *trials*, entrenando cada configuración durante tres épocas. Esta etapa permitió identificar combinaciones prometedoras sin incurrir en el costo de entrenar todos los modelos durante 15 épocas completas. La configuración original se mantuvo como *baseline* para comparar las configuraciones sugeridas por Optuna contra el punto de partida del modelo.

En la búsqueda exploratoria, la mejor configuración fue `optuna_trial_18`, con un MRR de validación de 0.184341 después de tres épocas. Esta configuración utilizó una dimensión de embedding de 128, ocho cabezas de atención, dos capas en el encoder, tamaño de lote de 128, longitud máxima de secuencia de 10 y tasa de aprendizaje de 0.000794.

Config.	MRR	Hit@10	Embed	Heads	Capas	Batch	LR
Baseline	0.172096	29.23 %	64	4	2	128	0.001000
Trial 18	0.184341	30.78 %	128	8	2	128	0.000794
Trial 17	0.183635	30.77 %	128	8	2	128	0.000718
Trial 14	0.182680	30.30 %	128	8	1	128	0.000607

Tabla 29: Principales configuraciones identificadas durante la búsqueda exploratoria de Optuna.

Después de la etapa exploratoria, se seleccionaron el baseline original y tres configuraciones candidatas para una validación final con entrenamiento completo a 15 épocas. Esta segunda etapa fue necesaria porque el desempeño con pocas épocas puede favorecer configuraciones que aprenden rápido al inicio, pero que no necesariamente mantienen el mejor resultado durante un entrenamiento completo. Por ello, cada candidato fue reentrenado desde cero y se conservó el checkpoint correspondiente a la época con mayor MRR de validación.

La validación final confirmó nuevamente a `optuna_trial_18` como el mejor modelo bajo el criterio de MRR@10. Aunque `optuna_trial_17` obtuvo un Hit@10 ligeramente mayor, el criterio de selección definido fue MRR@10, por lo que `trial_18` fue seleccionado como configuración final.

Modelo	MRR@10	Hit@1	Hit@5	Hit@10	Época
<code>optuna_trial_18</code>	0.188645	13.92 %	25.35 %	31.42 %	6
<code>optuna_trial_17</code>	0.188329	13.82 %	25.40 %	31.48 %	6
Baseline original	0.187640	13.87 %	25.17 %	31.20 %	13
<code>optuna_trial_14</code>	0.185970	13.75 %	24.94 %	30.87 %	6

Tabla 30: Validación final de los candidatos reentrenados durante 15 épocas.

La configuración final seleccionada se resume en la Tabla 31. Respecto al baseline, el principal cambio fue aumentar la dimensión de representación de 64 a 128 y el número de cabezas de atención de 4 a 8. El número de capas, el tamaño de lote y la longitud máxima de secuencia se mantuvieron iguales. La tasa de aprendizaje seleccionada por Optuna fue ligeramente menor que la original, lo que permitió entrenar una arquitectura de mayor capacidad de forma más estable.

Hiperparámetro	Baseline	Modelo seleccionado
EMBED_DIM	64	128
N_HEADS	4	8
N_LAYERS	2	2
BATCH_SIZE	128	128
LR	0.001000	0.000794
MAX_LEN	10	10
EPOCHS	15	15

Tabla 31: Hiperparámetros finales seleccionados para el Transformer.

En comparación con el baseline reentrenado bajo las mismas condiciones, la configuración final aumentó el MRR@10 de 0.187640 a 0.188645. Esto representa una mejora absoluta de 0.001005 y una mejora relativa aproximada de 0.54 %. También se observaron incrementos moderados en las métricas top- k : Hit@1 pasó de 13.87 % a 13.92 %, Hit@5 de 25.17 % a 25.35 % y Hit@10 de 31.20 % a 31.42 %.

Aunque la mejora cuantitativa es moderada, el refinamiento permitió validar que una representación de mayor capacidad mejora ligeramente el ordenamiento de recomendaciones sin

alterar la arquitectura general del sistema. En particular, el resultado sugiere que el Transformer se beneficia de una mayor dimensión de embedding y de más cabezas de atención, lo cual es consistente con el objetivo de capturar relaciones heterogéneas entre productos dentro de la canasta.

Finalmente, se observó que los modelos optimizados alcanzaron su mejor desempeño antes de completar las 15 épocas, alrededor de la época 6. Como trabajo posterior, esta observación podría aprovecharse para incorporar *early stopping* formal o estrategias adicionales de regularización. Para la versión final del proyecto, se conservó el checkpoint de mayor MRR@10 de validación del `optuna_trial_18`.

6.3. Evaluación

Dado que el problema corresponde a recomendación y ordenamiento de productos, no se evaluó como una clasificación binaria tradicional. En su lugar, se utilizaron métricas de ranking: **MRR@10** y **Hit@K** para $K \in \{1, 5, 10\}$.

El **MRR@10** mide qué tan arriba aparece el producto correcto dentro del ranking de recomendaciones. Para cada ejemplo de validación, se calcula el inverso de la posición en la que aparece el producto real, considerando únicamente las primeras 10 posiciones:

$$MRR@10 = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i},$$

donde $rank_i$ es la posición del producto correcto para el ejemplo i . Si el producto no aparece dentro del top-10, su contribución se considera cero. Esta métrica fue priorizada porque una recomendación ubicada en las primeras posiciones tiene mayor valor operativo que una recomendación correcta ubicada al final de la lista.

La métrica **Hit@K** mide la proporción de ejemplos en los que el producto real aparece dentro de las primeras K recomendaciones:

$$Hit@K = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(rank_i \leq K).$$

En este proyecto, Hit@1 mide acierto exacto en la primera recomendación, Hit@5 mide si el producto correcto aparece dentro de las primeras cinco sugerencias y Hit@10 mide si aparece dentro de las diez recomendaciones que se entregarían al usuario final.

El modelo seleccionado alcanzó los siguientes resultados en validación:

Métrica	Resultado del modelo final
MRR@10	0.188645
Hit@1	13.92 %
Hit@5	25.35 %
Hit@10	31.42 %
Época del mejor checkpoint	6

Tabla 32: Métricas finales del modelo seleccionado `optuna_trial_18`.

Los resultados muestran que el modelo logra ubicar el producto correcto dentro del top-10 en 31.42 % de los casos de validación. Esto es relevante para el caso de uso ANA-03 porque la salida operativa del sistema consiste precisamente en una lista corta de productos sugeridos. En otras palabras, el modelo no necesita predecir un único producto perfecto, sino ordenar de forma útil un conjunto de candidatos que puedan ser ofrecidos como complemento de la canasta actual.

El MRR@10 de 0.188645 indica que, cuando el producto correcto aparece en el ranking, tiende a ubicarse en posiciones relativamente tempranas. Esto es importante porque el valor práctico de una recomendación disminuye conforme baja su posición en la lista: los primeros lugares son los más visibles y, por lo tanto, los más accionables para el vendedor o la interfaz.

La comparación contra el baseline muestra una mejora moderada pero consistente. El modelo optimizado incrementó el MRR@10 de 0.187640 a 0.188645 y el Hit@10 de 31.20 % a 31.42 %. Aunque la diferencia no transforma radicalmente el desempeño, sí valida que la optimización de hiperparámetros permitió mejorar el ordenamiento de candidatos sin cambiar la arquitectura ni las señales de entrada.

Desde una perspectiva técnica, estos resultados sugieren que la fusión temprana de señales RNN, TDA y TS aporta una representación más rica que un enfoque basado únicamente en identificadores de producto. La señal RNN incorpora patrones históricos de canasta, TDA aporta relaciones estructurales del catálogo y TS agrega información de oportunidad temporal. El Transformer aprende a combinar estas fuentes de acuerdo con el contexto de cada canasta parcial.

Finalmente, es importante señalar que estas métricas corresponden a una evaluación offline. Por lo tanto, no deben interpretarse directamente como tasa de conversión real en tienda. La validación final del impacto comercial requeriría una prueba en producción, idealmente mediante A/B testing, midiendo adopción del vendedor, aceptación del cliente, incremento en artículos por ticket y variación en ticket promedio.

7. Implementación Técnica del Servicio

7.1. Cómputo en la Nube

La solución de inferencia del proyecto ANA-03 fue diseñada como un servicio desacoplado del ERP TCA Merksyst. En lugar de integrar el modelo directamente dentro del sistema transaccional, se construyó una API REST independiente, empaquetada en Docker y desplegada en Microsoft Azure. Esta decisión permite que el recomendador pueda ser consumido desde distintas interfaces del negocio, como una pantalla del ERP, un punto de venta o la aplicación web desarrollada para el proyecto.

7.1.1. Extracción y carga de datos

La extracción de datos para el desarrollo del modelo se realizó a partir de archivos Parquet proporcionados por TCA Software Solutions. Estos archivos representan tablas operativas del ERP Merksyst y fueron trabajados en un entorno controlado del equipo. Por confidencialidad, los archivos Parquet originales no se incluyeron en ningún repositorio público; todo se manejó de forma privada.

Las principales tablas utilizadas en la extracción fueron:

- **pdvhdr**: encabezado de tickets de punto de venta;
- **pdvdet**: detalle de productos vendidos dentro de cada ticket;
- **inviar**: catálogo maestro de artículos;
- **invars**: existencias por artículo y almacén;
- **invart**: precios, costos e información comercial por artículo.

La carga inicial se realizó directamente desde los archivos Parquet hacia notebooks de Python, utilizando herramientas como **pandas** y consultas tipo SQL mediante motores analíticos

locales. Esta estrategia fue adecuada para el alcance académico del proyecto porque permitió procesar datos tabulares de gran volumen sin requerir una base de datos transaccional adicional.

El flujo de extracción y carga puede resumirse en las siguientes etapas:

1. lectura local de archivos Parquet del ERP;
2. selección de columnas relevantes para el caso ANA-03;
3. estandarización de tipos de dato y limpieza de campos clave;
4. unión entre encabezado y detalle de venta para reconstruir tickets;
5. integración con catálogo, precios e inventario;
6. generación de datasets derivados para TDA, Series de Tiempo, RNN y Transformer;
7. serialización de los artefactos finales requeridos por la API.

La carga hacia la nube no consistió en subir los datos crudos del ERP, sino en publicar un contenedor autocontenido con el modelo y sus artefactos de inferencia. Esta decisión reduce el riesgo de exponer información transaccional sensible y simplifica el despliegue, ya que el servicio solo necesita cargar archivos serializados al iniciar.

Los artefactos cargados dentro del contenedor fueron:

- `transformer_fusion.pt`: pesos entrenados del modelo Transformer;
- `transformer_vocab.pkl`: vocabulario de productos y mapeo SKU-índice;
- `catalogo.pkl`: catálogo serializado con descripciones y precios;
- `scores_dict.pkl`: diccionario de *scores* por producto, integrando RNN, TDA y Series de Tiempo.

En un escenario productivo, la arquitectura podría extenderse incorporando una zona de datos en *Azure Blob Storage* o *Azure Data Lake Storage*, donde se depositen periódicamente las extracciones del ERP. Posteriormente, un proceso programado podría ejecutar la limpieza, generación de *scores*, reentrenamiento y actualización del contenedor. Sin embargo, para la versión actual del proyecto, la inferencia opera de forma estática sobre los artefactos ya entrenados y empaquetados.

7.1.2. Infraestructura de despliegue del modelo en la nube (API)

El servicio de inferencia se empaquetó como una imagen de contenedor Docker y se desplegó sobre **Azure Container Apps**, utilizando **Azure Container Registry (ACR)** como repositorio privado de imágenes.

Para acceso rápido a ver el API publicado funcionando, puede acceder a:

<https://reco-basket-app.gentlesea-cb2510e3.eastus.azurecontainerapps.io/view>

La carpeta API del repositorio contiene todos los elementos necesarios para construir y desplegar el servicio:

Archivo	Función dentro del servicio
app.py	Aplicación FastAPI. Carga el modelo, reconstruye la arquitectura, sirve el endpoint de recomendación y expone una vista web de prueba.
Dockerfile	Define la imagen de contenedor, instala dependencias, copia artefactos y ejecuta una prueba de humo en tiempo de construcción.
requirements.txt	Lista dependencias de la API, principalmente fastapi , uvicorn y pandas .
transformer_fusion.pt	Pesos entrenados del Transformer de fusión temprana.
transformer_vocab.pkl	Diccionario de vocabulario que traduce códigos de producto a índices del modelo.
catalogo.pkl	Catálogo utilizado para enriquecer la respuesta con descripción y precio.
scores_dict.pkl	Diccionario de señales RNN, TDA y TS por producto.

Tabla 33: Componentes principales de la carpeta API.

La API fue implementada con **FastAPI** y expone cuatro superficies principales:

Ruta	Propósito
/	Endpoint de salud. Verifica que el servicio esté activo y devuelve el número de productos en vocabulario.
/docs	Documentación interactiva generada automáticamente por FastAPI mediante Swagger UI.
/view	Vista web simple para probar recomendaciones desde el navegador.
POST /recommend	Endpoint principal de integración. Recibe una canasta de productos y devuelve el Top-10 recomendado.

Tabla 34: Endpoints disponibles en el servicio de inferencia.

El endpoint principal recibe un cuerpo JSON con una lista de uno a diez productos:

```
{
  "products": ["028253", "028593"]
}
```

La respuesta contiene la canasta recibida y una lista de recomendaciones con ranking, clave de artículo, descripción, precio, *score* relativo y salto frente al siguiente candidato:

```
{
  "canasta": ["028253", "028593"],
  "recomendaciones": [
    {
      "Ranking": 1,
      "CVE_Articulo": "028253",
      "Descripcin": "PASTELITO MINI GANSITO MARINELA 25 GR",
      "Precio": 8.9,
      "Score_Relativo": 1.0,
      "Salto_vs_Siguiente": 12.69
    }
  ]
}
```

La imagen Docker se construye sobre **python:3.12-slim**. Además de instalar las dependencias de la API, el **Dockerfile** instala la versión CPU de PyTorch y la dependencia de sistema **libgomp1**. Esta elección es suficiente para inferencia, ya que el servicio no requiere GPU para generar recomendaciones.

Un elemento importante del despliegue es la prueba de humo incluida dentro del **Dockerfile**. Durante la construcción de la imagen, se importa la aplicación, se carga el modelo y se ejecuta

una recomendación de ejemplo. Si alguno de estos pasos falla, la construcción de la imagen se detiene antes de publicar un contenedor defectuoso.

El flujo de despliegue se realizó en tres pasos:

1. construcción local de la imagen Docker;
2. publicación de la imagen en Azure Container Registry;
3. despliegue de la imagen en Azure Container Apps con ingreso externo en el puerto 8000.

El proceso de publicación y despliegue se resume conceptualmente de la siguiente forma:

```
az acr login --name acrrecobasket
docker tag reco-api acrrecobasket.azurecr.io/reco-api:latest
docker push acrrecobasket.azurecr.io/reco-api:latest

az containerapp up \
  --name reco-basket-app \
  --resource-group rg-reco-basket \
  --location eastus \
  --image acrrecobasket.azurecr.io/reco-api:latest \
  --ingress external \
  --target-port 8000 \
  --registry-server acrrecobasket.azurecr.io
```

La decisión de utilizar Azure Container Apps se tomó porque permite ejecutar contenedores serverless con escalamiento automático. Para el proyecto académico, esta alternativa fue más viable que un servicio de cómputo dedicado, ya que permite escalar a cero cuando no hay tráfico. La principal implicación operativa es el *cold start*: después de un periodo de inactividad, la primera solicitud puede tardar aproximadamente entre 10 y 20 segundos mientras el contenedor se reactiva y carga el modelo. Una vez inicializado, las respuestas posteriores son considerablemente más rápidas.

Desde el punto de vista de integración, la API tiene CORS habilitado para facilitar su consumo desde la aplicación web. En un entorno productivo, esta configuración debería restringirse a los dominios autorizados del ERP o del front-end oficial, y podría complementarse con autenticación, control de cuota y monitoreo de solicitudes.

7.2. Bases de Datos

La solución utiliza dos capas de almacenamiento con propósitos distintos. Durante el desarrollo analítico, la fuente principal fue un conjunto de tablas Parquet provenientes del ERP TCA Merksyst. Durante la inferencia, la API no consulta directamente una base de datos transaccional; en su lugar, consume artefactos serializados derivados del entrenamiento. Esta separación permite mantener la inferencia ligera y reproducible, mientras que la complejidad de transformación de datos queda concentrada en la etapa previa de preparación y modelado.

7.2.1. Filtrado de datos relevantes para el análisis

El filtrado de datos se enfocó en construir una base confiable para el caso ANA-03 Cross-selling. El objetivo fue conservar únicamente ventas válidas de punto de venta, excluir cancelaciones o líneas no activas y mantener productos con clave interpretable.

Las tablas centrales para este proceso fueron `pdvhdr` y `pdvdet`. La primera contiene el encabezado de cada ticket y la segunda contiene las líneas de artículos vendidos. La relación entre ambas se estableció mediante la llave compuesta:

$$\text{Suc} + \text{FecOpe} + \text{Caja} + \text{NumTra.}$$

A partir de los archivos Parquet disponibles, se identificaron 693,199 tickets en `pdvhdr` y 4,013,897 líneas en `pdvdet`. Las llaves de encabezado y detalle presentaron coincidencia completa a nivel de ticket, lo que permitió reconstruir las canastas sin pérdida de transacciones por falta de encabezado.

Los criterios de filtrado aplicados fueron:

- conservar operaciones de venta mediante `pdvhdr.Tipo = '00'`;
- conservar tickets con estatus válido, principalmente TI y TIF;
- conservar líneas activas del detalle mediante `pdvdet.status = '01'`;
- excluir registros con `cantidad <= 0`;
- excluir líneas sin clave de producto en `cve_art`;
- estandarizar claves como texto para evitar pérdida de ceros a la izquierda.

Después del filtrado, la base limpia quedó formada por 3,989,985 líneas de venta, 688,322 tickets válidos y 7,792 SKUs únicos. Esta base fue suficiente para generar canastas históricas, agregaciones semanales, señales por producto y datasets de entrenamiento para el Transformer.

Métrica	Valor
Tickets en <code>pdvhdr</code>	693,199
Líneas en <code>pdvdet</code>	4,013,897
Tickets válidos después de filtros	688,322
Líneas limpias después de filtros	3,989,985
SKUs únicos vendidos después de filtros	7,792
SKUs limpios con correspondencia en catálogo	7,792

Tabla 35: Resultado del filtrado de datos relevantes para ANA-03.

Además de las tablas transaccionales, se utilizaron tablas maestras para enriquecer y validar la base:

- **inviar**: permitió asociar cada SKU con descripción, línea, sublínea, familia, subfamilia, marca y estatus de artículo;
- **invars**: permitió integrar existencias por almacén y construir variables de disponibilidad;
- **invar**: permitió consultar precios, costos e información comercial del artículo.

Este filtrado no se ejecuta en tiempo real dentro de la API. Corresponde a la fase analítica y de entrenamiento. En producción, el servicio de inferencia ya recibe una canasta limpia desde la aplicación consumidora y únicamente consulta los artefactos serializados necesarios para generar el ranking de recomendaciones.

7.2.2. Uso de funciones agregadas y/o joins en las consultas

La preparación de datos utilizó operaciones equivalentes a consultas SQL sobre archivos Parquet. Estas operaciones incluyeron *joins* entre tablas, filtros por estatus, agregaciones por ticket, agregaciones por producto y agregaciones por semana.

La consulta base para reconstruir ventas válidas puede representarse de la siguiente forma:

```

SELECT
    h.Suc,
    h.FecOpe,
    h.Caja,
    h.NumTra,
    d.r_par,
    TRIM(d.cve_art) AS cve_art,
    d.des_art,
    d.cantidad,
    d.subtotal AS subtotal_linea,
    h.subtotal AS subtotal_ticket
FROM pdvdet d
INNER JOIN pdvhdr h
    ON d.Suc = h.Suc
    AND d.FecOpe = h.FecOpe
    AND d.Caja = h.Caja
    AND d.NumTra = h.NumTra
WHERE h.Tipo = '00'
    AND h.status IN ('TI', 'TIF')
    AND d.status = '01'
    AND d.cantidad > 0
    AND TRIM(d.cve_art) <> '';

```

A partir de esta base se construyeron canastas por ticket mediante funciones agregadas:

```

SELECT
    Suc,
    FecOpe,
    Caja,
    NumTra,
    COUNT(*) AS lineas_ticket,
    COUNT(DISTINCT cve_art) AS skus_unicos_ticket,
    SUM(cantidad) AS unidades_ticket,
    SUM(subtotal_linea) AS subtotal_ticket_calculado
FROM ventas_limpias
GROUP BY
    Suc, FecOpe, Caja, NumTra;

```

Esta agregación permitió medir la profundidad de canasta, identificar tickets con uno o más productos y generar las listas de SKUs utilizadas por los componentes RNN y Transformer.

Para el componente de Series de Tiempo, se agregaron ventas por SKU y semana:

```

SELECT
    cve_art,
    DATE_TRUNC('week', FecOpe) AS semana,
    SUM(cantidad) AS unidades_semana,
    SUM(subtotal_linea) AS venta_semana,
    COUNT(DISTINCT CONCAT(Suc, '-', FecOpe, '-', Caja, '-', NumTra)) AS tickets_semana
FROM ventas_limpias
GROUP BY
    cve_art,
    DATE_TRUNC('week', FecOpe);

```

Esta consulta permitió construir el panel producto-semana usado para calcular tendencia, recencia, volumen, estacionalidad y actividad reciente.

Para enriquecer productos con catálogo se utilizó un *join* entre la base limpia y *inviar*:

```

SELECT
    v.cve_art,
    c.des1 AS descripcion_catalogo,
    c.lin,
    c.s_lin,
    c.fam,

```

```

        c.s_fam,
        c.marca,
        c.status AS status_catalogo
FROM ventas_limpias v
LEFT JOIN invars c
    ON TRIM(v.cve_art) = TRIM(c.art);

```

Finalmente, para incorporar disponibilidad operativa se agregaron existencias desde **invars**:

```

SELECT
    TRIM(cve_art) AS cve_art,
    SUM(existencia) AS existencia_total,
    SUM(pedido) AS pedido_total,
    CASE
        WHEN SUM(existencia) > 0 THEN 1
        ELSE 0
    END AS has_stock
FROM invars
GROUP BY
    TRIM(cve_art);

```

Estas operaciones muestran que el proyecto sí utilizó estrategias de bases de datos y consultas analíticas, aunque no mediante un servidor SQL persistente en producción. La capa de datos se trabajó sobre archivos Parquet con lógica relacional: *joins*, filtros, agregaciones, agrupamientos, conteos distintos y generación de tablas derivadas. El resultado final de estas consultas alimentó los datasets de modelado y los artefactos serializados consumidos por la API.

7.3. MLOps

Para garantizar la reproducibilidad, la modularidad y el desacoplamiento de componentes, la arquitectura de ingeniería de datos y modelado se implementó utilizando la metodología de *data pipelines* mediante la herramienta **Kedro**.

El diseño del sistema rechaza las arquitecturas monolíticas tradicionales y adopta un enfoque orientado a componentes independientes. En lugar de procesar las fuentes transaccionales crudas en un único bloque, la solución bifurca el flujo de datos en tres pipelines paralelos especializados por técnica. Una vez que cada técnica computa e infiere sus resultados de manera aislada, el pipeline final del Transformer unifica los puntajes de cada técnica, entrena y hace una calificación final de los productos a recomendar a partir de una canasta parcial de productos.

A continuación, se detalla analíticamente la estructura jerárquica, los nodos y la lógica operativa de los pipelines divididos por técnica y fase.

7.3.1. Topological Data Analysis (TDA)

Limpieza de datos El sub-pipeline `data_cleaning_tda` prepara la información categórica y estructural optimizando la huella de memoria en tres nodos secuenciales:

- **clean_strings_dataframes_tda** (`_clean_strings_tda`): Aplica un formateo ligero e instrumental sobre las columnas clave de cruce (**Tipo**, **status**, **cve_art** y **art**), ejecutando casteos explícitos a texto y remoción de espacios en blanco mediante operaciones vectorizadas (`.str.strip()`) para evitar desalineaciones en las uniones relacionales.
- **merge_dataframes_tda** (`_join_data_tda`): Registra los *DataFrames* en **DuckDB** y ejecuta una consulta SQL optimizada mediante expresiones de tabla comunes (*CTEs*). Restringe las cabeceras a ventas normales (**Tipo** = '00') con estatus operativos ('TI', 'TIF', 'TIFD'), filtra detalles activos (**status** = '01') y realiza un **INNER JOIN** simétrico por ticket (Suc, FecOpe, Caja, NumTra), cruzando finalmente con el catálogo de artículos e inventarios.

- **clean_merged_dataframes_tda** (`_clean_data_join_tda`): Proyecta de manera estricta los atributos de la estructura jerárquica del producto (`cve_art`, `lin`, `fam`, `s_fam`), elimina redundancias mediante una remoción de registros duplicados y exporta la matriz base unificada.

Preprocesamiento de datos El pipeline `data_preprocessing_tda` se enfoca en vectorizar y proyectar las características categóricas dentro de un espacio de baja dimensionalidad a través de dos nodos clave:

- **encode_product_features_tda** (`_encode_product_features_tda`): Consolida un corpus textual unificado concatenando los valores únicos de las jerarquías de catálogo (`lin`, `fam` y `s_fam`). Instancia la clase `LabelEncoder` de *scikit-learn* para ajustar un mapeo entero global y transforma vectorialmente cada columna para exportar la matriz numérica indexada (`lin_encoded`, `fam_encoded` y `s_fam_encoded`).
- **mapper_projection_tda** (`_mapper_projection_tda`): Utiliza la librería especializada **KeplerMapper** (`km.KeplerMapper`) para configurar el espacio geométrico subyacente. Aplica un proceso secuencializado de reducción de dimensionalidad (*stacked projection*) combinando un embedding global no lineal **Isomap** de 3 componentes espaciales analizando 20 vecinos locales (`manifold.Isomap(n_components=3, n_neighbors=20)`), encadenado directamente con una proyección adaptativa de preservación de topología basada en **UMAP** (`umap.UMAP(n_components=2)`) para mapear los SKUs a un plano latente estricto de dos dimensiones.

Modelo El pipeline `tda` instrumenta la extracción de abstracciones geométricas relacionales a partir del espacio de menor dimensión calculado previamente empleando un nodo principal:

- **build_mapper_graph_tda** (`_build_mapper_graph_tda`): Configura y ejecuta el algoritmo *Mapper* estructurando la abstracción topológica del catálogo de productos. El sub-pipeline define un objeto hiper-cúbico de cobertura (`km.Cover`) parametrizado con un mallado de 7 cubos multidimensionales (`n_cubes=7`) y un factor de solapamiento geométrico del 30 % (`perc_overlap=0.3`). Sobre estas intersecciones, aplica un agrupamiento adaptativo basado en densidad mediante el algoritmo **DBSCAN** utilizando una métrica de distancia de coseno (`metric=cosine`), un umbral de proximidad de vecindad de valor unitario (`eps=1`) y una restricción de densidad mínima de 100 muestras (`min_samples=100`), generando un grafo simplicial complejo (`tda_graph`) que condensa las cercanías estructurales de las familias de artículos.

Puntaje El sub-pipeline `score_tda` instrumenta el motor de inferencia relacional del espacio geométrico para evaluar la similitud topológica de artículos externos frente a una canasta de consumo dada a través de tres nodos integrados:

- **extract_graph_nodes_tda** (`_extract_graph_nodes_tda`): Extrae las colecciones discretas de los nodos calculados por el algoritmo *Mapper* a partir del grafo simplicial estructurado (`tda_graph`), tipificando el mapeo para aislar las listas de identificadores enteros correspondientes a cada hiper-cubo geométrico.
- **compute_scores_tda** (`_compute_scores_tda`): Evalúa la proximidad latente entre una colección de productos objetivo (`params:tda_basket`) y el resto de los SKUs mapeados en la red. El nodo construye estructuras inversas de indexación y calcula el grado de asociación mediante una modificación estricta del índice de similitud de Jaccard. Si dos artículos

comparten hiper-cubos, el puntaje base se pondera sumando la intersección al coeficiente clásico:

$$S_{\text{Jaccard_mod}}(t, p) = |I_t \cap I_p| + \frac{|I_t \cap I_p|}{|I_t \cup I_p|} \quad (1)$$

En caso de no registrar intersección directa ($S_{\text{Jaccard_mod}} = 0$), el algoritmo inicializa una búsqueda en anchura (*BFS*) sobre la topología del grafo para deducir la distancia geodésica mínima (d) entre los nodos que contienen a los SKUs. Si la ruta es viable ($0 < d < 1000$), penaliza la fuerza asociativa de forma inversamente proporcional a la distancia; de lo contrario, la anula:

$$S_{\text{Topológico}}(t, p) = \begin{cases} S_{\text{Jaccard_mod}}(t, p) & \text{si } |I_t \cap I_p| > 0 \\ \frac{1}{d} & \text{si } |I_t \cap I_p| = 0 \wedge 0 < d < 1000 \\ 0,0 & \text{en otro caso} \end{cases} \quad (2)$$

Para canastas con múltiples productos objetivos, computa de manera iterativa los perfiles de proximidad y exporta el promedio aritmético de las afinidades agregadas, removiendo explícitamente los SKUs de entrada de la lista final de recomendaciones para mitigar la redundancia.

- **rank_products_tda** (`_rank_products_tda`): Convierte el diccionario relacional de similitudes en una estructura tabular, ordena los vectores de proximidad geométrico-topológica de forma descendente y cruza los índices espaciales del *Mapper* con las claves reales del catálogo (`cve_art`) para consolidar la matriz de ordenamiento (`tda_scores_ranked`).

7.3.2. Series de Tiempo (TS)

Limpieza de datos El sub-pipeline `data_cleaning_ts` extrae la regularidad transaccional y estructura un panel histórico agregando datos a nivel semanal mediante cuatro nodos enlazados:

- **validation_ts** (`_filter_data_ts`): Ejecuta consultas directas sobre los archivos crudos en Parquet usando DuckDB en memoria. Valida la consistencia de los datos filtrando por tipo de venta '00', estatus de cabecera 'TI'/'TIF', líneas de detalle activas y cantidades mayores a cero, retornando métricas globales de control operativo.
- **weekly_sku_ts** (`_weekly_sku_ts`): Replica las restricciones de integridad y trunca la dimensión temporal al inicio de la semana (`date_trunc('week', ...)`). Agrupa por artículo y semana para calcular los agregados de cantidad acumulada (`qty`), ventas monetarias (`sales`), conteo de tickets únicos y sucursales activas.
- **product_dim** (`_product_dim`): Extrae y limpia la dimensión de productos desde el catálogo, deduplicando por la clave del artículo (`cve_art`) para asegurar la integridad referencial de la dimensión.
- **weekly_sku_summary** (`_sku_summarize`): Agrupa el panel semanal por SKU e identifica la elegibilidad del producto aplicando tres reglas de negocio restrictivas encaminadas a eliminar series con alta intermitencia o ruido estocástico:

$$\text{Semanas Activas} \geq 4 \quad \wedge \quad \text{Tickets Totales} \geq 20 \quad \wedge \quad \text{Cantidad Total} \geq 20 \quad (3)$$

Preprocesamiento de datos El pipeline `data_preprocessing` de series de tiempo rectifica la estructura temporal del panel histórico e impone consistencia matricial y ventanas de corte mediante tres nodos en serie:

- **weekly_eligible_ts** (`_weekly_eligible_ts`): Ejecuta un filtrado indexado sobre la matriz temporal original conservando estrictamente las observaciones semanales asociadas a los artículos declarados viables en el resumen relacional (`weekly_sku_summary`).
- **weekly_full_ts** (`_weekly_full_ts`): Resuelve el problema de desalineación temporal y la ausencia de registros de venta. Genera una secuencia regular ininterrumpida de fechas (`freq="W-MON"`). Construye un índice cartesiano multidimensional usando la estructura `MultiIndex.from_product` entre los SKUs ordenados y las semanas sintéticas. Reindexa el set eligible rellenando de forma determinista con ceros (`.fillna(0)`) los registros vacíos, consolidando la matriz mediante un acoplamiento por la izquierda (`LEFT JOIN`) con la dimensión estática de artículos.
- **weekly_model_ts** (`_weekly_model_ts`): Establece la ventana analítica para mitigar el sesgo operacional de semanas inconclusas. Identifica dinámicamente la fecha máxima real e implementa un cálculo de umbral adaptativo (`cutoff_week`): si la última operación ocurre a mitad de semana (día de la semana ¡6), retrocede una semana completa en el tiempo mediante desplazamientos vectoriales (`pd.Timedelta`) para asegurar que el entrenamiento del modelo consuma exclusivamente intervalos con el ciclo transaccional completo de siete días.

Modelo El pipeline `timeseries` unifica la formulación analítica de la velocidad de movimiento de los SKUs y la inferencia predictiva multi-criterio empleando un nodo unificado:

- **build_ts_features** (`build_ts_features`): Construye y evalúa el modelo heurístico de series de tiempo a través de ventanas dinámicas y transformaciones estadísticas robustas. En primera instancia, establece ventanas temporales respecto a la fecha de corte (`cutoff_week`) aislando los periodos recientes frente a históricos (ventanas de 4 y 8 semanas) para computar volúmenes agregados de demanda (`qty`), facturación (`sales`) y recurrencia transaccional (`tickets`). A nivel dinámico, instrumenta la función de estimación lineal local `slope_last_n` que deduce la tasa de cambio de la demanda en las últimas 12 semanas aplicando una regresión polinomial por mínimos cuadrados ordinarios (`np.polyfit`):

$$m = \frac{\text{slope}}{\mu_{\text{qty}} + 10^{-6}} \quad (4)$$

Adicionalmente, computa un índice de estacionalidad mensual cruzando el comportamiento promedio del mes en curso contra el histórico global del artículo.

Para mitigar los efectos de valores atípicos operativos (*outliers*), el modelo define una normalización robusta `robust_minmax` mediante cortes por cuantiles extremos entre el 2 % y el 98 %:

$$x_{\text{scaled}} = \frac{x_{\text{clipped}} - \text{mín}(x_{\text{clipped}})}{\text{máx}(x_{\text{clipped}}) - \text{mín}(x_{\text{clipped}})} \quad (5)$$

Las dinámicas acelerativas se modelan a través de variaciones porcentuales logarítmicas acotadas (`safe_growth`) que evalúan el crecimiento reciente frente a su contraparte rezagada, mientras que la obsolescencia comercial se modela mediante un decaimiento exponencial estricto basado en la recencia temporal en días de la última transacción del SKU:

$$\text{Score Recencia} = \exp\left(-\frac{\Delta t_{\text{días}}}{28}\right) \quad (6)$$

Puntaje El pipeline `score_ts` ejecuta la jerarquización final de los artículos evaluados bajo la heurística predictiva estática mediante un único nodo especializado:

- **rank_products_ts** (`rank_products_ts`): Consume la tabla consolidada de características multivariadas (`ts_feature_table`) y extrae la métrica compuesta del modelo lineal-estacional robusto (`score_TS`). El nodo ejecuta un ordenamiento determinista en sentido descendente y genera un índice secuencial entero que formaliza el rango operativo del SKU (`ts_rank`), aislando las dinámicas comerciales prioritarias para la unificación del pipeline maestro.

7.3.3. Redes Neuronales Recurrentes (RNN)

Limpieza de datos El sub-pipeline `data_cleaning_rnn` transforma el esquema tabular de transacciones en colecciones ordenadas secuencialmente a través de cuatro nodos:

- **clean_rnn_data** (`_clean_rnn_data`): Estandariza las columnas de texto rellendo valores nulos y eliminando espacios vacíos. Convierte de forma explícita las cadenas operativas a formato cronológico de tipo `datetime64[ns]` de Pandas, controlando errores de casteo de forma estricta.
- **build_rnn_sales** (`_build_rnn_sales`): Filtra los tickets válidos con estatus 'TI' y líneas '01'. Calcula las unidades netas reales restando devoluciones (cantidad – devueltas) y descarta registros nulos o negativos. Construye además la clave sintética compuesta de transacción (`ticket_id`).
- **build_rnn_baskets_raw** (`_build_rnn_baskets_raw`): Agrupa los datos netos por `ticket_id` y aplica una agregación de lista sobre la clave de artículo, compactando los productos de un mismo ticket en un vector secuencial ordenado.
- **filter_rnn_baskets** (`_filter_rnn_baskets`): Evalúa la longitud de cada vector de compra y descarta los registros que no cumplan con el umbral mínimo de contexto transaccional:

$$\text{basket_size} \geq 3 \quad (7)$$

Preprocesamiento de datos El pipeline `data_preprocessing_rnn` transforma los vectores textuales de transacciones en estructuras tensoriales particionadas aptas para algoritmos auto-regresivos mediante cuatro nodos:

- **build_rnn_vocab** (`_build_rnn_vocab`): Aplana la colección de canastas y evalúa las frecuencias acumuladas utilizando la estructura eficiente `Counter`. Extrae los identificadores del catálogo restringiendo el vocabulario a los productos más comunes según un parámetro adaptativo (`vocab_size`). Inyecta explícitamente los tokens reservados de control e integridad estructural `<PAD>` y `<UNK>`, mapeando bidireccionalmente las colecciones en diccionarios numéricos inversos.
- **encode_rnn_baskets** (`_encode_rnn_baskets`): Sustituye vectorialmente los SKUs de las canastas de compra por sus respectivos índices discretos deducidos del vocabulario maestro, redirigiendo de manera nativa cualquier artículo imprevisto hacia el índice correspondiente del token no reconocido (`UNK_IDX`).
- **generate_rnn_sequences** (`_generate_rnn_sequences`): Aplica un método iterativo de ventana deslizante para segmentar las canastas indexadas previamente purgadas de tokens `<UNK>`. Genera sub-secuencias históricas de entrenamiento con un límite de contexto dinámico regulado por el hiperparámetro de longitud de secuencia máxima (`max_seq_len`). La ventana requiere un historial transaccional de al menos dos productos ($i \geq 2$) para consolidar el arreglo de características contextuales (`sequence`), aislando la posición subsiguiente como la etiqueta objetivo a aprender (`target`).

- **split_rnn_sequences** (`_split_rnn_sequences`): Instrumenta la separación estocástica del dataset recurrente importando el algoritmo balanceado `train_test_split`. Divide las colecciones secuenciales en dos subconjuntos disjuntos: datos de entrenamiento (`rnn_train_dataset`) y validación interna (`rnn_validation_dataset`) de acuerdo con el porcentaje delimitado por `test_size`, garantizando la reproducibilidad matemática mediante una semilla de inicialización aleatoria controlada (`random_state`).

Modelo El pipeline `model_rnn` orquesta la declaración estructural, inicialización, optimización matemática y tracking del modelo autoregresivo basado en aprendizaje profundo mediante el nodo principal:

- **train_and_eval_rnn_model** (`train_rnn_model`): Entrena una red neuronal recurrente profunda instrumentando la inicialización automática de **MLflow** para el registro y auditoría del ciclo de vida del modelo. La canalización encapsula el conjunto de datos mediante la abstracción `BasketDataset` de PyTorch y estructura un cargador asíncrono `DataLoader` acoplado a una función de agregación personalizada (`collate_fn`), la cual empaqueta dinámicamente las secuencias del lote a una dimensión homogénea mediante la inyección del token <PAD> a través de `pad_sequence(..., batch_first=True)`.

La arquitectura del modelo (`BasketGRU`) consta de una capa incrustada lineal inicializada bajo una distribución normal uniforme ($\mu = 0, \sigma = 0,01$), seguida de un bloque recurrente de unidades compuertas de retroalimentación **GRU** (`nn.GRU`) cuyos pesos de compuertas entrada-estado oculto (`weight_ih`) se inicializan mediante la técnica de **Xavier Uniforme** y los pesos estado-estado oculto (`weight_hh`) mediante una base ortogonal estricta (`nn.init.orthogonal_`). El flujo de tensores se optimiza aplicando empaquetamiento de secuencias de longitud variable mediante `pack_padded_sequence`, protegiendo las activaciones a través de capas de regularización `nn.Dropout(dropout)` y una capa de normalización por lotes unidimensional (`nn.BatchNorm1d`) justo antes de proyectar la clasificación final sobre el vocabulario mediante una capa densa (`nn.Linear`).

La función de pérdida implementa entropía cruzada con parametro de (`smoothing=0.05`), ignorando la influencia del token de relleno en el cálculo de gradientes. El proceso de optimización estocástica emplea el algoritmo **AdamW** acoplado a una política de tasa de aprendizaje dinámica basada en un ciclo único (`optim.lr_scheduler.OneCycleLR`) y una restricción estricta por norma para mitigar la explosión de gradientes mediante `clip_grad_norm_` a un umbral de 1.0. El bucle incorpora un criterio de parada temprana (*Early Stopping*) con una tolerancia de paciencia (`patience=3`) basada en el monitoreo estricto de la pérdida en validación, evaluando el desempeño predictivo en cada época bajo las métricas de clasificación multiclase *Hit@5*, *Hit@10* y el rango recíproco medio (*MRR*).

Puntaje El sub-pipeline `score_rnn` instrumenta la evaluación estadística fuera de muestra, el cálculo de métricas de diagnóstico agregadas y la estructuración del indicador de preferencia secuencial a través de dos nodos enlazados:

- **compute_rnn_item_metrics** (`compute_rnn_item_metrics`): Configura la evaluación asíncrona del modelo recurrente profundo sobre el dataset de validación interna empleando hardware optimizado (`cuda/mps`). El nodo procesa las sub-secuencias históricas, extrae la distribución logarítmica de verosimilitud de las predicciones e instrumenta operaciones tensoriales vectorizadas mediante `torch.softmax` para mapear las salidas a un espacio probabilístico estricto. Computa la probabilidad directa asignada al artículo objetivo real mediante operadores de recolección indexada (`probs.gather`) y evalúa la presencia de la etiqueta correcta dentro de las primeras cinco y diez fronteras de decisión (`torch.topk`), acumulando la frecuencia operacional del SKU como elemento de canasta y como objetivo subsecuente. Finalmente, calcula el promedio global de desempeño y registra en

MLflow las métricas maestras de control de la red: `mean_hit_rate`, `mean_top5_rate` y `mean_top10_rate`.

- **normalize_rnn_scores** (`normalize_rnn_scores`): Aplica la función de normalización robusta por cuantiles extremos (`robust_minmax` al 2 % y 98 %) sobre los estimadores brutos de verosimilitud y acierto acumulado. El nodo modela el indicador sintético final del componente recurrente (`score_RNN`) mediante una combinación lineal ponderada que prioriza la probabilidad neta del artículo sobre la consistencia posicional en los rangos top-k:

$$\text{Score}_{\text{RNN}} = 0,40 \cdot P_{\text{norm}} + 0,35 \cdot \text{HitRate}_{\text{norm}} + 0,25 \cdot \text{Top5}_{\text{norm}} \quad (8)$$

El algoritmo asigna un puntaje restrictivo de cero a cualquier artículo clasificado fuera de los límites de contingencia del vocabulario estricto (`~in_vocab`), asocia descripciones de catálogo mediante acoplamientos relacionales por la izquierda (`LEFT JOIN`) y ordena el dataset de manera descendente según la fuerza del score secuencial.

7.3.4. Transformer de Fusión Temprana (Transformer)

Limpieza de datos El sub-pipeline `data_cleaning_transformer` unifica las métricas de inferencia procedentes de los tres componentes base y estructura las secuencias históricas mediante tres nodos secuenciales:

- **build_vocab** (`_build_vocab`): Genera la estructura de indexación global tomando como base las transacciones registradas en `rnn_scores`. Extrae de forma unívoca las claves de artículo (`cve_art`), forzando su casteo a cadenas de texto operativas, y asigna a cada elemento un identificador entero correlativo correlacionado desde la unidad ($i + 1$). Asimismo, inyecta explícitamente el token reservado de control dimensional `<PAD>` asociado de manera determinista al índice cero (0) para consolidar el catálogo maestro `transformer_vocab`.
- **build_fusion** (`_build_fusion_scores`): Instrumenta el motor de alineación relacional inter-técnica. Extrae los pares clave-puntaje de `rnn_scores`, `tda_scores_ranked` y `ts_scores`, aplicando transformaciones vectoriales de limpieza de espacios en blanco mediante el método `.str.strip()`. Ejecuta acoplamientos sucesivos externos por la derecha e izquierda (`outer MERGE`) tomando como eje el identificador del SKU y subsana la asimetría dimensional imputando con valores nulos absolutos (0,0) las ausencias de score. Finalmente, aplica una normalización lineal robusta *Min-Max* sobre cada columna de afinidad:

$$S_{\text{técnica_norm}}(p) = \frac{S(p) - \min(S)}{\max(S) - \min(S)} \quad (9)$$

Mapea el universo consolidado contra los índices de `transformer_vocab`, remueve registros duplicados sobre el plano indexado y exporta la matriz unificada `df_fusion`.

- **build_transformer_baskets** (`_build_transformer_baskets`): Extrae y purga la regularidad secuencial de compra cruzando los históricos de cabeceras (`pdvhdr`) y detalles (`pdvdet`). El nodo restringe las transacciones a tickets cerrados válidos (`status = 'TI'`) y artículos activos (`status = '01'`), deduciendo las unidades netas reales mediante la sustracción de devoluciones (`cantidad - devueltas > 0`). Estructura la clave compuesta sintética `ticket_id`, ordena cronológicamente por la fecha de operación (`fecha_ope`) y compacta los SKUs en listas secuenciales. Aquellas canastas con una longitud de contexto transaccional viable (≥ 2) son segmentadas mediante una ventana deslizante de un único paso hacia adelante, aislando la sub-secuencia histórica de entrada frente a la última etiqueta objetivo:

$$X_{\text{sequence}} = (p_1, p_2, \dots, p_{n-1}) \longrightarrow Y_{\text{target}} = p_n \quad (10)$$

Preprocesamiento de datos El pipeline `data_preprocessing_transformer` fragmenta los conjuntos de datos, vectoriza las señales multidimensionales latentes y estructura las colecciones tensoriales de PyTorch a través de tres nodos de acoplamiento:

- **basket_split** (`_prepare_basket_splits`): Instrumenta la separación estocástica del dataset transaccional mediante el algoritmo balanceado `train_test_split`. Divide las colecciones secuenciales en subconjuntos disjuntos de entrenamiento y validación con base en la tasa delimitada por `params:transformer_test_size`, garantizando la replicabilidad de las particiones mediante la fijación estricta de una semilla pseudoaleatoria (`params:transformer_random_state`).
- **build_scores_dict** (`_build_scores_dict`): Transforma el dataframe unificado de fusiones (`df_fusion`) mapeando de forma indexada los enteros de los SKUs (`idx.art`) hacia sub-diccionarios locales balanceados que almacenan vectorialmente las tres señales normalizadas de los componentes previos:

$$\mathbf{s}_i = [S_{\text{RNN}}(i), S_{\text{TDA}}(i), S_{\text{TS}}(i)]^T \quad (11)$$

Esta estructura optimizada se almacena bajo el catálogo intermedio `scores_dict` para agilizar los accesos aleatorios en tiempo de ejecución.

- **build_data loaders** (`_build_data loaders`): Instancia la clase `EnrichedBasketDataset` y configura cargadores asíncronos distribuidos (`DataLoader`) gobernados por el tamaño de lote `params:transformer_batch_size`. El dataset ejecuta dinámicamente un parseo literal de secuencias mediante `ast.literal_eval` y mapea los SKUs a sus enteros correspondientes de vocabulario.

Para dar consistencia matricial a lotes heterogéneos, aplica una operación determinista de truncamiento y relleno (*padding*) hacia la izquierda fijada por el hiperparámetro de holgura temporal máxima `params:transformer_max_len`: si la longitud de la canasta excede el umbral, se preservan estrictamente las últimas interacciones; de lo contrario, se inyectan ceros correlativos (0) asociados al token <PAD>. De manera paralela, el nodo interroga a `scores_dict` para construir una matriz tensorial paralela de floats bidimensionales de tamaño (`max_len, 3`) que fusiona las afinidades de cada artículo de la canasta, retornando en cada iteración los tensores listos para el entrenamiento:

$$\mathcal{B} = \left\{ \mathbf{X}_{\text{ids}} \in \mathbb{Z}^{\text{max_len}}, \mathbf{X}_{\text{scores}} \in \mathbb{R}^{\text{max_len} \times 3}, \mathbf{Y}_{\text{target}} \in \mathbb{Z} \right\} \quad (12)$$

Modelo El pipeline `model_transformer` encapsula la declaración formal, la optimización paramétrica y el bucle autorregresivo profundo de la red neuronal mediante dos nodos de ejecución:

- **build_model** (`build_model`): Declara e inicializa la arquitectura híbrida `EarlyFusionTransformer`. La red implementa una estrategia de fusión analítica en etapas tempranas dividiendo la dimensión oculta total del sistema (d_{model}) en dos bloques simétricos independientes de mitad de dimensionalidad ($d_{\text{half}} = d_{\text{model}}/2$). Para cada secuencia, el ID entero del artículo es proyectado a un espacio vectorial denso a través de una capa de embeddings clásica (`nn.Embedding`):

$$\mathbf{e}_{\text{id}} = \text{Embedding}(\mathbf{X}_{\text{ids}}) \in \mathbb{R}^{\text{max_len} \times d_{\text{half}}} \quad (13)$$

En paralelo, el tensor multivariado de scores proveniente de las tres técnicas previas ingresa a un bloque secuencial denso (`nn.Sequential`) integrado por una transformación lineal, una activación de rectificación lineal (`ReLU`) y una proyección de salida:

$$\mathbf{e}_{\text{score}} = \mathbf{W}_2 \cdot \max(0, \mathbf{W}_1 \cdot \mathbf{X}_{\text{scores}} + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^{\text{max_len} \times d_{\text{half}}} \quad (14)$$

Ambas representaciones latentes se acoplan en el plano final de características mediante una operación de concatenación sobre el último eje dimensional ($\mathbf{x} = [\mathbf{e}_{\text{id}}; \mathbf{e}_{\text{score}}] \in \mathbb{R}^{\text{max_len} \times d_{\text{model}}}$). El nodo inyecta la noción de orden ordinal sumando un embedding posicional entrenable (`pos_embedding`) sobre los vectores fusionados, y transfiere el tensor resultante al codificador Transformer troncal (`nn.TransformerEncoder`), el cual está parametrizado con múltiples cabezas de atención multifactorial (`nhead`) y capas repetitivas (`num_layers`) configuradas en modo `batch_first=True`. La clasificación y predicción probabilística sobre el vocabulario de productos se ejecuta aislando el vector de activación del último paso temporal e introduciéndolo a una capa lineal final (`fc_out`):

$$\mathbf{z}_{\text{logits}} = \mathbf{W}_{\text{out}} \cdot \mathbf{x}_{[\text{último_token}, :]} + \mathbf{b}_{\text{out}} \in \mathbb{R}^{\text{vocab_size}} \quad (15)$$

- **train_transformer** (`train_transformer`): Gobierna el bucle de optimización estocástica y gestiona el registro analítico de métricas. El nodo inicializa la función de costo mediante la entropía cruzada estándar (`nn.CrossEntropyLoss`) y propaga los gradientes aplicando el optimizador **Adam** parametrizado bajo la tasa de aprendizaje del sistema (`params:transformer_learning_rate`).

En cada época, tras concluir el paso de entrenamiento, se invoca de manera interna la función relacional `evaluate_transformer` sobre el cargador `val_loader` en un entorno libre de gradientes (`torch.no_grad()`). Esta sub-rutina extrae la distribución de probabilidad mediante un operador `torch.softmax` y calcula las métricas de acierto posicional *Hit@1*, *Hit@5* y *Hit@10* analizando las fronteras máximas de decisión de los elementos vía `torch.topk`. Adicionalmente, evalúa la precisión del ordenamiento calculando el Rango Recíproco Medio (*MRR*):

$$\text{MRR} = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{i=1}^{|\mathcal{D}_{\text{val}}|} \frac{1}{\text{rango}_i} \quad (16)$$

Donde rango_i corresponde a la posición fáctica indexada del artículo objetivo real dentro de la ordenación predictiva del modelo (asignando un valor nulo si no se localiza). El pipeline realiza un seguimiento del desempeño histórico, almacena las colecciones temporales e instrumenta la persistencia del modelo óptimo (`transformer_model_trained`) bajo el control de versiones de **MLflow**, tomando como criterio de selección el pico máximo del estimador `best_mrr`.

Puntaje El pipeline de inferencia operativa `score_transformer` instrumenta el motor de recomendación predictivo en tiempo real para evaluar las transacciones activas de los usuarios en base a una canasta parcial empleando un único nodo especializado:

- **score_transformer_node** (`recomendar_siguiente_producto`): Ejecuta el paso de inferencia hacia adelante (*forward pass*) a partir de los artículos declarados en curso por el cliente (`params:transformer_input_basket`) utilizando la instancia entrenada del transformador. El nodo inicializa la red en modo de evaluación (`model.eval()`) y construye el diccionario inverso de índices para decodificar las salidas del vocabulario. Codifica vectorialmente los elementos de entrada aplicando una restricción de truncamiento o relleno homóloga por la izquierda mediante ceros sintéticos ajustada al límite temporal de `params:transformer_max_len`. Adicionalmente, mapea las señales asociativas multivariadas interrogando la estructura compacta `scores_dict` para generar los tensores paralelos correspondientes.

Dentro de un contexto libre de cálculo de gradientes (`torch.no_grad()`), el nodo transfiere los tensores indexados al hardware configurado en `params:transformer_device`, computa

los logits agregados y proyecta la distribución estricta de verosimilitud de *cross-selling* mediante una capa `torch.softmax`:

$$P(p \mid B_{\text{parcial}}) = \frac{\exp(\mathbf{z}_{\text{logits},p})}{\sum_j \exp(\mathbf{z}_{\text{logits},j})} \quad (17)$$

Aísla las mejores candidaturas delimitadas por el hiperparámetro de corte `params:top_k` extrayendo tanto los estimadores probabilísticos como sus índices discretos a través de `torch.topk`.

Para robustecer el análisis comparativo del ordenamiento, define un indicador de Score Relativo ponderando de forma proporcional el acierto de cada candidato candidato contra el artículo líder de máxima probabilidad (P_{base}):

$$\text{Score Relativo}(p) = \frac{P(p \mid B_{\text{parcial}})}{P_{\text{base}}} \quad (18)$$

De igual manera, estima analíticamente la ventaja competitiva inter-producto localizando la probabilidad del elemento inmediatamente subsecuente (P_{i+1}) para computar el grado de dominancia o factor de superioridad en la frontera de decisión:

$$\text{Ventaja}(p_i) = \frac{P(p_i \mid B_{\text{parcial}})}{P(p_{i+1} \mid B_{\text{parcial}}) + 10^{-9}} \quad (19)$$

Finalmente, el nodo ejecuta un acoplamiento asociativo con el catálogo relacional estructurado en `catalogo_dict` indexando los SKUs decodificados reales (`cve_art`) para extraer dinámicamente sus atributos comerciales de descripción (`des_art`) y precio monetario neto (`precio_con_imp`), exportando los vectores tabulares finales ordenados en un *DataFrame* de Pandas (`score_transformer`) que expone de forma directa el Top-*K* de recomendaciones del MVP.

Nota: Para la sección de transformers, es necesario ejecutar el siguiente comando (`kedro run -p transformer_training`), ejecuta los pipelines en conjunto hasta el (`model_transformer`), ya que hay variables temporales necesarias y serían impráctico guardarlas con el catálogo de Kedro.

7.4. Seguimiento de Modelos

Para garantizar la gobernanza, reproducibilidad y auditoría analítica de los modelos desarrollados, se implementó un marco sistemático de seguimiento de experimentos que integra **MLflow** como registro centralizado de artefactos y parámetros, y **Optuna** como motor automatizado de optimización de hiperparámetros (HPO). Esta infraestructura fue instrumentada de manera nativa dentro del ecosistema de **Kedro**, abstrayendo el acoplamiento directo en el código mediante abstracciones de datos configurables y la activación de mecanismos de telemetría automática a través de la directiva `mlflow.pytorch.autolog()`.

Nota: Hubo divergencias al inicio y durante de las dependencias de empezando con MLflow 2.X, el proyecto se termino desarrollando en Kedro 1.3.1 y MLflow 3.13.0. Al intentar migrarlo adecuadamente, esto también provoco no poder recapturar experimentos anteriores.

7.4.1. Estrategia de MLOps con Kedro y MLflow

El versionamiento, registro y persistencia de las ejecuciones se estructuró de forma desacoplada aprovechando la arquitectura de componentes de Kedro. En lugar de inyectar llamadas imperativas complejas dentro del bucle de entrenamiento, los pipelines se conectan a componentes catalogados de MLflow (*MLflow Datasets*) o exponen sus métricas e hiperparámetros de manera síncrona por época al servidor de seguimiento.

Al finalizar los nodos de entrenamiento, el sistema exporta de manera determinista las entidades clave hacia el servidor de control bajo dos enfoques homólogos basados en la arquitectura evaluada:

- **Pipeline de Transformers (`train_transformer`):** Exporta la serialización del estado binario óptimo de la red (*state dict* del objeto `EarlyFusionTransformer`), el indicador `best_mrr` como métrica de control de calidad y el `training_history` como una colección tabular indexada que almacena las funciones de pérdida de entrenamiento y validación (*Loss Trajectories*), así como la evolución posicional de los estimadores *Hit@1*, *Hit@5* y *Hit@10*.
- **Pipeline de Redes Recurrentes (`train_rnn_model`):** Instrumenta un modelo basado en compuertas GRU (`BasketGRU`) optimizado mediante una política de tasa de aprendizaje dinámica `OneCycleLR`. El nodo registra de forma nativa los hiperparámetros estructurales del vocabulario (`vocab_size`, `embed_dim`, `hidden_dim`) y las trazas temporales de desempeño por época, aplicando una política de parada temprana (*early stopping*) condicionada por la convergencia de la pérdida de validación (`best_val_loss`).

Este enfoque unificado centraliza en un único panel comparativo el desempeño global del MVP, contrastando el comportamiento de las arquitecturas recurrentes y de atención bajo las mismas condiciones de auditoría.

7.4.2. Optimización y Persistencia de Hiperparámetros con Optuna

El proceso de exploración y refinamiento del espacio de búsqueda no lineal se automatizó mediante **Optuna**. Cada configuración paramétrica evaluada se define formalmente como un *trial*, el cual explora estocásticamente las fronteras dimensionales de las arquitecturas, tales como la tasa de aprendizaje (η), el tamaño de lote ($\mathcal{B}$), la dimensión latente del embedding (d_{model} o `embed_dim`), el número de capas recurrentes o de atención, y el límite de holgura temporal (`max_len`).

Para robustecer este proceso frente a interrupciones computacionales y permitir una auditoría post-hoc, las ejecuciones distribuidas se configuraron empleando un almacenamiento persistente basado en un motor relacional local:

$$\mathcal{S}_{\text{storage}} = \text{sqlite:///optuna_study.db} \quad (20)$$

La persistencia bajo **SQLite** mitiga el riesgo de pérdida de estado transaccional entre sesiones independientes y facilita el análisis comparativo mediante queries estructuradas. Al finalizar la optimización, las configuraciones candidatas son ordenadas jerárquicamente en función de la maximización estricta del Rango Recíproco Medio de validación ($\text{máx MRR}_{\text{val}}$), aislando los parámetros óptimos que posteriormente gobiernan la instanciación de los modelos definitivos en el entorno productivo de MLflow.

8. Conclusiones y Recomendaciones

8.1. Resultados

La solución final integró el modelo de recomendación con una aplicación web funcional llamada **Canasta**, diseñada como una plataforma de comercio electrónico de abarrotes. Esta interfaz permitió demostrar el caso de uso ANA-03 en un entorno cercano a operación real: un cliente puede navegar productos, construir una canasta, finalizar una compra y, en paralelo, un colaborador puede consultar indicadores, auditoría y recomendaciones generadas por el modelo.

El resultado principal de esta etapa fue cerrar el ciclo completo entre datos, modelo e interfaz: el modelo no quedó únicamente como un artefacto entrenado en notebooks, sino como un servicio consumible desde una aplicación web. De esta manera, el proyecto pasó de una solución analítica a un prototipo operativo capaz de mostrar cómo se utilizarían las recomendaciones dentro de un flujo comercial.

8.1.1. Interfaz de usuario y despliegue

La capa de usuario se materializó en **Canasta**, una aplicación web de comercio electrónico que funciona como interfaz de consumo del sistema de recomendación. El *frontend* fue desarrollado como una aplicación de página única con HTML y JavaScript, servida desde el propio backend de ASP.NET Core. Esta decisión permitió mantener una solución ligera, fácil de ejecutar localmente y sin requerir un proceso adicional de compilación de frontend.

La experiencia del cliente incluye un catálogo de productos, búsqueda, filtrado por categoría, paginación, carrito de compra, cálculo de envío, flujo de pago en tres pasos y persistencia local de datos de usuario como direcciones y métodos de pago. Aunque esta capa no sustituye a un sistema de comercio electrónico productivo, sí reproduce un flujo de compra suficientemente realista para probar el recomendador en contexto.

La arquitectura funcional se organizó en tres capas:

Capa	Tecnología	Responsabilidad principal
Presentación	HTML + JavaScript	Experiencia de cliente, catálogo, carrito, flujo de compra y panel colaborador.
Backend web	ASP.NET Core (.NET)	Autenticación de demostración, registro de compras, auditoría y proxy hacia el recomendador.
Modelo	FastAPI en Azure Container Apps	Servicio de inferencia que recibe canastas y devuelve recomendaciones Top-10.

Tabla 36: Arquitectura funcional de la aplicación Canasta.

Uno de los elementos más importantes de integración fue el uso de un *proxy* en el backend. En lugar de que el navegador consulte directamente el endpoint del modelo, la aplicación envía las solicitudes al endpoint interno `/api/recomendaciones`; posteriormente, el backend reenvía la petición al servicio FastAPI desplegado en Azure Container Apps. Esta decisión permitió centralizar el consumo del modelo, controlar errores desde el servidor y evitar problemas de CORS en el navegador.

El panel del colaborador expone el recomendador de forma interactiva. A partir de una lista de entre una y diez claves de artículo, el sistema devuelve un ranking de productos sugeridos con clave, descripción, precio, *score* relativo y diferencia frente al siguiente candidato. Esta salida permite que un usuario de negocio inspeccione las recomendaciones sin necesidad de consultar directamente la API ni ejecutar código.

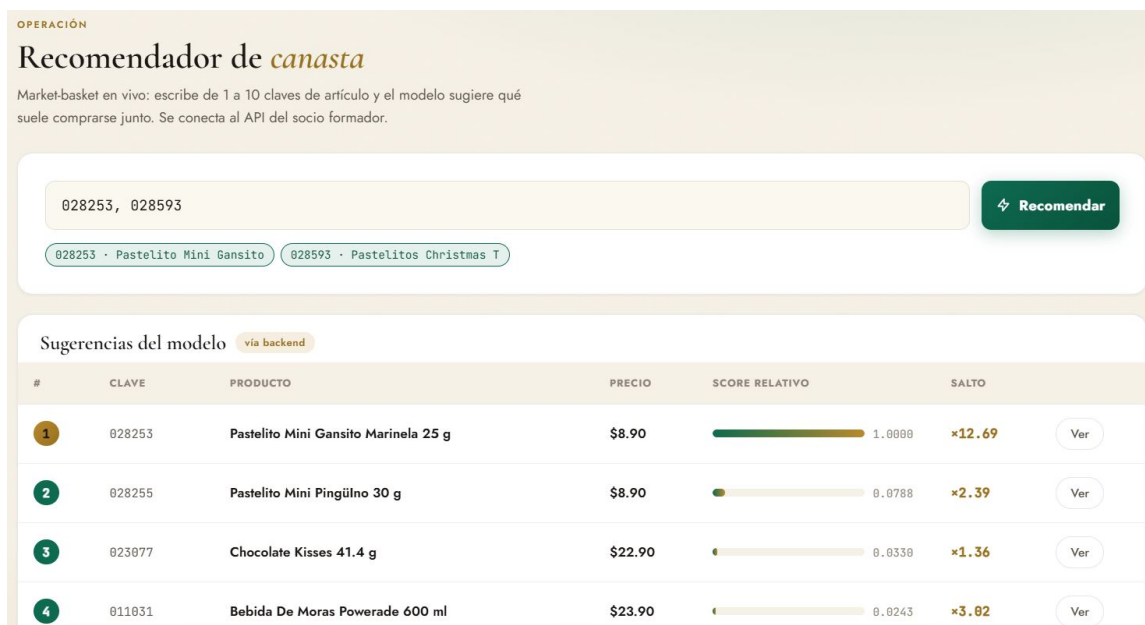


Figura 3: Módulo de recomendación en vivo. A partir de claves de artículo, el sistema devuelve un ranking de productos relacionados con su *score* relativo y el salto frente al siguiente candidato.

Además del recomendador, el backend registra compras finalizadas y mantiene una bitácora de auditoría. La persistencia se implementó mediante archivos planos JSON y CSV, lo cual resulta adecuado para el alcance académico del proyecto porque facilita la revisión manual de registros, la trazabilidad básica y la ejecución local sin depender de un motor de base de datos externo.

8.1.2. Resultados

La integración de las capas permitió construir un prototipo funcional de extremo a extremo. Desde la perspectiva del cliente, la aplicación permite explorar el catálogo, seleccionar productos, agregarlos al carrito y completar una compra. Desde la perspectiva del colaborador, el sistema permite consultar indicadores de ventas, productos destacados, segmentación de clientes, zonas de cobertura, registros de auditoría y recomendaciones generadas por el modelo.

El panel de inteligencia de negocio consolida la información operativa en una interfaz única. Entre los elementos visualizados se encuentran indicadores de ventas y ganancia estimada, ticket promedio, productos más vendidos, ventas por categoría, ventas por ciudad y comportamiento de compra por día y franja horaria. Estos elementos permiten interpretar la actividad simulada o registrada por la aplicación y ofrecen una base para analizar oportunidades comerciales.

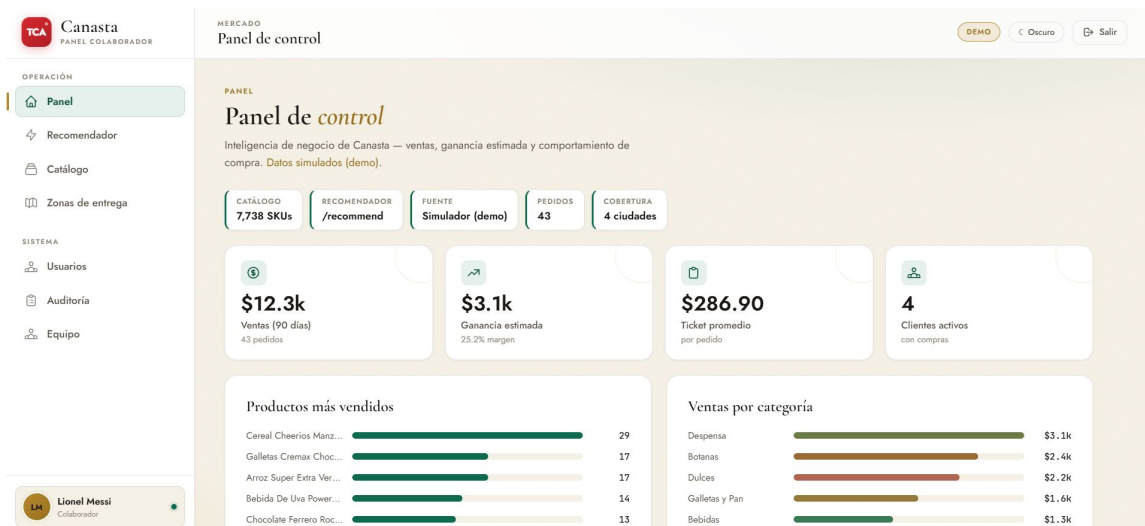


Figura 4: Panel de inteligencia de negocio del rol colaborador. Consolida indicadores clave, productos más vendidos, desglose por categoría y comportamiento de compra por día y franja horaria.

El resultado más relevante para el caso ANA-03 es que las recomendaciones dejaron de ser una salida aislada del modelo y se integraron en una interfaz de decisión. El colaborador puede probar canastas específicas, observar el ranking sugerido y evaluar si los productos propuestos son interpretables desde el punto de vista comercial. Esto facilita la validación cualitativa del modelo y aproxima el sistema a un flujo real de uso en punto de venta o comercio electrónico.

También se incorporó un módulo de cobertura logística basado en zonas de entrega. Este módulo utiliza una representación tipo Voronoi para visualizar áreas atendidas por distintos centros o ciudades, lo que permite complementar la experiencia de compra con una lectura geográfica de cobertura. Aunque este componente no forma parte directa del modelo de recomendación, aporta contexto operativo al prototipo y muestra cómo la solución podría convivir con decisiones de distribución, entrega y atención al cliente.

En conjunto, los resultados muestran que el proyecto logró integrar cuatro niveles de salida:

- un modelo de recomendación entrenado y evaluado con métricas de ranking;
- una API REST desplegada en la nube para exponer la inferencia;
- una aplicación web que consume el modelo mediante un backend intermedio;
- un panel colaborador que permite interpretar recomendaciones e indicadores operativos.

Esta integración confirma que la solución no se limita a la generación de *scores* o al entrenamiento de un modelo, sino que demuestra cómo el recomendador puede insertarse dentro de una experiencia de negocio accionable.

8.1.3. Ejecución del servicio

La aplicación web se ejecuta desde la solución de Visual Studio incluida en la carpeta *RecepcionFacturasWeb*. El despliegue local está orientado a Windows, utilizando Visual Studio 2022 o superior, el SDK de .NET correspondiente y conexión a internet para consumir el recomendador en la nube y los recursos de mapa en tiempo real.

El flujo de ejecución local es el siguiente:

1. abrir la solución del proyecto en Visual Studio;

2. revisar el archivo `appsettings.json`, especialmente la ruta de salida del CSV y la URL base del recomendador;
3. restaurar paquetes y compilar la solución;
4. ejecutar con `F5`;
5. iniciar sesión con una cuenta de demostración para probar el flujo de cliente o el panel colaborador.

Al ejecutarse, el backend sirve el frontend como contenido estático y habilita los endpoints internos utilizados por la aplicación. Entre ellos se encuentran:

Endpoint interno	Función
<code>/api/auth/login</code>	Autenticación de demostración mediante usuario y contraseña.
<code>/api/compras</code>	Registro y consulta de compras finalizadas.
<code>/api/recomendaciones</code>	Proxy hacia el modelo de recomendación desplegado en Azure.
<code>/api/auditoria</code>	Consulta de bitácora de actividad para el rol colaborador.

Tabla 37: Endpoints principales del backend de Canasta.

El servicio de recomendación se ejecuta de forma independiente como API FastAPI. En entorno local puede levantarse mediante Docker con los comandos:

```
docker build -t reco-api .
docker run -p 8000:8000 reco-api
```

Una vez activo, el servicio queda disponible en `http://localhost:8000/view` para inspección visual y en `http://localhost:8000/docs` para documentación interactiva. En la nube, el servicio fue publicado en Azure Container Apps y expone los mismos recursos principales: `/view`, `/docs` y `/recommend`.

Es importante considerar que, al utilizar Azure Container Apps con escalamiento a cero, la primera solicitud después de un periodo de inactividad puede experimentar *cold start*. Este comportamiento no impide la operación del prototipo, pero debe tomarse en cuenta para una versión productiva. En un escenario comercial, podría mitigarse mediante una instancia mínima activa, monitoreo de disponibilidad o llamadas programadas de calentamiento.

Finalmente, la solución contempla degradación funcional. Si no existe conexión a internet o el servicio externo no está disponible, la aplicación conserva elementos demostrativos como datos simulados y visualizaciones locales. Esto permitió mantener la capacidad de demostración del frontend aun cuando el recomendador o los mapas externos no estuvieran disponibles.

8.2. Trabajo a futuro

Aunque el proyecto logró entregar un MVP funcional compuesto por un modelo de recomendación, una API desplegada en la nube y una aplicación web integrada, todavía existen áreas de mejora que podrían desarrollarse en una siguiente etapa con mayor tiempo, recursos y acceso al entorno productivo de la empresa.

En primer lugar, la aplicación web entregada debe entenderse como un prototipo funcional y no como una versión final lista para operación comercial. El MVP permite demostrar el flujo de compra, el consumo del recomendador, el panel colaborador y la visualización de indicadores; sin embargo, todavía contiene algunos *placeholders* visuales y elementos de interfaz que deberían sustituirse por contenido definitivo de la empresa. En particular, una mejora relevante sería incorporar imágenes reales de los productos. Dado el tamaño del catálogo y la cantidad de SKUs disponibles, completar manualmente imágenes para todos los artículos era un trabajo de

gran escala que excedía el periodo disponible para el proyecto. En una fase futura, esta tarea podría resolverse mediante integración con el catálogo visual de TCA Merksyst, proveedores de imágenes, almacenamiento en la nube o procesos automatizados de enriquecimiento de catálogo.

En segundo lugar, el vocabulario del modelo final se restringió a aproximadamente 3,000 SKUs. Esta decisión permitió entrenar y desplegar una solución funcional dentro de las restricciones de tiempo, cómputo, orquestación y manejo de artefactos del proyecto. No obstante, la empresa cuenta con un catálogo más amplio de productos, por lo que una línea natural de evolución es ampliar el modelo para cubrir la totalidad de los SKUs relevantes. Para ello sería necesario invertir en mayor capacidad de procesamiento, optimización del pipeline de entrenamiento, mejor manejo de memoria, estrategias de particionamiento y mecanismos más robustos para gestionar productos de baja frecuencia o reciente incorporación.

En tercer lugar, sería necesario fortalecer la compatibilidad técnica y operativa con los servicios existentes de la empresa. Aunque la solución fue diseñada como una API REST desacoplada y potencialmente integrable con el ERP, una implementación productiva requeriría validar formatos de entrada y salida, políticas de autenticación, tiempos de respuesta aceptables, manejo de errores, permisos, seguridad de red y compatibilidad con los flujos reales de punto de venta. También sería importante definir si el recomendador se consumirá desde el ERP, desde una pantalla de vendedor, desde un comercio electrónico o desde otros módulos operativos de TCA Merksyst.

En cuarto lugar, el sistema debe evolucionar hacia un esquema de actualización continua. En esta versión, el modelo opera con artefactos entrenados y empaquetados previamente. En una etapa futura, sería conveniente automatizar la extracción periódica de datos, el recálculo de *scores*, el reentrenamiento del modelo, la validación de métricas y la publicación controlada de nuevas versiones. Esto permitiría que el recomendador se adapte a cambios en demanda, temporadas, inventario, promociones y composición del catálogo.

En quinto lugar, el impacto comercial debe validarse mediante pruebas en operación real. Las métricas reportadas en el proyecto, como MRR@10 y Hit@K, permiten evaluar el desempeño offline del modelo, pero no sustituyen una medición de conversión en tienda. Como siguiente paso, se recomienda diseñar una prueba A/B en la que se comparen tickets con y sin sugerencias del recomendador, midiendo adopción por parte del vendedor, aceptación del cliente, incremento en artículos por ticket, variación del ticket promedio y disponibilidad de los productos sugeridos.

Finalmente, el sistema podría enriquecerse con reglas de negocio adicionales. Por ejemplo, se podrían priorizar productos con inventario disponible, productos con mayor margen, artículos en promoción, productos de temporada o categorías estratégicas para la empresa. Estas reglas no deberían reemplazar al modelo, sino actuar como una capa complementaria de negocio que permita adaptar las recomendaciones a objetivos comerciales específicos.

8.3. Conclusiones finales del proyecto

El proyecto ANA-03 permitió desarrollar una solución integral de ciencia de datos orientada a resolver una oportunidad real de negocio: transformar la venta reactiva en una venta asistida por recomendaciones inteligentes. A partir de datos transaccionales del ERP TCA Merksyst, se construyó un sistema capaz de analizar canastas históricas, generar señales complementarias por producto y entregar recomendaciones de artículos que podrían agregarse a una compra en curso.

Desde la perspectiva de datos, el proyecto demostró que las tablas operativas del ERP contienen información suficiente para modelar afinidad de canasta. La reconstrucción de tickets, la limpieza de ventas válidas, el enriquecimiento con catálogo y la integración de inventario permitieron pasar de registros transaccionales a datasets analíticos útiles para modelado. El análisis exploratorio confirmó que existe volumen suficiente de transacciones, diversidad de productos y profundidad de canasta para justificar una solución de recomendación.

Desde la perspectiva de modelado, la solución integró tres señales complementarias: TDA

para capturar estructura del catálogo, Series de Tiempo para medir oportunidad temporal y RNN para aprender patrones secuenciales de canasta. Estas señales se combinaron dentro de un Transformer de fusión temprana, permitiendo que el modelo aprendiera recomendaciones a partir de identidad de producto, contexto de canasta y características externas previamente calculadas. El modelo final fue refinado con Optuna y evaluado con métricas de ranking adecuadas para el problema, como MRR@10 y Hit@K.

Desde la perspectiva de implementación, el proyecto no se limitó al entrenamiento del modelo. La solución fue empaquetada como una API REST con FastAPI, contenedorizada con Docker y desplegada en Azure Container Apps. Además, se desarrolló una aplicación web funcional que consume el recomendador y permite visualizar su uso dentro de un flujo comercial. Esta integración permitió demostrar el ciclo completo entre datos, modelo, servicio e interfaz de usuario.

El principal valor del proyecto está en haber construido un MVP funcional que conecta analítica avanzada con una necesidad operativa concreta. La solución muestra cómo TCA Merksyst podría incorporar recomendaciones inteligentes dentro de sus servicios para ayudar a vendedores, colaboradores o plataformas digitales a sugerir productos complementarios de manera más sistemática y basada en evidencia histórica.

En conclusión, el proyecto representa una base técnica sólida para evolucionar hacia un sistema productivo de recomendación. Aunque todavía requiere mejoras en cobertura total de SKUs, enriquecimiento visual del catálogo, compatibilidad con servicios internos, automatización de actualizaciones y validación comercial en campo, el MVP entregado demuestra la viabilidad técnica, analítica y operativa de incorporar un motor de recomendación de canasta dentro del ecosistema TCA Merksyst.

A. Apéndices

A.1. Liga de Repositorio

El código del proyecto se encuentra disponible en el repositorio de GitHub:

https://github.com/DiegoA-ML/basket_recommender

En caso de no poder acceder favor de contactar cualquiera de los correos TEC o EXATEC de las matrículas de los alumnos listados en la portada de este reporte (matrícula + @tec.mx o @exatec.tec.mx).

El repositorio está organizado por módulos funcionales. Cada carpeta concentra una parte específica del proyecto: servicio de inferencia, aplicación web, notebooks de entrenamiento, componentes individuales de modelado, reportes y artefactos finales. Esta estructura permite que cada componente pueda consultarse, ejecutarse o documentarse de forma independiente.

La estructura actual del repositorio es la siguiente:

```
basket_recommender/
|-- README.md          Documentacion general del repositorio global
|-- LICENSE            Terminos de licencia de software del proyecto
|-- gitignore          Archivos y extensiones ignorados por Git
|
|-- API/              Servicio de inferencia desplegable
|   |-- app.py         Aplicacion FastAPI para el servicio
|   |-- Dockerfile     Imagen Docker del servicio
|   |-- requirements.txt Dependencias de Python de la API
|   |-- readme.md      Documentacion de la API
|   |-- Guide_API_Into_AmazonMockUp.ipynb Guia de integracion y prueba
|   |-- transformer_fusion.pt Pesos del modelo Transformer
|   |-- transformer_vocab.pkl Vocabulario SKU-indice
|   |-- catalogo.pkl   Catalogo serializado
|   |-- scores_dict.pkl Scores RNN / TDA / TS por producto
|
|-- Executive Presentation/ Presentacion ejecutable para stakeholders
|   |-- ANA-03_TCA_Eq2.pdf Archivo de la presentacion ejecutiva
|
|-- Front-end/        Aplicacion web Canasta
|   |-- readme.md      Documentacion y video del frontend
|   |-- catalogo.pkl   Catalogo serializado para la interfaz
|   |-- vocabulario.pkl Vocabulario serializado
|   |
|   |-- RecepcionFacturasWeb/ Proyecto ASP.NET Core
|       |-- Controllers/ Controladores del backend
|       |-- Models/      Modelos de datos
|       |-- Properties/  Configuracion del proyecto
|       |-- Services/    Servicios internos
|       |-- wwwroot/     Archivos estaticos del frontend
|       |-- bin/         Artefactos generados de compilacion
|       |-- obj/         Artefactos generados de compilacion
|       |-- AuthController.cs Controlador de autenticacion
|       |-- EmailService.cs Servicio auxiliar de correo
|       |-- Program.cs   Punto de entrada de la aplicacion
|       |-- RecepcionFacturasWeb.csproj Proyecto .NET
|       |-- RecepcionFacturasWeb.csproj.user Configuracion local de usuario
|       |-- RecepcionFacturasWeb.http Pruebas HTTP del backend
|       |-- RecepcionFacturasWeb.slnx Solucion de Visual Studio
|       |-- appsettings.json Configuracion general
|       |-- appsettings.Development.json Configuracion de desarrollo
|       |-- Catalogo front.json Catalogo utilizado por la vista
|       |-- catalogo.json Catalogo de productos
|       |-- Combos.json  Datos de combos
```

		-- Destacados.json	Productos destacados
		-- usuarios.json	Usuarios de demostracion
	--	Kedro/	Gestion de pipelines de datos con Kedro
		-- README.md	Manual explicativo del componente
		-- KedroRepo.md	Enlace al repositorio externo de Kedro
	--	RNN/	Componente secuencial RNN/GRU
		-- README.md	Documentacion del modulo RNN
		-- requirements.txt	Dependencias de Python para RNN
		-- docs/	Documentos del modulo
		-- DOCUMENTACION_RNN.md	Archivo de documentacion de la RNN
		-- notebooks/	Cuadernos de trabajo de RNN
		-- RNN_LSTM_BasketRecommender_FINAL.ipynb	Cuaderno de desarrollo del modelo RNN
		-- outputs_rnn/	Resultados y salidas del componente RNN
		-- rnn_scores.csv	Archivo de scores de RNN
		-- curvas_entrenamiento_gru.png	Imagen de curvas de entrenamiento de GRU
		-- distribucion_canastas.png	Imagen de distribucion de canastas
		-- distribucion_score_rnn.png	Imagen de distribucion de scores de RNN
		-- embeddings_pca_gru.png	Imagen de embeddings PCA de GRU
	--	TDA/	Componente de Analisis Topologico de Datos
		-- README.md	Documentacion del modulo TDA
		-- requirements.txt	Dependencias de Python para TDA
		-- notebooks/	Cuadernos de trabajo de TDA
		-- TDA_Mapper_Documentado.ipynb	Cuaderno del algoritmo Mapper documentado
	--	Reports/	Reportes finales del proyecto
		-- Technical Report.pdf	Archivo del reporte tecnico
		-- Executive Report.pdf	Archivo del reporte ejecutivo
	--	Time Series/	Componente temporal score_TS
		-- README.md	Documentacion del modulo de series de tiempo
		-- requirements.txt	Dependencias de Python para series de tiempo
		-- .gitignore	Archivo gitignore especifico del modulo
		-- data/	Carpeta de datos del modulo
		-- README_data.md	Documentacion de los datos de series de tiempo
		-- docs/	Documentos del modulo
		-- Documentacion_series_tiempo_ANA03.docx	Documento de texto de la documentacion
		-- notebooks/	Cuadernos de trabajo de series de tiempo
		-- ANA03_series_tiempo_recomendador.ipynb	Cuaderno de desarrollo de series de tiempo
		-- outputs_ana03/	Resultados y salidas de series de tiempo
		-- weekly_series_ana03.csv	Archivo de series semanales ANA03
		-- ts_scores_ana03.csv	Archivo de scores de series de tiempo ANA03
		-- statistical_tests_by_sku.csv	Archivo de pruebas estadisticas por SKU
		-- statistical_tests_summary.csv	Archivo de resumen de pruebas estadisticas
		-- pie_estacionariedad_series.png	Imagen de estacionariedad de las series
		-- pie_ciclicidad_series.png	Imagen de ciclicidad de las series
		-- bar_tipo_ciclo_series.png	Imagen de tipo de ciclo de las series
	--	Transformer-fusion/	Modelo final Early Fusion Transformer
		-- README.md	Documentacion del modulo Transformer-fusion
		-- Data/	Datos del modulo Transformer-fusion
		-- README.md	Documentacion de la carpeta de datos
		-- catalogo.pkl	Archivo de catalogo serializado
		-- scores_dict.pkl	Archivo de diccionario de scores serializado
		-- transformer_vocab.pkl	Archivo de vocabulario del Transformer serializado
		-- Model/	Componentes del modelo Transformer
		-- README.md	Documentacion de la carpeta del modelo
		-- requirements.txt	Dependencias de Python para el modelo
		-- Operation_Manual.pdf	Archivo del manual de operacion

```

| | |-- Transformer_fusion_final.ipynb Cuaderno del modelo final Transformer-fusion
| | |-- transformer_fusion.pt Archivo de pesos del Transformer en PyTorch
| |
| |-- Model Optimization/ Optimizacion del modelo Transformer
| | |-- README.md Documentacion de la carpeta de optimizacion
| | |-- requirements.txt Dependencias de Python para optimizacion
| | |-- data/ Carpeta de datos de optimizacion
| | |-- notebooks/ Cuadernos de trabajo de optimizacion
| | |-- Optuna_optimized_model.ipynb Cuaderno del modelo optimizado con Optuna
| | |-- outputs_optuna/ Carpeta de salidas de Optuna
| | |-- outputs_optuna_rnn_item_scores/ Carpeta de salidas de scores de RNN en Optuna
| |
| |-- Recommendation Inference Function/ Funcion de inferencia de recomendaciones
| |-- README.md Documentacion de la funcion de inferencia
| |-- requirements.txt Dependencias de Python para la inferencia
| |-- Inference Function.ipynb Cuaderno de la funcion de inferencia

```

Nota de confidencialidad: por motivos de seguridad y confidencialidad empresarial, las bases de datos transaccionales crudas (`pdvhdr.parquet`, `pdvdet.parquet`, `inviar.parquet` e `invars.parquet`) no se incluyen en el repositorio. Para reproducir el procesamiento de datos o reentrenar los modelos desde cero es necesario solicitar y colocar localmente dichos archivos. Para probar únicamente la inferencia con el modelo ya optimizado basta con los artefactos serializados (`.pkl`) y los pesos del modelo (`.pt`).

A.2. Manual

Cada carpeta del repositorio incluye un archivo README con instrucciones detalladas a manera de manual del componente correspondiente. Esta sección resume el flujo completo de la solución, desde la generación de señales con Kedro hasta el despliegue del servicio y la ejecución de la aplicación web. Para detalles específicos de cada componente se recomienda consultar el README de su respectiva carpeta.

A.2.1. Flujo general de la solución

El flujo de extremo a extremo de la solución es el siguiente:

1. **Pipelines de señales (Kedro).** Los *pipelines* de TDA, Series de Tiempo y RNN procesan los datos transaccionales del ERP y generan un *score* por SKU cada uno.
2. **Integración (Transformer).** El *pipeline* Transformer une los tres *scores* por `cve_art`, entrena el `EarlyFusionTransformer` y produce los artefactos de inferencia.
3. **Empaquetado (API).** Los artefactos se copian a la carpeta `API/` y se construye la imagen Docker con FastAPI.
4. **Despliegue (Azure).** La imagen se publica en Azure Container Registry y se despliega en Azure Container Apps.
5. **Consumo (Front-end).** La aplicación web Canasta consume el servicio mediante un *proxy* de su *back-end*.

A.2.2. Ejecución de los pipelines (Kedro)

Se recomienda instalar las dependencias con `uv`. Debido a que el Transformer de fusión temprana depende de variables intermedias en memoria, los *pipelines* de TDA, Series de Tiempo y RNN deben ejecutarse antes que el integrador:

```
# Instalacion de dependencias
pip install uv
uv pip install -r requirements.txt

# Ejecucion integral hasta el entrenamiento del Transformer
kedro run -p transformer_training

# Los pipelines individuales se ejecutan por su nombre:
# kedro run -p tda
# kedro run -p time_series
# kedro run -p rnn
```

A.2.3. Ejecución local del API

El servicio se puede construir y ejecutar localmente con Docker Desktop desde la carpeta API/:

```
cd API
docker build -t reco-api .
docker run -p 8000:8000 reco-api
```

Una vez en ejecución, las superficies del servicio quedan disponibles en:

```
http://localhost:8000/view    # tabla visual de recomendaciones
http://localhost:8000/docs    # probador Swagger
```

La construcción de Docker ejecuta una prueba de humo que carga el modelo y realiza una recomendación; si el modelo no puede cargarse, la construcción falla de inmediato. Para ejecutar `app.py` fuera de Docker se requiere instalar primero la versión CPU de PyTorch:

```
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt
```

A.2.4. Consumo del API

El *endpoint* de integración es `POST /recommend`, con encabezado `Content-Type: application/json`. Recibe entre 1 y 10 códigos de producto como cadenas (el orden es relevante) y devuelve el Top-10 de recomendaciones:

```
curl -X POST \
  https://reco-basket-app.gentlesea-cb2510e3.eastus.azurecontainerapps.io/recommend \
  -H "Content-Type: application/json" \
  -d '{ "products": ["028253", "028593"] }'
```

El servicio tiene CORS habilitado y no requiere clave de API. La guía para uso general completa de integración para desarrolladores de *front-end* se encuentra en `API/readme.md` y en el *notebook* `API/Guide_API_Into_AmazonMockUp.ipynb`. La guía de cómo nosotros específicamente desarrollamos nuestro Front-End se encuentra en `Front-end/readme.md`

A.2.5. Despliegue en Azure

El despliegue se realiza publicando la imagen en Azure Container Registry y desplegándola en Azure Container Apps:

```
az acr login --name acrrecobasket
docker tag reco-api acrrecobasket.azurecr.io/reco-api:latest
docker push acrrecobasket.azurecr.io/reco-api:latest

az containerapp up \
```

```
--name reco-basket-app \  
--resource-group rg-reco-basket \  
--location eastus \  
--image acrrecobasket.azurecr.io/reco-api:latest \  
--ingress external \  
--target-port 8000 \  
--registry-server acrrecobasket.azurecr.io \  
--registry-username acrrecobasket \  
--registry-password <ACR_PASSWORD>
```

El servicio escala a cero en inactividad, por lo que la primera solicitud tras un periodo sin tráfico tarda aproximadamente 10–20 segundos (*cold start*).

A.2.6. Ejecución de la aplicación web (Front-end de Canasta)

La aplicación Canasta solo está soportada en Windows, ya que se ejecuta desde Visual Studio sobre el *runtime* de .NET. Requiere Windows 10 u 11, Visual Studio 2022 o superior con la carga de trabajo *ASP.NET y desarrollo web*, el SDK de .NET correspondiente y conexión a internet (para el recomendador en la nube y el mapa en tiempo real). Los pasos son:

1. Descomprimir el proyecto conservando la estructura de carpetas.
2. Abrir la solución en Visual Studio mediante el archivo `.sln`.
3. Revisar `appsettings.json`: la carpeta de salida del CSV (`CarpetaCsv`) y la URL del recomendador (`RecomendadorUrl`) deben corresponder al entorno.
4. Restaurar paquetes NuGet y compilar con `Ctrl+Shift+B`.
5. Ejecutar con F5; Visual Studio levanta el servidor local y abre el navegador (por defecto `https://localhost:7237`).
6. Iniciar sesión con una de las cuentas de demostración.

La configuración del *back-end* se realiza en `appsettings.json`, donde se indica la carpeta de escritura del CSV y la URL base del modelo:

```
{  
  "Canasta": {  
    "CarpetaCsv": "C:\\ruta\\donde\\se\\guarda\\compras.csv",  
    "RecomendadorUrl": "https://reco-basket-app.gentlesea-cb2510e3.eastus.azurecontainerapps.io"  
  }  
}
```

El detalle completo de la aplicación, cuentas de demostración, estructura del proyecto, *end-points* del *back-end*, funcionalidades del cliente y del panel del colaborador, así como notas y limitaciones, se encuentra documentado a manera de manual en `Front-end/readme.md`.